HYDRIS AI

HYDRIS FIELD GLASSES · PRODUCT BOOK

Hydris Field Glasses Product Book

Every product and sub-product we are building: the problem, the users, the flows, the requirements and where each one stands

DOCUMENT	VERSION	STATUS	DATE
HFG-BOOK	1.0	Draft for review	October 2026

Contents

1 Hydris Field Glasses at a glance 6
1.1 Executive summary 6
1.2 Vision and principles 6
1.3 The problem in wastewater plants 7
1.4 Personas and jobs 8
1.5 The product family 9
1.6 One account, every surface 10
1.7 The operator's day 10
1.8 The core loops 11
1.9 Status, gaps and the path to pilot 13
1.10 How to read this book 14
2 Website and demo 15
2.1 Purpose and problem 15
2.2 Users and jobs 15
2.3 Scope 15
2.4 Key flows 16
2.5 Requirements 17
2.6 Experience notes 18
2.7 Success metrics 18
2.8 Status, gaps and next steps 18
3 Operator console 19
3.1 Purpose, users and information architecture 19
3.2 Overview dashboard 21
3.3 Plant model: sites, assets, sensors and the plant map 22
3.4 Telemetry and sensor monitoring 23
3.5 Alarms 24
3.6 Inspections and maintenance 26
3.7 Documents and SOPs 27
3.8 Field glasses fleet 28
3.9 Audit documents: review and approval 30
3.10 Reports, insights and the assistant 31
3.11 Organization admin 31
3.12 Console success metrics and gaps 32
4 Glasses app (heads-up and phone) 34
4.1 Purpose and problem 34
4.2 Users and jobs 34
4.3 Scope 34
4.4 Runtimes 35
4.5 Screen map and navigation 36
4.6 Lifecycle and pairing 36

4.7 Core experience ... 38
4.8 UI profiles ... 39
4.9 Voice entry point ... 40
4.10 Requirements ... 41
4.11 Success metrics ... 42
4.12 Status, gaps and next steps ... 42
5 Hydris Field Android app ... 43
5.1 Purpose and problem ... 43
5.2 Users and jobs ... 43
5.3 Scope ... 43
5.4 Navigation ... 44
5.5 Session and sync ... 44
5.6 Home dashboard ... 46
5.7 Work ... 47
5.8 Glasses tab ... 49
5.9 Permissions ... 49
5.10 Performance and native principles ... 49
5.11 Requirements ... 50
5.12 Success metrics ... 50
5.13 Status, gaps and next steps ... 51
6 Ray-Ban Meta integration ... 52
6.1 Purpose and problem ... 52
6.2 Users and jobs ... 52
6.3 Scope ... 52
6.4 What the glasses can do ... 53
6.5 The Meta AI app dependency ... 53
6.6 DAT lifecycle ... 53
6.7 Two integration routes ... 55
6.8 Registering glasses as a Hydris device ... 56
6.9 Setup guide ... 57
6.10 Troubleshooting ... 58
6.11 Requirements ... 58
6.12 Success metrics ... 59
6.13 Limitations and publishing constraints ... 59
6.14 Hardware acceptance status ... 59
7 Audit document scanning ... 60
7.1 Purpose and problem ... 60
7.2 Users and jobs ... 60
7.3 Scope ... 60
7.4 From paper to verified PDF ... 60
7.5 Guided capture ... 62
7.6 Reading the page twice ... 63

7.7 Structuring and cross-checking ... 64
7.8 Accuracy design ... 65
7.9 Review, verify and approve ... 65
7.10 Requirements ... 66
7.11 Success metrics ... 66
7.12 Status, gaps and next steps ... 67
8 Hey Hydris voice assistant ... 68
8.1 Purpose and problem ... 68
8.2 Users and jobs ... 68
8.3 Scope ... 68
8.4 Wake word flow ... 68
8.5 Speech engines ... 69
8.6 Grounded answering ... 70
8.7 Guardrails ... 71
8.8 Requirements ... 72
8.9 Success metrics ... 72
8.10 Status, gaps and next steps ... 72
9 Platform: API, data and integrations ... 74
9.1 Architecture in one picture ... 74
9.2 The modular API ... 74
9.3 The request lifecycle ... 76
9.4 Identity, sessions and device tokens ... 76
9.5 Realtime events ... 77
9.6 The data model by domain ... 78
9.7 Tenancy and data isolation ... 80
9.8 Telemetry ingestion and integrations ... 81
9.9 AI services and the human in the loop ... 81
9.10 Background jobs ... 82
9.11 Key architecture decisions ... 82
9.12 Platform requirements and NFRs ... 83
9.13 The technical debt that matters ... 84
10 Security, delivery and operations ... 85
10.1 Security model ... 85
10.2 Compliance mapping ... 87
10.3 Deployment topologies ... 87
10.4 Quality ... 89
10.5 Operations ... 90
10.6 Known gaps register ... 92
11 Roadmap and future products ... 94
11.1 Purpose of the roadmap ... 94
11.2 Horizons: now, next, later ... 94
11.3 Timeline, Q4 2026 to Q4 2027 ... 95

11.4 Known gaps to close first 95
11.5 Future products in brief 96
11.6 Value versus effort 98
11.7 Success metrics per horizon 99
11.8 Release cadence 99

1 Hydris Field Glasses at a glance

1.1 Executive summary

Wastewater operators spend their shift walking the plant: reading instruments, checking pumps and blowers, sampling effluent, writing values on paper and responding to alarms that usually reach a control room before they reach the person standing next to the equipment. The records they leave behind are the evidence a plant shows its regulator, and much of that evidence is still on paper.

Hydris Field Glasses puts the live state of the plant, the next task and the approved procedure in the operator's line of sight, hands free, and turns every reading, check and paper record they touch into a record that can be trusted. It does this through a family of connected products that share one API, one tenant model and one permission matrix: a public website with a live demo, an operator console, a glasses heads-up app, the native Android app Hydris Field, a Ray-Ban Meta integration, audit scanning, a grounded voice assistant and the platform underneath.

5
product surfaces
website, console, glasses app, Android app, API

143
API routes
across 15 modules

67
automated tests passing
26 domain + 41 API, 22 Oct 2026

10
demo scenarios
deterministic, demo tenants only

6
roles
38 permissions, one fixed matrix

16
measurands
pH to residual chlorine

12
glasses screens
six-key directional input

42
database tables
row-level security on every tenant table

The platform core is **Built** and runs at `https://hydrisview.vercel.app` in demo mode, with named guest accounts on synthetic plant data. The main gaps before a customer pilot are validation of the glasses paths on physical Ray-Ban Meta hardware, email delivery for invitations and password resets, continuous integration, a scheduled-job cadence suited to real data on serverless hosting, and an offline outbox in the Android app. The plan sequences these into hardening in Q4 2026, a pilot at one or two Indian plants in Q1 2027 and general availability in Q2 2027 (plan, not commitment).

> **DECISION**
> Hydris is read-only towards the plant. It never writes to PLCs, SCADA or equipment. The grounded assistance endpoint states "Hydris does not control equipment" in every response, and every voice-assistant tool is read-only. This keeps Hydris outside the plant's control-system safety case.

1.2 Vision and principles

Vision. Every operator in a water or wastewater plant sees what matters, does the next right thing and leaves behind a record that stands up to an audit, without putting down their tools. In three years we want Hydris to be the default way an Indian ETP, STP or CETP runs its rounds, responds to alarms and keeps its compliance records.

Four principles carry most of the weight. When requirements conflict, they decide.

Table 1. The four principles that shape every Hydris product

Principle	What it means for users	How the product enforces it
Advisory only	Hydris informs and guides; people operate the plant	No write path to equipment; assistant tools are read-only; safety notice on every grounded answer
Hands-free	Every operator action works with six directional keys or voice; the camera runs only on intent	Glasses input vocabulary (up, down, left, right, select, back); "Hey Hydris"; camera only on scan screens

Principle	What it means for users	How the product enforces it
Grounded	A number is never a guess; every value has a source and an age	Freshness states (live, delayed, stale, no data); provenance on every reading; assistant answers only from tool results
Human verification	Machines propose, a person confirms, a different person approves	Audit document review and four-eyes approval; corrections append, originals stay visible

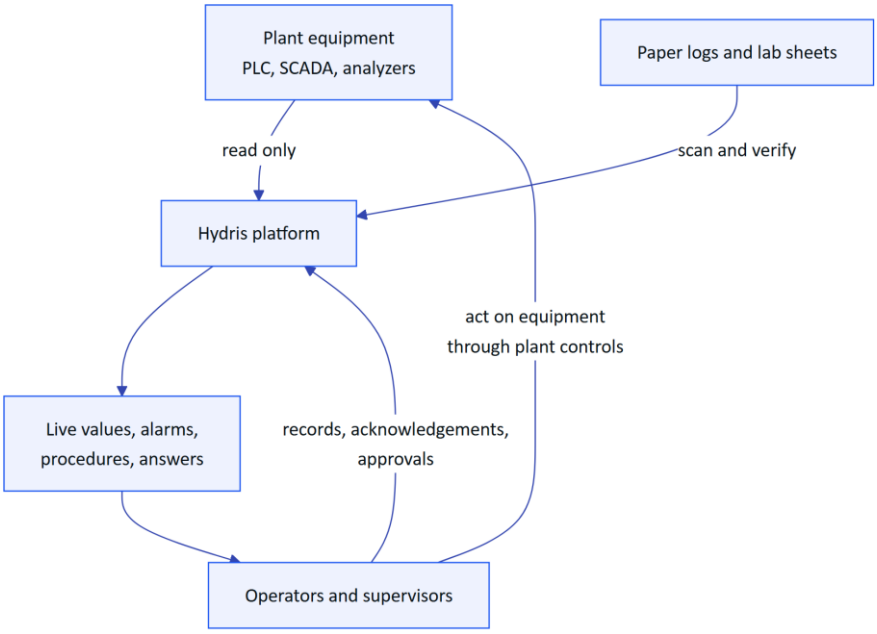


Figure 1. *The advisory boundary: Hydris reads from the plant and informs people, who stay in control*

Hydris sits beside the control system, never inside it. Data flows in from instruments and paper; decisions and actions stay with people, who use the plant's own controls.

1.3 The problem in wastewater plants

Effluent, sewage, water and common effluent treatment plants (ETPs, STPs, WTPs, CETPs) run continuously, with operators walking fixed rounds through intake, equalization, aeration, clarification, filtration, disinfection, sludge handling and the discharge point. Four problems recur:

1. **Paper rounds drift from reality.** Readings are written on clipboards and transcribed later. Nobody can tell which value was observed and which was copied.
2. **Alarms reach the wrong person.** The operator nearest the pump finds out last, and acknowledging an alarm gets confused with fixing it.
3. **Compliance evidence is unsearchable.** Logbooks, lab sheets and checklists are paper, so producing evidence for an inspection means digging through files.
4. **Phones are the wrong tool at the asset.** Gloves, ladders, wet hands and sampling bottles make phones awkward.

Indian compliance context. Discharges are regulated under the Water (Prevention and Control of Pollution) Act, 1974, through consents to establish and operate issued by State Pollution Control Boards (SPCBs) and Pollution Control Committees. General discharge standards are set under the Environment (Protection) Rules, 1986, with plant-specific limits in each consent. In 2014 the Central Pollution Control Board (CPCB) directed the 17 categories of highly polluting industries, and CETPs, to install online continuous effluent and emission monitoring systems (OCEMS) connected to CPCB and SPCB servers.

> **NOTE**
> Hydris does not certify compliance and is not a certified OCEMS. It produces records (telemetry with provenance, inspection results, verified documents and an append-only audit trail) that make compliance easier to demonstrate. OCEMS analyzers can feed Hydris through the gateway ingestion API so operators see the values the regulator sees.

Why now. Camera glasses such as Ray-Ban Meta are light, affordable and acceptable on a plant floor; Meta opened a device access toolkit (DAT) to third-party apps; and language models can now answer spoken questions from structured plant data reliably enough to help, provided they are grounded and constrained.

1.4 Personas and jobs

Six personas map onto the six roles in the permission matrix; a seventh, the Hydris platform administrator, is a flag on the user.

Table 2. Personas, primary surfaces and top jobs

Persona	Role	Primary surface	Top jobs
Field operator (Ravi)	Field operator	Glasses, Android app	Walk the round, see live values at the asset, acknowledge alarms, scan paper logs, ask by voice
Shift supervisor (Meena)	Supervisor	Console, Android app	Dispatch rounds, triage and assign alarms, review inspections, verify scans
Maintenance engineer (Arjun)	Maintenance engineer	Android app, glasses	Work orders and checklists, resolve alarms, SOPs at the asset
Plant manager (Priya)	Factory manager	Console	KPIs and reports, alarm rules, approve documents, decide what the glasses show
Compliance auditor (Mr Rao)	Viewer with report access	Console, PDF exports	Read approved records and their verification trail; never change anything
Organization admin	Org admin	Console	Members, invitations, sites, glasses UI profiles
Platform admin	Platform flag	Console platform page	Create and suspend customer organizations

Table 3. Jobs to be done

Job	When ... I want ... so that ...
J1 Round	When I start my round, I want my inspections in order, so I do not miss an asset
J2 Trust	When I stand at an asset, I want its live readings and their age, so I can trust what I record
J3 Alarm	When an alarm fires near me, I want to acknowledge it and see the action and SOP, so the right person is accountable
J4 Evidence	When I fill a paper log, I want to capture it once and have it verified, so the record is searchable and defensible
J5 Voice	When my hands are busy, I want a short spoken answer grounded in our data
J6 Control	As a manager, I want to control what each role sees on the glasses without an app update
J7 Devices	As an admin, I want to pair, reassign, suspend or revoke glasses in seconds
J8 Audit	As an auditor, I want every approved document traceable to a person and a time
J9 Evaluate	As a prospect, I want to try the product in minutes without talking to sales

1.5 The product family

Hydris is one platform with eight products. Each has its own part in this book, written as a compact PRD.



Figure 2. *The Hydris product family and its sub-products*

Table 4. Products, what they do and where they stand

Product	What it does	Status
Website and demo	Seven public pages, contact and demo request, live glasses demo, one-click named demo accounts	Built
Operator console	Sixteen sections in five groups (Operate, Work, Plant, People and devices, Insight) plus admin and platform pages	Built
Glasses app	Twelve heads-up screens with directional input, offline inspection queue, scan and voice; phone layout and browser simulator	Built (web); native display rendering Planned
Hydris Field Android	Sign-in, home, work, alarms, sensors, audit scan with on-device OCR, "Hey Hydris", glasses connection	Partial: built in code, glasses paths not validated on hardware
Ray-Ban Meta integration	Camera, capture button, microphone and speakers through Meta DAT 1.0.1; registration as a Hydris device	Partial: not validated on physical glasses
Audit scanning	Guided A4 capture, dual OCR, optional vision engine, field review, four-eyes approval, searchable PDF	Built
Voice assistant	Wake phrase, short spoken answers from tenant data through read-only tools, rules fallback	Built (English); other languages Planned
Platform API and data	143 routes, row-level security, realtime events, append-only audit trail, demo mode	Built

1.6 One account, every surface

A user signs in once and gets the same permissions and data on the console, the glasses and the phone. Every surface talks to one API; the database enforces tenant isolation, not the application code.

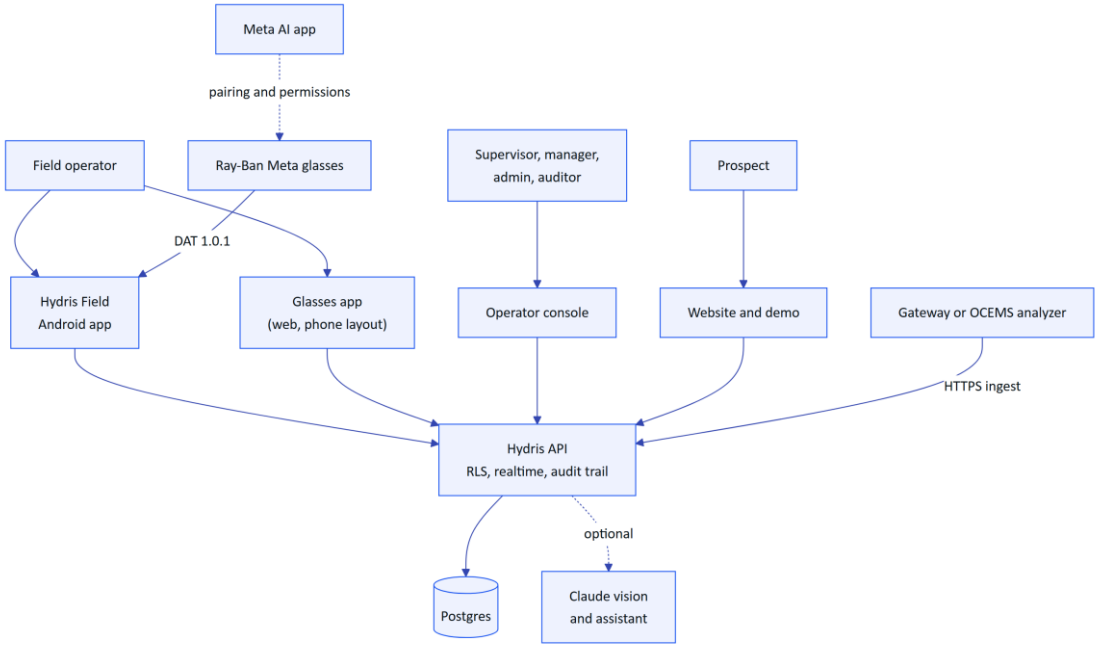


Figure 3. *System context: people, Hydris surfaces and external systems*

> **IMPORTANT**
>
> Ray-Ban Meta glasses reach third-party apps only through Meta's DAT, and they are paired and permissioned through the Meta AI app. That app must stay installed on every operator phone; Hydris Field checks for it and sends the user there to connect.

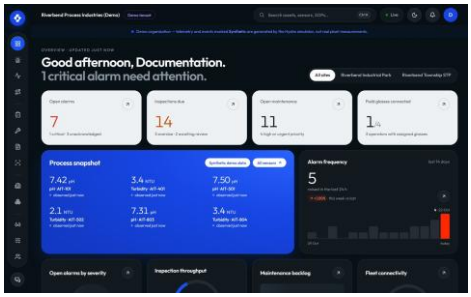


Figure 4. *The operator console overview: KPIs, open alarms and live measurements*

1.7 The operator's day

The journey below is the target state for a field operator across one shift. Scores are design targets (1 to 5), not measurements.

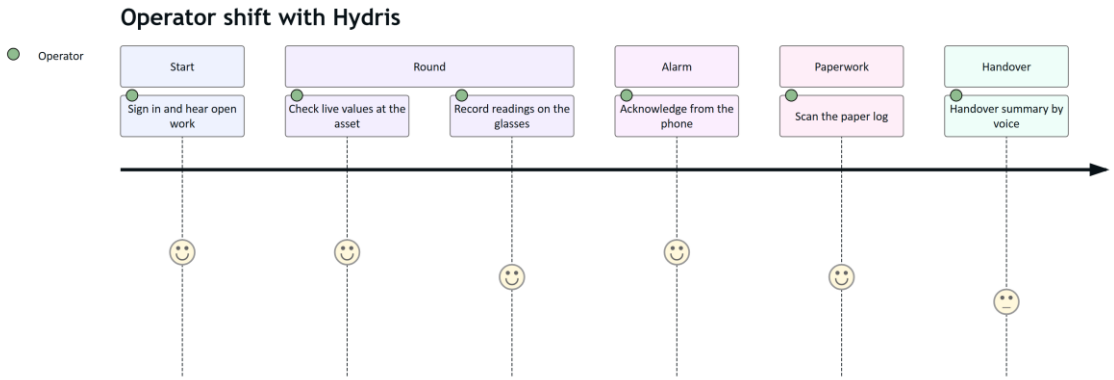


Figure 5. *Field operator journey across one shift (target state)*

The product is strongest today at alarm acknowledgement, scanning and voice lookups. The handover summary is **Planned**, and inspection execution on the Android app is **Planned**; rounds are executed in the glasses app today.



Figure 6. *Hydris Field home: greeting, alarm trend, asset health and live measurements*

1.8 The core loops

Four loops explain the whole product. Each crosses several products, which is why they share one platform.

See and act on the plant

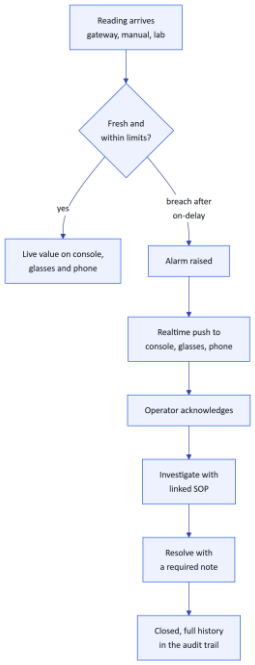


Figure 7. *Loop one: a reading becomes an alarm, the right person acts and the record closes*

Every reading passes one ingestion path with provenance. Alarm rules use deadband and on-delay so values that hover do not flap, and the lifecycle refuses to resolve an alarm without a note, so "acknowledged" is never mistaken for "fixed".

Do the work right

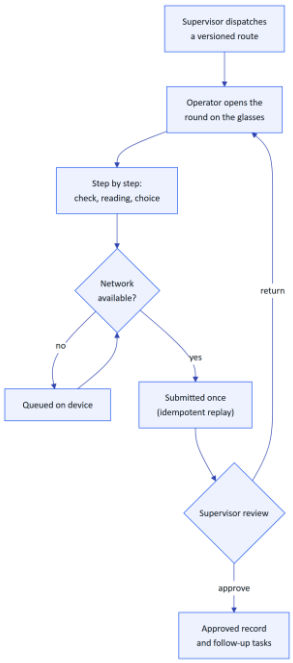


Figure 8. *Loop two: a dispatched round is executed hands-free, even offline, and reviewed*

Inspection templates are versioned, so a round keeps the version it was dispatched with. Responses are queued locally and replayed exactly once when the network returns.

Paper to verified record



Figure 9. *Loop three: a paper log becomes a verified, approved, searchable record*

Machines propose and people decide. The original extracted value is kept beside every correction, and approval must come from someone other than the verifier, except in demo tenants where self-approval is allowed and recorded.

Ask the plant

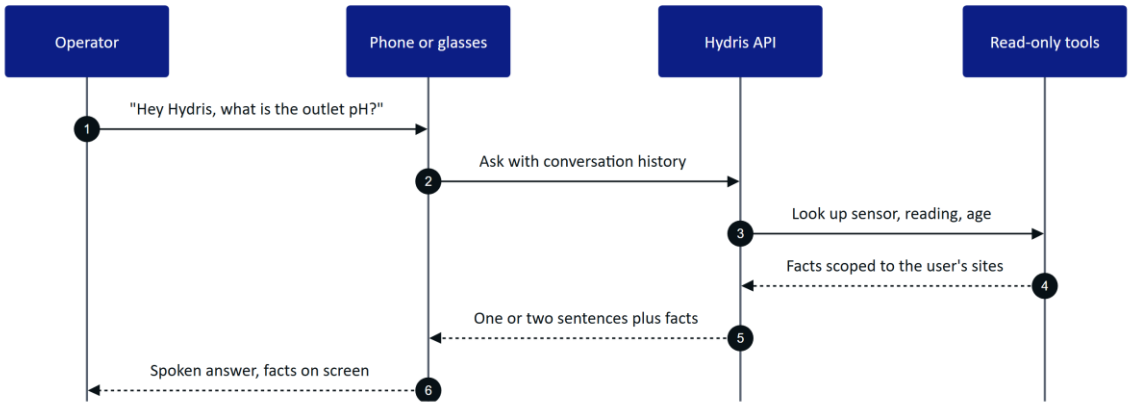


Figure 10. *Loop four: a spoken question answered from the tenant's own data*

The assistant answers only from tool results in the same conversation and states the age of each reading. If the model is unavailable, a rules engine answers instead, and an unknown question gets "can't answer", never a guess.

1.9 Status, gaps and the path to pilot

Table 5. Top gaps before a customer pilot

Gap	Effect today	Plan
Glasses paths not run on physical Ray-Ban Meta	Camera, capture button and glasses microphone are Partial	Hardware test matrix, Q4 2026 to Q1 2027
Email not sent	Invitations and resets sit in an outbox	Email transport in hardening, Q4 2026

Gap	Effect today	Plan
No continuous integration	Regressions can reach the demo	CI gates on every change, Q4 2026
Daily cron on serverless hosting	Stale-data alarms evaluated once a day	Per-minute schedule, Q4 2026
No Android offline outbox	Phone actions need connectivity	Planned for GA, Q2 2027

North star
verified field records per active operator per week
rises only with real, trusted field use

95 %
on-time rounds
pilot target

Under 5 min
median alarm acknowledgement
pilot target, critical and high

0
hallucinated numbers
target for voice answers

1.10 How to read this book

Each product part follows the same spine: purpose and problem, users and jobs, scope, key flows, requirements (with the stable FR IDs from the PRD and status **Built**, **Partial** or **Planned**), experience notes, success metrics (targets labelled as targets) and status with next steps.

Table 6. Part map

Part	Read it if you want to know
Hydris Field Glasses at a glance	What Hydris is, for whom, and how the products fit together
Website and demo	How a prospect discovers and tries Hydris in minutes
Operator console and its sub-products	How supervisors, managers and admins run the plant, the work and the devices
Glasses app	What the operator sees and does on the heads-up display and phone layout
Hydris Field Android app	The native companion app for operators and engineers
Ray-Ban Meta integration	How the glasses' camera, button, microphone and speakers reach Hydris
Audit scanning	How paper becomes verified, approved evidence
Voice assistant	How "Hey Hydris" answers from plant data without guessing
Platform API and data	Tenancy, security, realtime and the shared data model underneath
Roadmap and future products	What comes next, in what order, and how we will know it worked

2 Website and demo

2.1 Purpose and problem

Industrial buyers rarely get to touch a product before a sales cycle starts, and smart-glasses software is hard to picture without the hardware. The website explains Hydris and, more importantly, lets a prospect try it in minutes: drive the real glasses viewport in the browser, then sign in to the console or the Android app with just a name. Everything runs on synthetic data that is labelled as such and can never reach a customer tenant.

2.2 Users and jobs

Table 7. Users of the website and demo

User	Job
Prospect (plant manager, IT lead, consultant)	Understand the product and try it without talking to sales (J9)
Partner or investor	See the real product working, on any laptop or phone
Hydris sales and product team	Run a repeatable demo from any browser, with named accounts that survive between sessions
Trainer	Replay a deterministic scenario for a class, the same way every time

2.3 Scope

Table 8. Scope of the website and demo

In scope	Out of scope
Seven public pages (`/`, `/product`, `/technology`, `/operations`, `/demo`, `/contact`, `/privacy`), phone and desktop, light and dark	Self-serve sign-up for customer tenants
Contact and demo-request form	Pricing pages, payments
Live glasses demo, fully client-side, ten scenarios	Real plant data of any kind in the demo
Browser simulator and pairing page	A demo that runs on physical glasses without the app
One-click named demo accounts on console and Android	Persistent guest data beyond the shared store's 30-day TTL

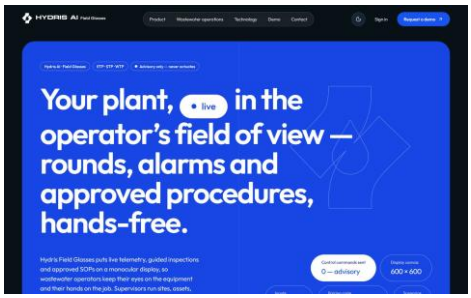


Figure 11. *The public website home page*

2.4 Key flows

From visitor to signed-in guest

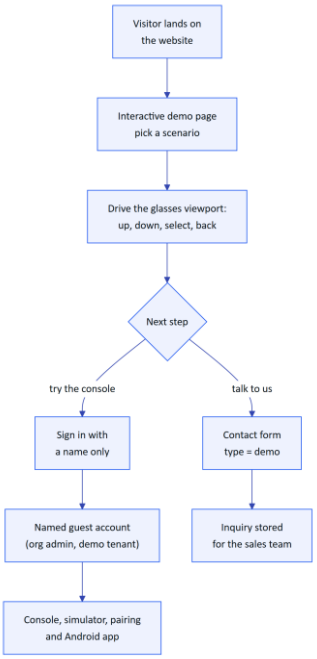


Figure 12. *The prospect's path from the website to a named demo account*

The `/demo` page runs the production glasses UI in a browser frame against a deterministic simulator of the reference plant, with no API calls. The ten scenarios cover normal operations, pump vibration, tank high level, abnormal pH, a scheduled inspection, equipment offline, glasses network loss, a high-priority alert, a maintenance checklist and device reassignment. Each replays identically, which makes it useful for training. From there a visitor books a guided demo or signs in by name.

One-click named demo account

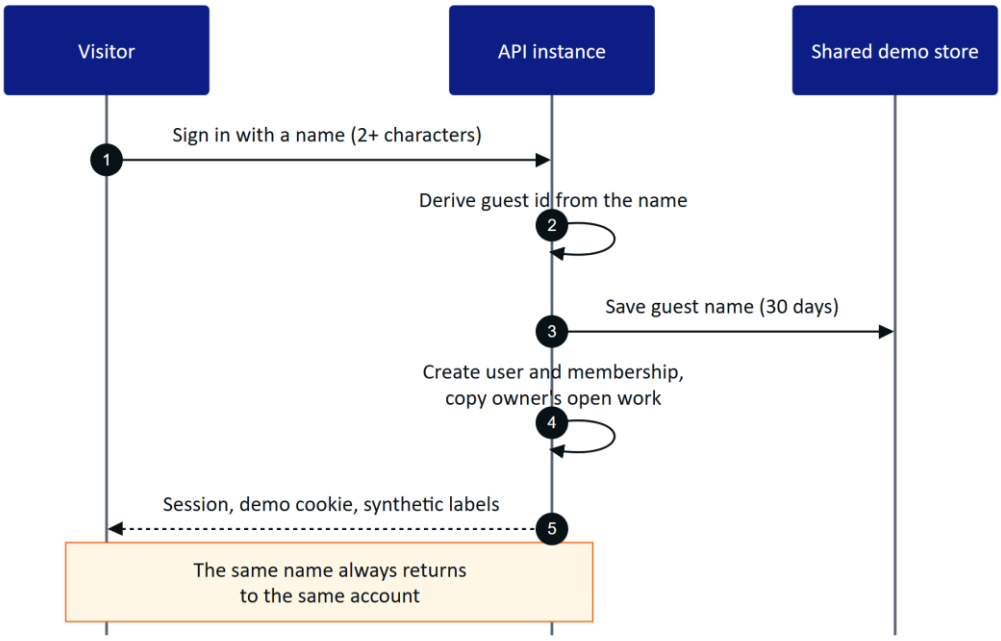


Figure 13. *Demo sign-in creates or reuses a named guest and starts a normal session*

A guest is an organization administrator of the demo tenant with access to all sites. The account id is derived deterministically from the normalized name, so returning with the same name restores the same account. Each guest gets personal copies of the demo owner's scheduled inspections and open maintenance tasks, so two guests acting at once never disturb each other's work.

Cross-instance demo state

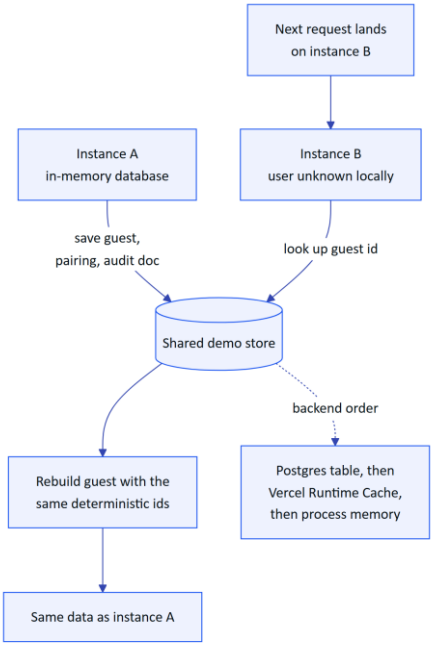


Figure 14. *A guest created on one serverless instance is rebuilt on another from the shared store*

Demo mode turns on automatically on Vercel when no database is configured. Every serverless instance seeds the same deterministic tenant in memory, so only a little state must travel between instances: named guests, pairing challenges and their approvals, and audit documents (snapshotted after each write). A resilient wrapper falls back to memory on any store error, so the demo never fails because of the store. Demo device refresh tokens are signed and self-contained, so any instance can rebuild a paired device.

2.5 Requirements

Table 9. Website and demo requirements

ID	Requirement	Priority	Status
FR-100	Marketing website with seven pages and a privacy page, phone and desktop, light and dark	Must	Built
FR-101	Contact and demo-request form, rate limited to 5 per 10 minutes per client	Must	Built
FR-102	Live glasses demo from the website, fully client-side, plus the browser simulator	Must	Built
FR-103	Named demo guests with personal copies of scheduled inspections and open tasks	Must	Built
FR-083	Zero-configuration synthetic demo tenant, named guests, cross-instance pairing	Must	Built
FR-084	Ten deterministic simulation scenarios, refused outside demo tenants	Must	Built
FR-091	Demo sign-in with a name in the Android app	Must	Built
FR-029	Synthetic data labelled everywhere and refused for customer tenants	Must	Built

2.6 Experience notes

- **Simulator.** `/simulator` renders the 600 by 600 heads-up display with a simulated camera that films a sample A4 sheet, so the whole scan pipeline can be shown without hardware. The website may embed `/glasses` and `/simulator`; every other page refuses framing.
- **Pairing in the demo.** A guest can pair the simulator or the phone layout by QR or typed code on `/pair`, and the pairing works even when the steps land on different instances.
- **Honest labelling.** The demo page shows a synthetic-data notice, and every value and voice answer in the demo tenant says it is synthetic demonstration data.
- **Limits.** Demo sign-in is limited to 20 per minute per client; the inquiry form to 5 per 10 minutes. The client IP on inquiries is stored hashed.

> **CAUTION**
> Demo mode relaxes security on purpose: public signing secrets, 12-hour access tokens instead of 15 minutes, self-contained device refresh tokens, acceptance of tokens whose session family the instance has never seen, and self-approval of audit documents. It is acceptable only because demo data is synthetic and flagged; production sets `DEMO_MODE=false` and `SIMULATOR_ENABLED=false`.

2.7 Success metrics

Table 10. Website and demo metrics (targets)

Metric	Target
Time from landing to a signed-in demo account	Minutes, with no sales contact (target, J9)
Demo availability on a zero-configuration deployment	Works with no environment variables (acceptance criterion of FR-083)
Guest isolation	Zero cases of one guest's personal work changing another's (target)
Demo requests converted to guided demos	Tracked from inquiries once the pilot pipeline opens (target to be set)

2.8 Status, gaps and next steps

The website and demo are **Built** and live. Open items:

- Inquiries are stored but no notification email is sent, because email delivery (FR-086) is **Planned** for hardening in Q4 2026.
- The Vercel Runtime Cache backend is per region and best effort; a Postgres shared store is preferred where available.
- The live demo mimics glasses input with on-screen buttons; a native device adapter for display glasses is **Planned** (see the roadmap part).

3 Operator console

The operator console is the web application at `/console` where supervisors, managers, auditors and administrators run the plant record. Field operators produce data with the glasses and the Android app; the console is where that data is dispatched, triaged, reviewed, approved and reported. It is one Next.js surface (`apps/web/src/console`) over the same API that serves every other client, so a value acknowledged on the glasses is the same record a manager sees on the overview a second later.

16
Navigation destinations
`Shell.tsx`

6
Roles
38 permissions, `roles.ts`

5
Alarm lifecycle states
resolve needs a note

2 s
Realtime alarm target
NFR-002, not yet measured

3.1 Purpose, users and information architecture

Purpose and problem

Water and wastewater plants still run on paper log sheets, phone calls and spreadsheets. The supervisor cannot see which rounds are late, the manager cannot prove to a regulator who saw an alarm and when, and the auditor receives evidence that cannot be traced to a person. The console replaces that patchwork with one tenant-isolated, site-scoped workspace where every state change is attributed, timestamped and written to an append-only audit trail.

Users and jobs

Table 11. **Console users and their main jobs**

User	Role in code	Main jobs in the console
Shift supervisor	`supervisor`	Triage and assign alarms, dispatch rounds, review inspections, verify scanned documents, pair glasses
Plant manager	`factory_manager`	KPIs and reports, alarm rules, plant model, approve audit documents, devices and people
Compliance auditor	`viewer` plus `report.read`	Read the trail, approved documents and reports; export evidence
Organization admin	`org_admin`	Members, invitations, sites, glasses UI profiles, organization settings
Platform admin	`users.is_platform_admin`	Create and suspend customer organizations at `/console/platform`

Field operators and maintenance engineers can sign in to the console too, but their primary surfaces are the glasses and the Android app.

Information architecture

The navigation in `apps/web/src/console/Shell.tsx` groups destinations by intent. Each item declares the permission it needs; items the current role lacks are hidden rather than greyed out, and the API enforces the same check.

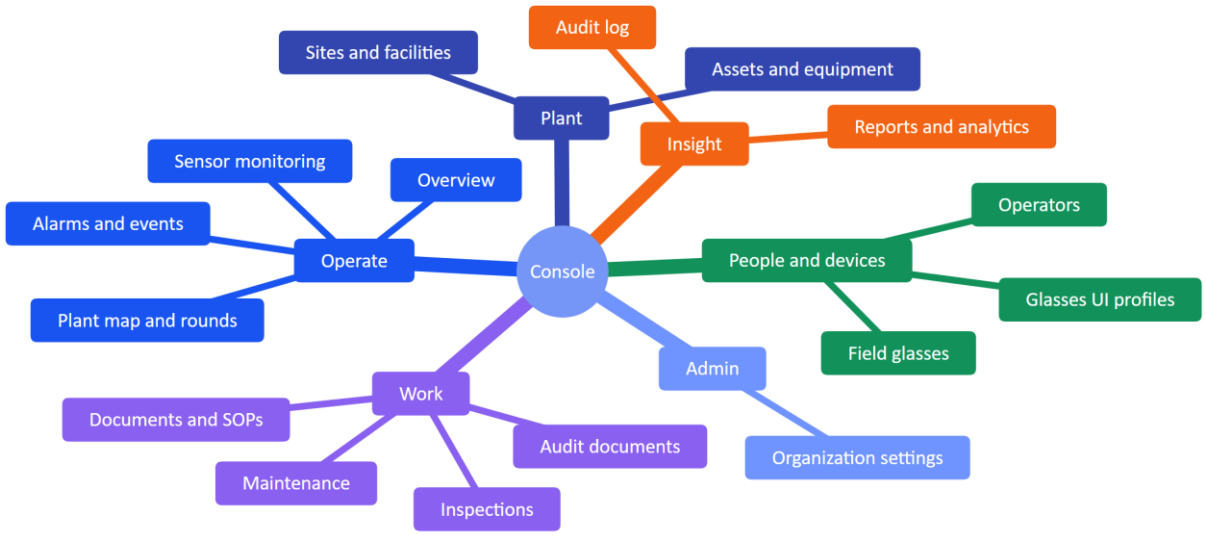


Figure 15. *Console navigation groups and destinations (Shell.tsx)*

A command palette (`CommandPalette.tsx`) gives keyboard access to every destination and to global search across assets, sensors, documents and work (FR-088). The console supports system, light and dark themes.

Role access summary

Permissions come from one fixed matrix (`packages/domain/src/roles.ts`, FR-012): `can(role, permission)` is the only check, and every role except `org_admin` is additionally scoped to its sites (FR-007). Row-level security in Postgres keeps tenants apart (FR-008).

Table 12. What each role can do in the console (summary of roles.ts)

Capability	Admin	Manager	Supervisor	Engineer	Operator	Viewer
Read plant, telemetry, alarms, work	Yes	Yes	Yes	Yes	Yes	Yes
Manage sites, assets, sensors, gateways	Yes	Yes	No	No	No	No
Manage alarm rules	Yes	Yes	No	No	No	No
Acknowledge alarms	Yes	Yes	Yes	Yes	Yes	No
Resolve alarms / close alarms	Yes / Yes	Yes / Yes	Yes / Yes	Yes / No	No / No	No / No
Manage and review inspections	Yes	Yes	Yes	No	No	No
Manage maintenance	Yes	Yes	Yes	No	No	No
Pair glasses / manage devices and profiles	Yes / Yes	Yes / Yes	Yes / No	Yes / No	Yes / No	No / No
Manage documents and SOPs	Yes	Yes	No	No	No	No
Verify / approve audit documents	Yes / Yes	Yes / Yes	Yes / No	No / No	No / No	No / No
Reports, audit log	Yes	Yes	Reports	No	No	Reports
Members and invitations / org settings	Yes / Yes	Yes / No	Read / No	No	No	No

> **NOTE**
> The auditor persona maps to `viewer`, which today holds read access plus `report.read` but not `audit.read`. Giving auditors the audit log page needs either a role change or a dedicated auditor role; this is an open product decision.

3.2 Overview dashboard

Purpose and scope

The overview is the first screen after sign-in. It answers "is anything wrong right now, and where?" in one glance: KPI tiles, open alarms by severity, asset health, live measurements with freshness, and recent activity. It is served by `GET /overview` (FR-089), the same endpoint that feeds the Android home screen (FR-092), so the console and the phone never disagree.

In scope: KPIs, open alarms, health roll-up, live values with freshness and synthetic labels, recent activity, realtime refresh. Out of scope: user-configurable widgets and cross-site benchmarking.

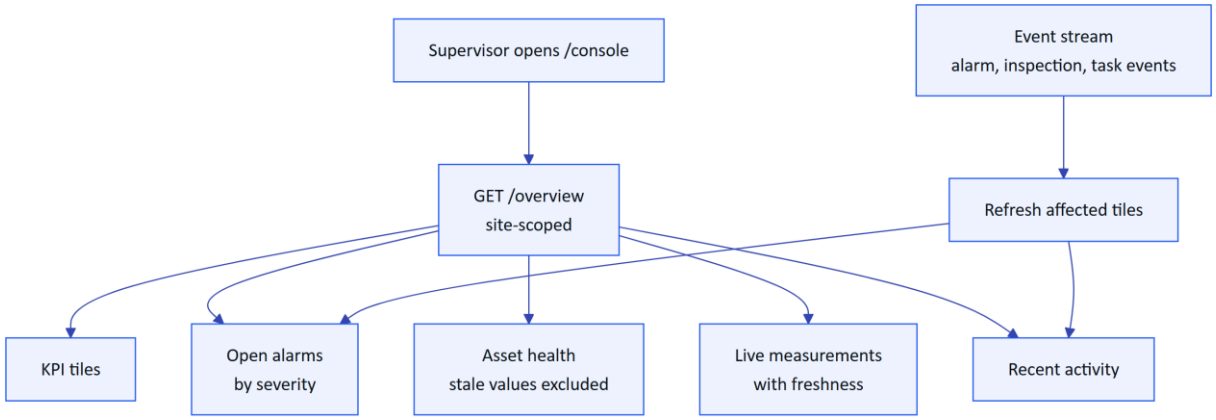


Figure 16. *How the overview is assembled and kept current*

Freshness is computed per sensor from its expected interval, and stale values are excluded from health roll-ups (`packages/domain/src/freshness.ts`). Synthetic demo values carry a visible label wherever they appear (FR-029).

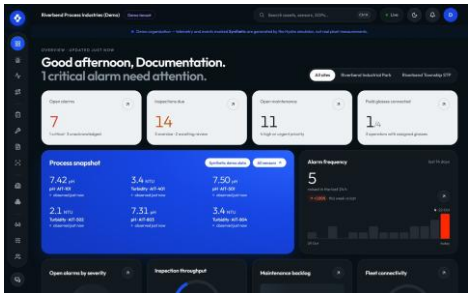


Figure 17. *Overview dashboard in the console*

Table 13. Overview requirements

ID	Requirement	Pri	Status
FR-089	Overview with KPIs, open alarms, health and live measurements	M	Built
FR-026	Freshness states shown; missing data never shown as a value	M	Built
FR-085	In-app notifications in realtime, mark as read	M	Built
FR-088	Global search across assets, sensors, documents and work	S	Built

Status: Built. Next step: per-user default site and a compliance tile once FR-120 lands.

3.3 Plant model: sites, assets, sensors and the plant map

Purpose and scope

Everything in Hydris hangs off the plant model: alarms, rounds, documents and devices are all scoped to a site and usually an asset. The plant model lets managers describe their plant once (site, facility, process area, asset, sensor) and place it on a map, so that operators walking the plant see the right asset and the right values.

In scope: hierarchy with map positions, archive and restore (FR-020), sensors for 16 measurands with unit normalization (FR-021), gateways with `hgw_` API keys, the plant map with rounds overlay. Out of scope: digital twins, P&ID import, write paths to equipment (Hydris is read-only towards the plant).

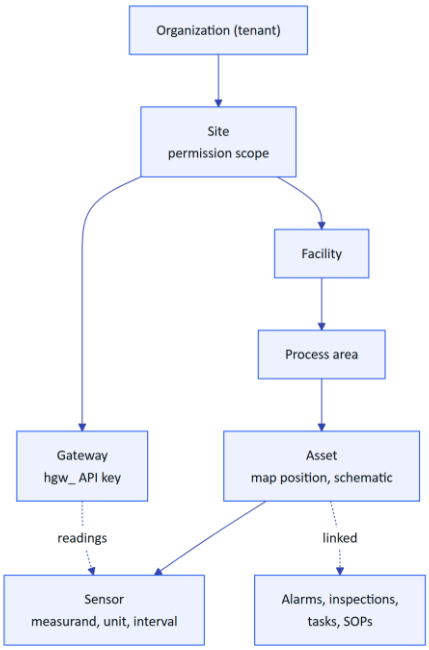


Figure 18. *Plant hierarchy and what attaches at each level*

The site level is the permission boundary: a site-scoped supervisor sees nothing from other sites. Archiving an asset removes it from glasses screens and assistant answers while keeping its history.

Plant map and rounds

The plant map (`/console/map`, `apps/web/src/console/plant/map`) places assets on the site layout and overlays route stops, so a supervisor can see where a round goes and where open alarms sit. The hierarchy navigator and asset schematic (`HierarchyNavigator.tsx`, `AssetSchematic.tsx`) give the same model as a tree and as a per-asset view of its sensors.

Table 14. **Plant model requirements**

ID	Requirement	Pri	Status
FR-020	Site, facility, area and asset hierarchy with map positions, archive and restore	M	Built
FR-021	Sensors for 16 measurands with unit normalization	M	Built
FR-007	Site-scoped permissions	M	Built
FR-088	Search by asset tag finds the asset and its open work	S	Built

Status: Built. Gaps: no bulk import of the asset register; map layout is drawn per site rather than imported from drawings.

3.4 Telemetry and sensor monitoring

Purpose and scope

Sensor monitoring shows what the plant measures, how fresh each value is, and where a value came from. The core promise is honesty: a missing reading is shown as "No data", never as zero, and every value carries its source (gateway, manual, lab or synthetic) and its time.

In scope: HTTPS gateway ingestion (FR-022), manual and lab entry with provenance (FR-023), append-only corrections (FR-024), history and CSV export (FR-025), freshness states (FR-026). Out of scope today: MQTT and OPC UA adapters (FR-027) and read-only historian connectors (FR-028), both Planned.

Ingestion flow

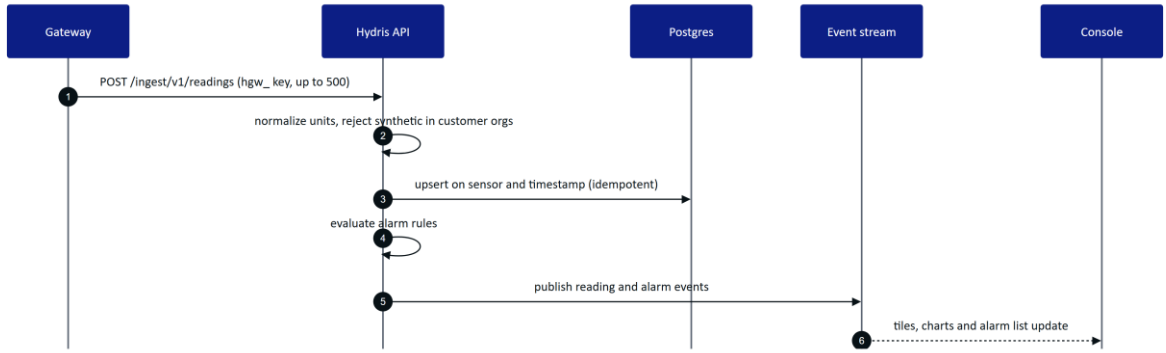


Figure 19. *From a gateway reading to the console (telemetry.ts, alarms.ts, realtime.ts)*

Replaying the same batch never creates duplicates because readings are keyed on sensor and timestamp. Synthetic readings sent into a customer organization are refused (FR-029).

Freshness

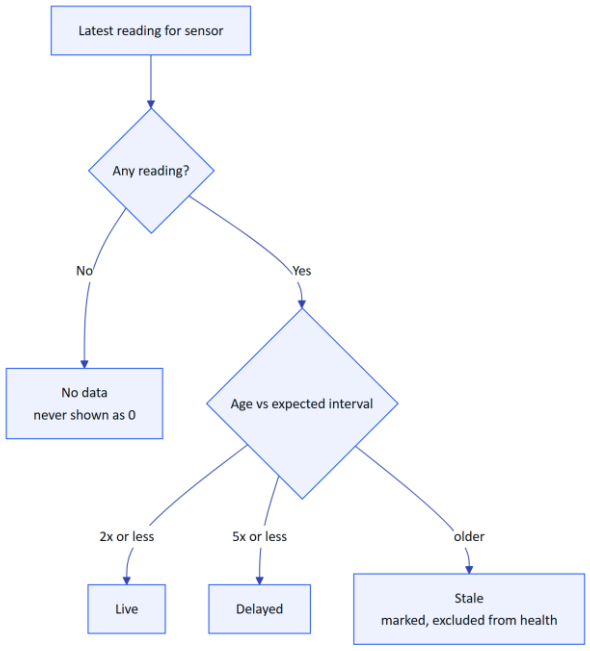


Figure 20. *Freshness state of a sensor value (freshness.ts)*

Corrections and history

A supervisor who finds a wrong value does not edit it. They add a correction or invalidate the reading; the original stays visible next to the correction with its author and reason (FR-024). Manual and lab entries (`ManualEntryDialog.tsx`) carry their provenance everywhere they appear. The sensor detail shows raw readings and history, and exports CSV with formula injection escaped (FR-025).

Table 15. Telemetry requirements

ID	Requirement	Pri	Status
FR-022	Gateway ingestion, 500 per call, idempotent	M	Built
FR-023	Manual and lab entry with provenance	M	Built
FR-024	Append-only corrections (correct or invalidate)	M	Built
FR-025	History, raw readings, CSV export	S	Built
FR-026	Freshness live, delayed, stale, no data	M	Built
FR-029	Synthetic data labelled, refused for customers	M	Built
FR-027	MQTT and OPC UA adapters	S	Planned
FR-028	Read-only SCADA and historian connectors	C	Planned

Status: Built for HTTPS. Next: MQTT and OPC UA adapters so pilots can connect existing gateways without custom code.

3.5 Alarms

Purpose and scope

Alarms turn readings into attributed work. The console is where alarm rules are defined, where supervisors triage and assign, and where the evidence of who saw what and when is kept. Acknowledging records that a person has seen the alarm; it never changes the process.

In scope: threshold and stale rules with deadband and on-delay (FR-030), the five-state lifecycle (FR-031), assignment, notes, history and linked SOP (FR-032, FR-112), realtime push (FR-033). Planned: escalation (FR-037), shelving (FR-038), push notifications when the app is closed (FR-034).

Rule evaluation

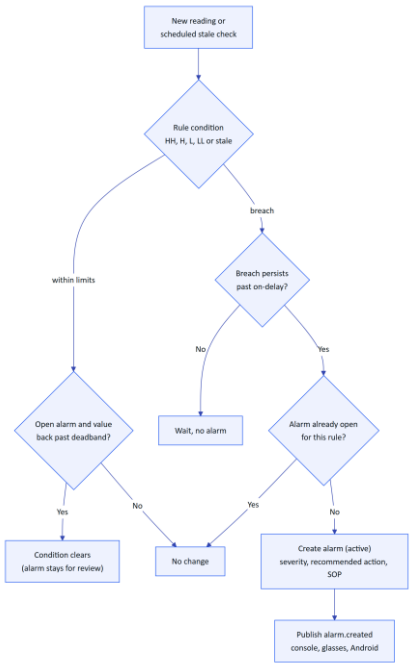


Figure 21. *How a rule turns readings into an alarm (alarms.ts)*

The deadband stops a value oscillating around a limit from raising and clearing repeatedly, and the on-delay ignores short spikes. Stale rules run on a schedule: every 30 s on a long-running server, but only once a day on the current Vercel cron (FR-035, Partial).

Lifecycle and permissions

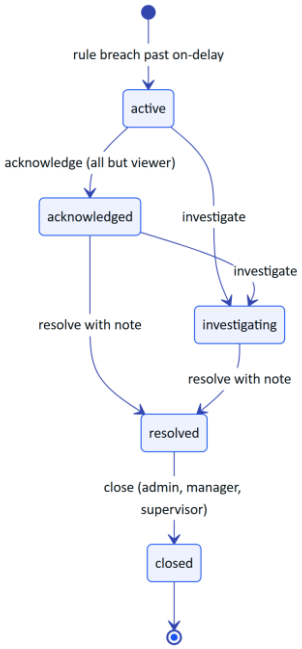


Figure 22. *Alarm lifecycle with the permission each step needs (packages/domain/src/alarms.ts)*

There is no path from active straight to resolved: someone must first see the alarm. Resolving requires a note, and the API rejects a resolve without one. Investigate and resolve are held by admins, managers, supervisors and maintenance engineers. Every transition writes a history row and an audit event.

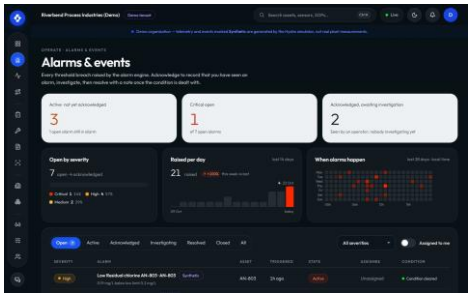


Figure 23. *Alarms and events in the console*

Escalation

Escalation is Planned for the pilot (FR-037): an unacknowledged critical alarm will notify the site supervisor after a configurable number of minutes, and the escalation will appear in the alarm history. Until then, supervisors rely on the realtime alarm list, the Android notification for critical and high alarms while the app runs (FR-097), and the median time to acknowledge in the alarm report.

Table 16. Alarm requirements

ID	Requirement	Pri	Status
FR-030	Threshold rules with deadband and on-delay	M	Built
FR-031	Five-state lifecycle, no resolve from active, note required	M	Built
FR-032	Assignment, notes, history, recommended action, SOP	M	Built
FR-033	Realtime push to console, glasses, Android	M	Built
FR-112	Rule links a procedure shown with the alarm	M	Built
FR-035	Stale-data alarms on a schedule	M	Partial
FR-037	Escalation when unacknowledged	S	Planned

ID	Requirement	Pri	Status
FR-038	Shelving with reason and expiry	C	Planned
FR-034	Push notifications when the app is closed	S	Planned

Status: core lifecycle Built. Gaps: stale evaluation cadence on serverless, escalation and shelving. Next: escalation first, because time to acknowledge is the headline alarm metric.

3.6 Inspections and maintenance

Purpose and scope

Rounds and maintenance are the daily work of the plant. The console lets a supervisor design what must be checked, send it to the right operator, and review the result; and lets anyone turn an alarm or an abnormal reading into a tracked task. Execution happens on the glasses (FR-041) and, for checklists, on the phone (FR-046).

In scope: versioned templates with five step kinds and routes with dispatch (FR-040), review with approve or return (FR-042), maintenance tasks with checklists and follow-ups (FR-043). Planned: recurring schedules (FR-047) and inspection execution on Android (FR-044).

Templates, routes and dispatch

A template is built in the console (`TemplateBuilder.tsx`) from steps: check, reading with expected range, choice with abnormal options, text and photo. Editing creates a new version, so inspections already dispatched keep the version they started with. Routes order templates along a walk and are dispatched to an operator. On the glasses, a reading outside its range, a failed check or an abnormal choice marks the step abnormal automatically.

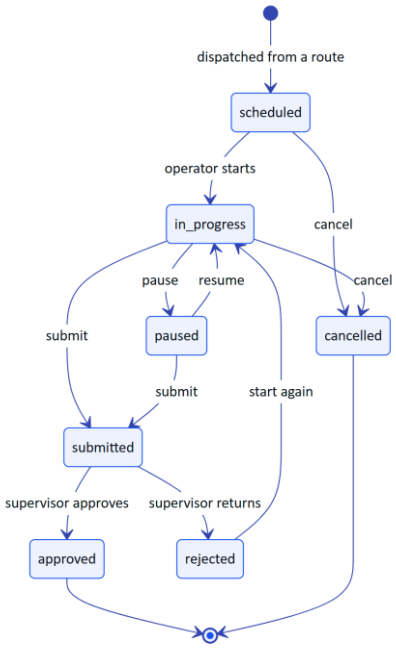


Figure 24. *Inspection lifecycle as the supervisor sees it (workflows.ts)*

Review is the console's job. The supervisor opens a submitted inspection, sees each step with abnormal ones highlighted and any photos, and either approves it or returns it with a reason. A returned inspection can be restarted and resubmitted, and history shows both submissions. Offline responses from the glasses are replayed idempotently, so the reviewer never sees duplicates.

Maintenance

Maintenance tasks are corrective, preventive, inspection follow-ups or alarm follow-ups, with a priority from low to urgent, a checklist and an activity log. A follow-up created from an alarm or inspection links back to its source.

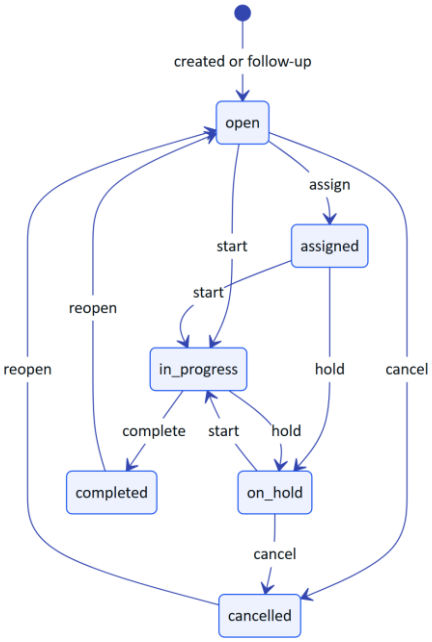


Figure 25. *Maintenance task lifecycle (MAINTENANCE_TRANSITIONS in workflows.ts)*

Table 17. **Inspection and maintenance requirements**

ID	Requirement	Pri	Status
FR-040	Versioned templates, five step kinds, routes, dispatch	M	Built
FR-041	Glasses execution with offline queue	M	Built
FR-042	Submit, review, approve or return	M	Built
FR-043	Maintenance tasks, checklists, follow-ups	M	Built
FR-046	Tick checklist items from Android	M	Built
FR-045	Photo steps from glasses or phone camera	S	Partial
FR-044	Inspection execution in the Android app	S	Planned
FR-047	Recurring schedules (calendar or runtime)	S	Planned

Status: Built for manual dispatch and review. Gap: every round is dispatched by hand. Next: recurring schedules, which also make on-time completion meaningful as a KPI.

3.7 Documents and SOPs

Purpose and scope

Operators need the right procedure at the asset, and only an approved one. The document library (`/console/documents`) holds SOPs, manuals, datasheets, safety notices and troubleshooting guides, each with versions and a scope of site, asset or asset type (FR-110). Only approved documents reach the glasses, the assistant and grounded retrieval (FR-111).

Lifecycle: draft, approved, archived. Managers and admins hold `document.manage`; everyone else reads. Alarm rules link a procedure that is shown with the alarm (FR-112). The library page also exposes grounded SOP retrieval (`/assist`): a question returns passages from approved documents with a safety notice (FR-070), so a supervisor can check what an operator will be told before it happens.

Table 18. Documents requirements

ID	Requirement	Pri	Status
FR-110	Library with draft, approved, archived; versions; scope	M	Built
FR-111	Only approved documents reach glasses and assistant	M	Built
FR-112	Alarm rules link a procedure	M	Built
FR-070	Grounded SOP retrieval with safety notice	M	Built
FR-113	SOPs cached on glasses for offline reading	S	Built
FR-114	Upload PDF manuals with text extraction	C	Planned

Status: Built for authored documents. Gap: existing PDF manuals cannot yet be uploaded and searched (FR-114).

3.8 Field glasses fleet

Purpose and scope

Glasses are shared, often unattended devices. The console makes them a managed fleet: pair a new pair to a site and an operator, see which are connected, suspend or revoke one that is lost, and control what each pair shows through UI profiles. The glasses never hold a user password; they hold a device credential that the console can end.

In scope: pairing approval (FR-050), device tokens and revocation (FR-051, FR-052), the device permission ceiling (FR-053), UI profiles (FR-054). The camera, capture button and Android registration are covered in the glasses part of this book.

Pairing from the console

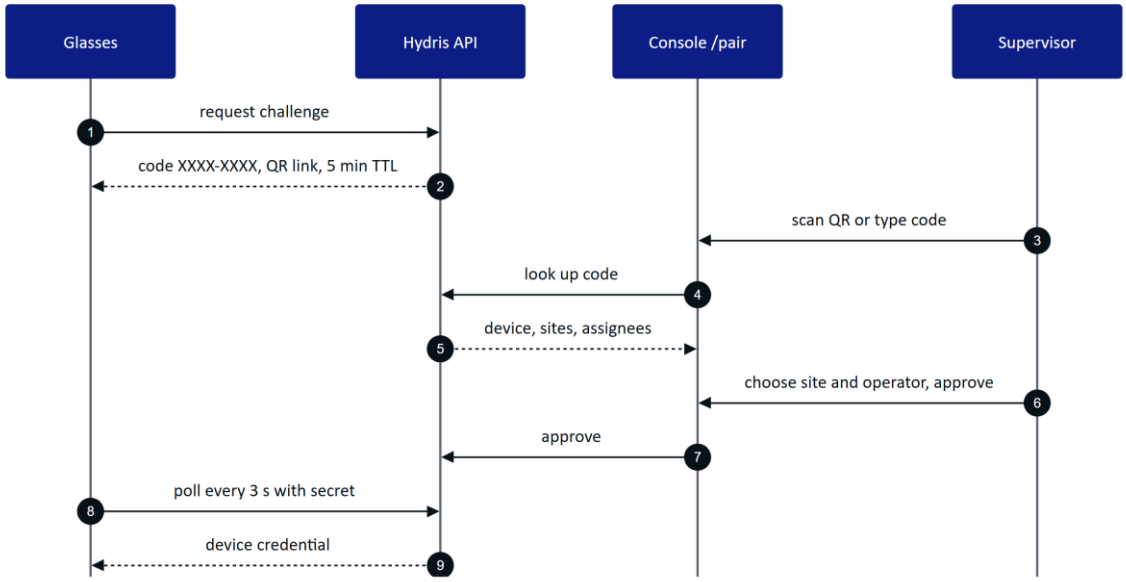


Figure 26. *Approving a pairing from the console (devices.ts)*

The code avoids confusable characters (no 0, O, 1, I, L, U), wrong-code lookups are rate limited, and the QR link never carries a credential. The approver needs `device.pair`, which every role except viewer holds.

Device states

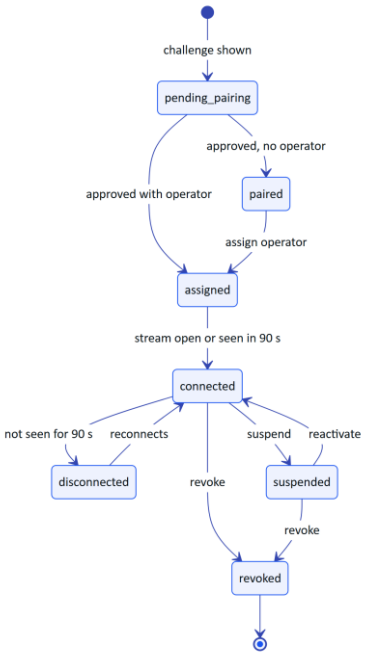


Figure 27. *Device states shown in the fleet list (deriveDeviceState in devices.ts)*

Revocation takes effect on the next request and closes an open event stream within 30 s (FR-052). Device access tokens last 15 minutes and refresh tokens rotate with reuse detection. Whatever role the assigned operator has, a device can never exceed the device permission ceiling: glasses paired to an admin still cannot manage members (FR-053).

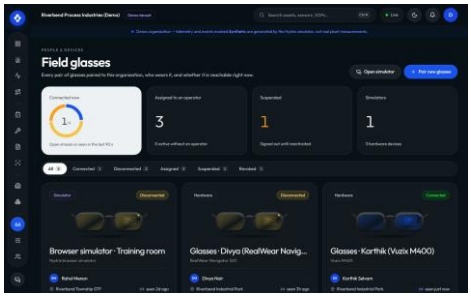


Figure 28. *Field glasses fleet in the console*

UI profiles

A UI profile controls what the glasses show (`/console/glasses-profiles`, `ui-profiles.ts`). Profiles are edited as drafts, published, and can be rolled back. Assignment can be made to a device, a user, a role or the whole organization; the most specific assignment with a published version wins. Publishing pushes the change live, so the glasses screens change without a restart. Managing profiles needs `device.manage` (admins and managers).

Table 19. Fleet requirements

ID	Requirement	Pri	Status
FR-050	Pairing code, QR, 5 min TTL, 3 s polling, approval	M	Built
FR-051	Device tokens, rotation, reuse detection	M	Built
FR-052	Suspend, reactivate, revoke, reassign	M	Built
FR-053	Device permission ceiling	M	Built
FR-054	UI profiles: draft, publish, rollback, precedence, live push	S	Built
FR-058	Log out and unpair from the glasses menu	M	Built

Status: Built. Next: fleet health signals (battery, app version) once the glasses bridge reports them.

3.9 Audit documents: review and approval

Purpose and scope

Paper log sheets scanned in the field become evidence only after a person has checked them and a second person has approved them. The console side (`/console/audit-docs`, `work/auditdocs.tsx`) is that review desk. Capture, page detection and OCR are covered in the scanning part of this book.

In scope: field review (confirm, correct, flag, reset) with the original value kept (FR-064), bulk-confirm where engines agree (FR-069), four-eyes approval (FR-065), the evidence PDF (FR-066). Planned: electronic signature with re-authentication (FR-068).

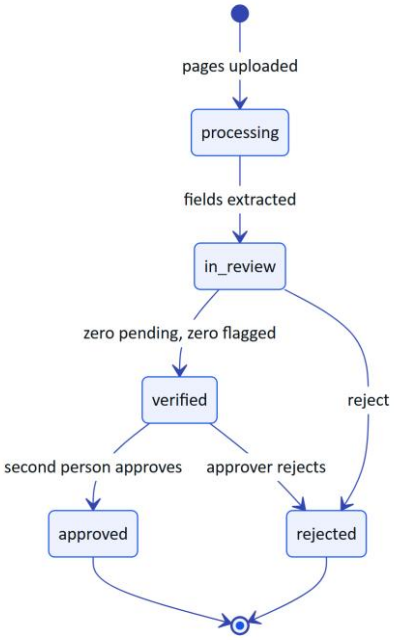


Figure 29. *Audit document states on the console side (AuditDocStatus in auditdoc.ts)*

The reviewer sees each extracted value beside its crop of the page, with the engines that read it and their agreement. Values where at least two independent engines match with confidence of 0.8 or more can be confirmed in bulk; everything else is confirmed, corrected or flagged one by one, and the original reading is always kept. Verify is blocked while any item is pending or flagged. Approval needs `auditdoc.approve` (admins and managers) and must come from someone other than the verifier, except in demo tenants where self-approval is allowed and recorded as such. The approved PDF carries the verification trail, the searchable original and a reconstructed page.

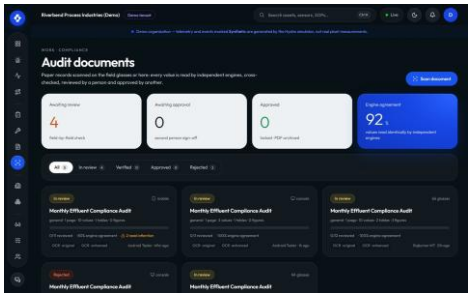


Figure 30. *Audit documents review in the console*

Table 20. Audit review requirements

ID	Requirement	Pri	Status
FR-064	Confirm, correct, flag, reset; verify gate	M	Built

ID	Requirement	Pri	Status
FR-069	Bulk-confirm agreed multi-engine values	S	Built
FR-065	Independent approval (four eyes)	M	Built
FR-066	PDF with trail, searchable original, reconstruction	M	Built
FR-068	Electronic signature with re-authentication	S	Planned

Status: Built. Next: electronic signature, which regulated customers will ask for before audit documents replace paper.

3.10 Reports, insights and the assistant

Purpose and scope

Managers and auditors need numbers they can defend. Reports (`/console/reports`, `insights.ts`) cover inspections, alarms, maintenance, devices and operators, in JSON for the page and CSV for export (FR-081). The alarm report includes median minutes to acknowledge; the inspection report includes on-time rate and the share submitted via glasses. Those fields are the data sources for most of the pilot KPIs below.

The assistant is grounded and read-only. In the console it appears as SOP retrieval on the documents page (FR-070); the voice assistant (`/assistant/ask`) answers only from tenant data through read tools and has no write tools (FR-072, FR-078). Its full design is in the assistant part of this book.

Table 21. Reports and insights requirements

ID	Requirement	Pri	Status
FR-081	Reports for five areas in JSON and CSV	M	Built
FR-080	Realtime events with replay	M	Built
FR-084	Simulation scenarios in demo tenants only	M	Built
FR-120	Compliance summary per discharge point	S	Planned
FR-121	Scheduled report delivery by email	C	Planned
FR-122	Grounded shift handover summary	C	Planned

Status: operational reports Built. Gap: no compliance summary against consent limits, which is the report regulators actually ask for. Next: FR-120 for the pilot.

3.11 Organization admin

Purpose and scope

Admins set up who can do what. The admin area covers operators (members), roles, invitations, sessions, organization settings, the audit log and, for Hydris staff, platform administration. Its rule is no escalation: nobody can grant a role above their own (`assignableRoles`).

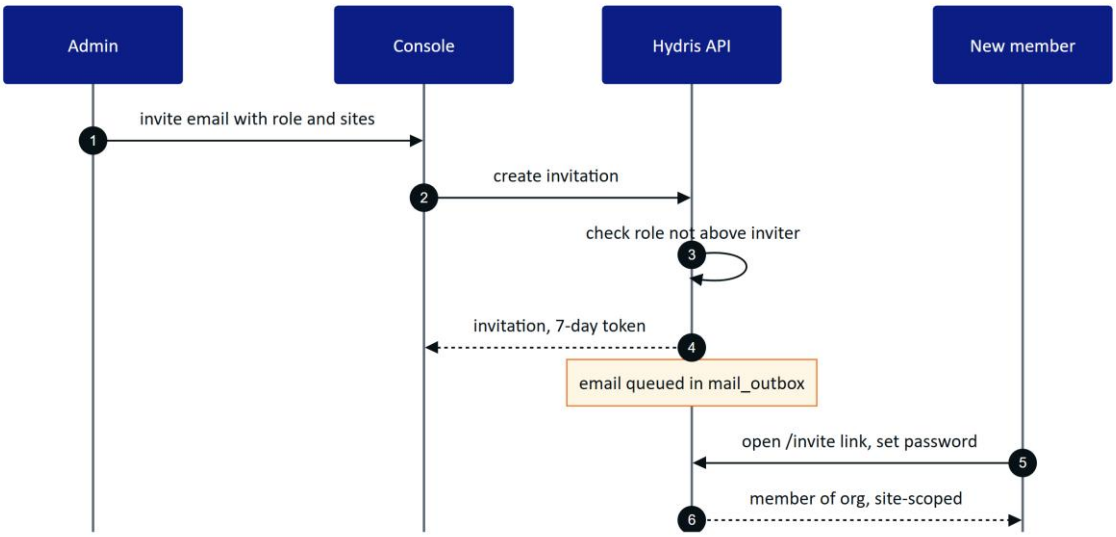


Figure 31. *Inviting a member (org.ts, auth.ts)*

Invitation emails are written to `mail_outbox` but not yet sent; until email delivery lands (FR-086), the admin shares the link. Members can belong to several organizations and switch between them (FR-006); sessions can be listed and revoked (FR-009); TOTP MFA is available (FR-002). Organization settings need `org.manage`, held only by `org_admin`.

The audit log page (`/console/audit`, FR-087) filters the append-only trail by action and entity and shows actor, time and IP; no API path updates or deletes an audit event (FR-082). Platform admins create and suspend customer organizations at `/console/platform` (FR-010); members of a suspended organization cannot sign in to it.

Table 22. Organization admin requirements

ID	Requirement	Pri	Status
FR-004	Invitations, 7-day token, no escalation	M	Built
FR-006	Multi-organization membership and switch	S	Built
FR-009	Session listing and revocation	S	Built
FR-012	Fixed role matrix	M	Built
FR-082	Append-only audit trail	M	Built
FR-087	Audit log page with filters	M	Built
FR-010	Platform organization create and suspend	M	Built
FR-005	Password reset email	M	Partial
FR-086	Email delivery (SMTP, SES or Resend)	M	Planned
FR-011	SSO with SAML 2.0 and OIDC	S	Planned

Status: Built apart from email delivery and SSO. Next: email delivery before the pilot, SSO for general availability.

3.12 Console success metrics and gaps

Success metrics

The console is measured by whether supervisors and managers act on field records quickly and trust them. All values below are pilot targets from the PRD, not measured results.

Table 23. Console metrics (pilot targets)

Area	Metric	Source	Target
Alarms	Median time to acknowledge, critical and high	Alarm report	Under 5 minutes
Alarms	Acknowledged but unresolved older than 24 h	Alarm list	Visible daily, trending down
Rounds	On-time completion	Inspection report `onTimeRate`	95 % or more
Rounds	Share submitted via glasses	Inspection report `viaGlasses`	50 % or more
Evidence	Approval cycle, capture to approval	Audit documents	Under 24 hours
Evidence	Reviewer edits per reviewed field	Audit documents	Under 5 %
Security	Device revocation to stream end	Event stream	30 s or less
Security	Cross-tenant incidents	Security review	0

< 5 min
Time to acknowledge
target, critical and high

95 %
On-time rounds
target

< 24 h
Evidence approval
target

0
Cross-tenant incidents
target

Gaps and next steps

Table 24. Console gaps in priority order

Gap	Requirement	Effect today	Next step
No email delivery	FR-086, FR-005	Admin shares invite links by hand	Wire SMTP, SES or Resend with retries
No escalation	FR-037	Unacknowledged alarms rely on someone watching	Configurable escalation to site supervisor
Stale checks daily on Vercel	FR-035	Stale alarms late on serverless	Higher-frequency scheduler
No recurring schedules	FR-047	Rounds dispatched by hand	Calendar and runtime schedules
No compliance summary	FR-120	Regulator reports built outside Hydris	Daily min, max, average against consent
No SSO	FR-011	Local passwords only	SAML and OIDC per organization
No electronic signature	FR-068	Approval not Part 11-style	Re-authentication on approve
Auditor cannot open audit log	roles.ts	`viewer` lacks `audit.read`	Decide on an auditor role
Performance not measured	NFR-001, NFR-002	Targets unproven	Load and latency tests before pilot

IMPORTANT
The console is functionally complete for a supervised pilot. The items that block an unsupervised pilot are email delivery, alarm escalation and a reliable stale-data schedule.

4 Glasses app (heads-up and phone)

4.1 Purpose and problem

Field operators walk rounds with gloves on, a radio in one hand and a log sheet in the other. They need the next inspection, the live reading at the asset in front of them, the alarm near them and the right procedure, without stopping to unlock a phone or walk back to the control room. The Hydris glasses app is one web viewport (`apps/web/src/glasses/*`, served at `/glasses`) that puts this work in front of the operator's eyes on display glasses, inside the phone bridge for Ray-Ban Meta, and in a browser simulator for demos and testing.

12
Screens
gated by the UI profile

6
Input keys
up, down, left, right, select, back

600 x 600
HUD canvas
additive display

0
Lost offline steps
queue plus idempotent replay

Hands-free principles

The app follows a small set of rules that come from the design system and are enforced in `hud.tsx`:

- **One primary action per screen**, one lime focus ring, and every action reachable with six directional keys or voice (NFR-021).
- **Glanceable.** Body type at 20 px or more, light strokes on black, no large filled areas, because pure black is transparent on an additive display (NFR-020).
- **Records only.** Hydris never operates equipment. Acknowledge carries the hint "does not change equipment", and the task screen says "Records only".
- **Honest data.** Cached data is labelled as cached with its age, never as live (NFR-026). Demo data is labelled as synthetic.
- **Administered, not configured on the device.** What a pair of glasses shows is decided in the console through UI profiles, with no app update (FR-054).

4.2 Users and jobs

Table 25. Who uses the glasses app and why

User	Job on the glasses	Requirements
Field operator	Walk the assigned round, record readings, acknowledge nearby alarms, open the SOP	FR-041, FR-059, FR-112
Field operator	Capture a paper log once and send it for verification	FR-060, FR-061
Field operator, hands busy	Ask "Hey Hydris" and hear a short grounded answer	FR-071, FR-072
Shift supervisor	Pair a pair of glasses to an operator and site from a phone	FR-050
Plant manager or admin	Decide what each role sees, roll back a bad change	FR-054
Prospect and sales	Try the full app in the browser simulator	FR-059

4.3 Scope

Table 26. Scope of the glasses app

In scope (Built)	Out of scope or Planned
HUD layout for display glasses, phone layout for the bridge, browser simulator	Rendering on a real glasses display (Planned)

In scope (Built)	Out of scope or Planned
Device pairing by code and QR, device tokens, log out and unpair	Native Vuzix or RealWear hosts (Planned)
Home, alerts, inspections with offline queue, tasks, measurements, assets, plant map	Camera capture into inspection photo steps (FR-045 Partial)
SOP library and reader with offline cache	Hardware gestures such as a neural band or temple touchpad (Planned)
Audit scan entry, Hey Hydris voice entry	Multi-language voice (FR-073 Planned)

4.4 Runtimes

The same React viewport runs in three places. `useViewportLayout` in `hud.tsx` picks the layout: phone mode when the page runs inside the bridge (`HydrisNative` is present), when `?mode=phone` is set, or on a narrow portrait screen; HUD mode otherwise.

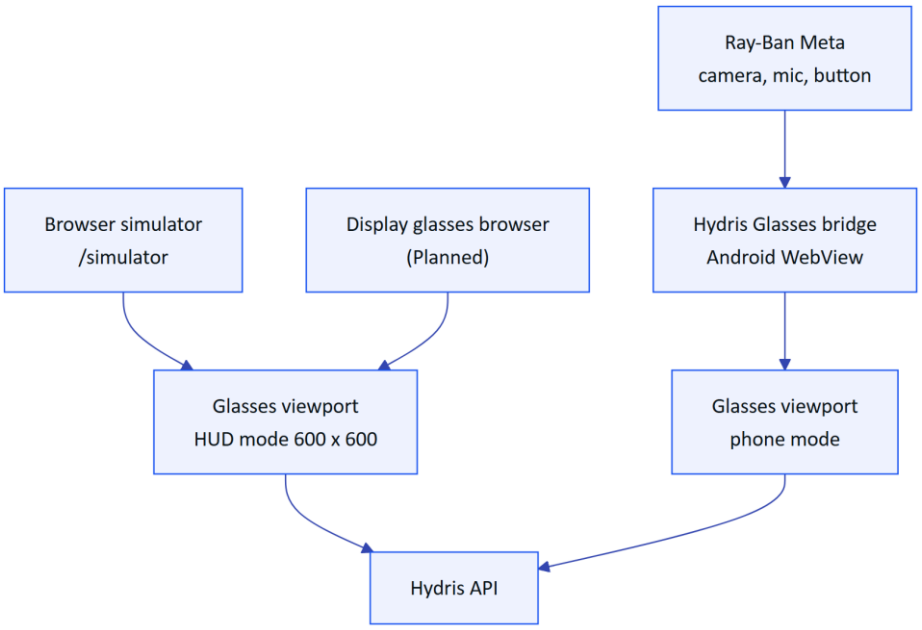


Figure 32. *One viewport, three runtimes, one backend*

The HUD scales a 600 x 600 canvas to the window and is driven by directional keys, swipes or the simulator's on-screen pad. Phone mode uses a 540-unit-wide layout at full height with safe-area insets and a bottom tab bar: Home or Back, Scan, Hey Hydris, Alerts. The simulator also offers 640 x 480 and 480 x 480 viewports, a network switch for simulating loss, and an event log.

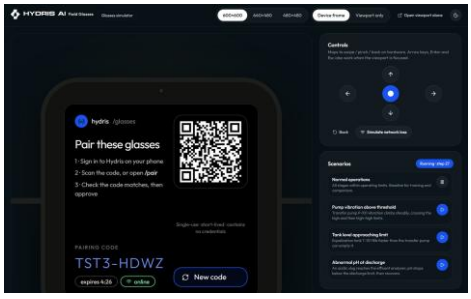


Figure 33. *The browser simulator running the HUD*

Table 27. HUD and phone mode compared

Aspect	HUD mode	Phone mode
Target	Display glasses, simulator	Bridge app on the phone paired with Ray-Ban Meta
Navigation	Six keys, swipes, `hydris:input` from the simulator	Bottom tab bar plus touch
Camera	Simulated or device camera	Glasses camera through the bridge
Voice	Browser speech	Glasses microphone through the bridge, browser fallback

4.5 Screen map and navigation

The app has twelve top-level screens (FR-059) plus detail screens for an alert, an inspection, a task, an asset and an SOP. Home is the menu: its item order and visibility come from the UI profile, and Home and Status (device and log out) can never be switched off.

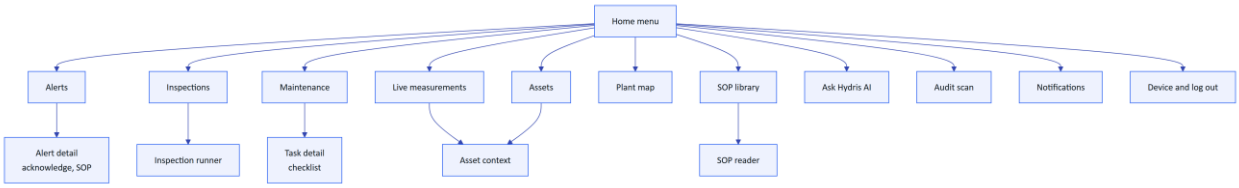


Figure 34. *Screen map (GlassesApp.tsx)*

Navigation is a stack. Every push also writes a browser history entry, so the hardware or browser back button walks back through glasses screens and never leaves the app; at the bottom of the stack it returns to Home. In HUD mode, up and down move the focus, select activates it, left or back pops the stack.

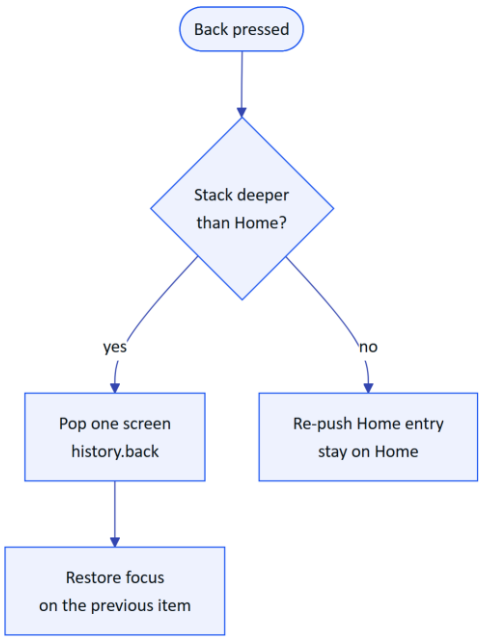


Figure 35. *Back navigation never leaves the app*

4.6 Lifecycle and pairing

The viewport moves through phases before the operator reaches Home. On boot it looks for a device credential; without one it shows the pairing screen. If the server reports the device as revoked, suspended or unassigned, the app shows that state plainly instead of failing silently.

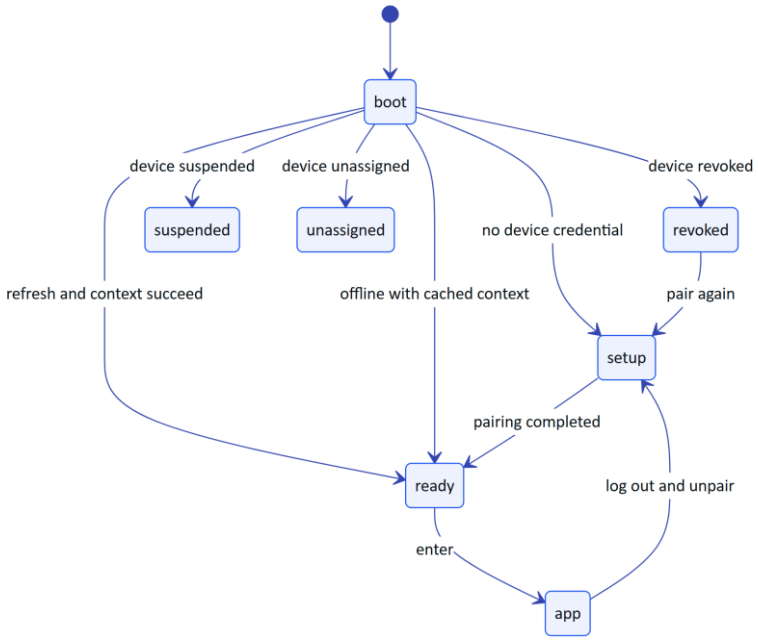


Figure 36. *Viewport phases from boot to the app*

Pairing follows the shape of the OAuth device authorization grant (RFC 8628). The glasses never type a password: they show a short code and a QR link, a signed-in supervisor approves on any phone or browser, and the glasses claim their own credential with a secret that is never displayed.

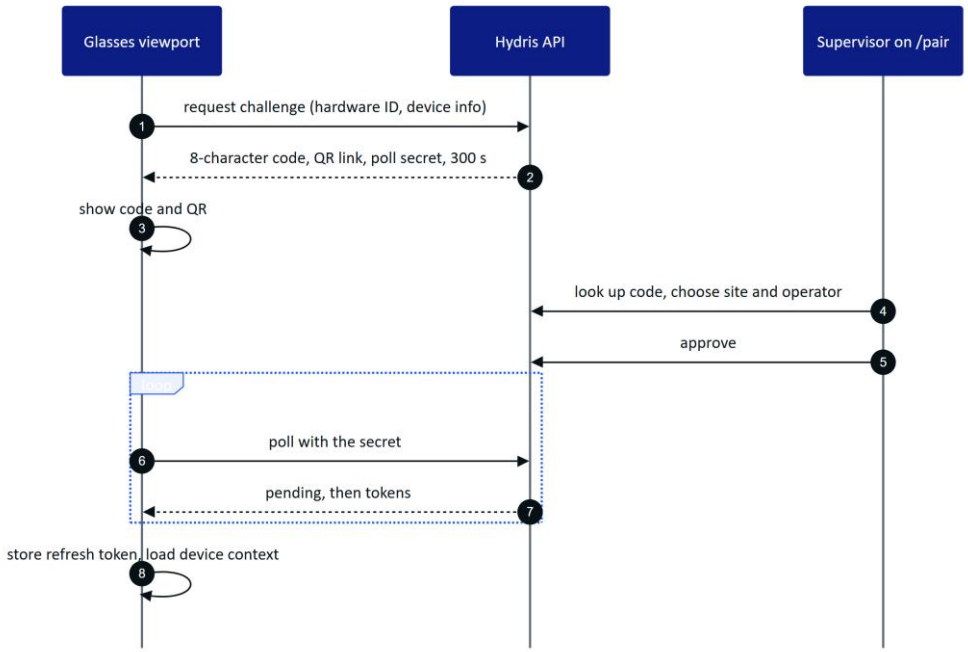


Figure 37. *Pairing from the glasses side*

The code uses 8 characters from an alphabet without 0, O, 1, I, L or U. Polling faster than 3 s returns `slow_down`; an expired code cannot be approved. After pairing the device holds a 15-minute access token and a 30-day rotating refresh token with reuse detection (FR-051). The viewport keeps its hardware ID (`hg-` plus a random UUID) and refresh token in local storage, which the code marks as the boundary for a hardware-backed store.

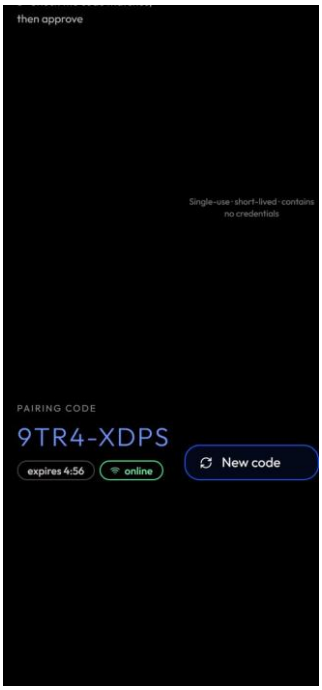


Figure 38. *Pairing code and QR in phone mode*

Log out and unpair lives on the Device screen behind a confirmation ("Log out and unpair these glasses?"). It ends the device's server sessions, after which the refresh token is rejected and anyone can pair the glasses again (FR-058). Revocation from the console takes effect on the next request and within 30 s on the live event stream (FR-052).

4.7 Core experience

Home

Home greets the operator by first name ("...'s shift") with the site name, then shows profile-controlled widgets: the next inspection card, a critical alarm badge, three measurements at a glance and checklist progress. Below is the menu, each item with a live count (open alerts and critical count, inspections assigned, sensors on site). Values outside their threshold are tinted.

Alarms

Alerts lists open alarms by severity. The detail shows the value, the rule, the recommended action and the linked SOP (FR-112). Acknowledge is the single primary action; it needs a connection, says so when offline, and confirms with "Acknowledged. The equipment state is unchanged."

Inspections with an offline queue

The inspection runner walks the operator one step at a time through check, reading, choice and text steps. A reading outside its expected range, a failed check or an abnormal choice marks the step abnormal automatically. Photo steps show "attach from console" today (FR-045 Partial).

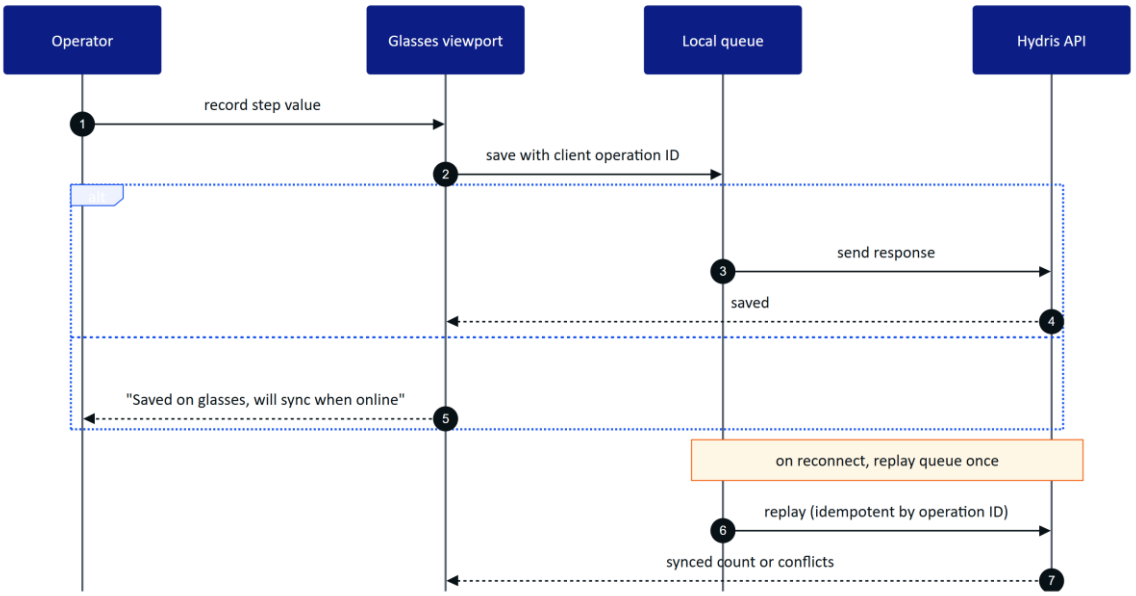


Figure 39. *Recording a step online or offline (data.ts)*

Every response is queued first and then flushed, so the same path serves online and offline work (FR-041, NFR-025). The status bar shows "N queued" and an offline strip states the age of cached data. On reconnect the app reports "Back online, N queued steps synced", or names the first conflict if the server rejected a step. Submitting an inspection needs a connection and stays disabled until every required step is complete.

SOPs and assets

The SOP library lists only approved documents (FR-111); a draft never reaches the glasses. An SOP opened once is cached and opens again with no network (FR-113). Asset context shows the asset's live readings with freshness and a short trend from the last 30 values, and links to Assist for grounded procedure passages with a safety notice (FR-070).

Audit scan entry point

Audit scan turns a paper log into a verified record. On the glasses it opens a guided A4 capture: live page detection, auto-capture when the page is aligned, steady and sharp, perspective correction, two on-device OCR passes, review of up to six pages, then upload for verification in the console (FR-060, FR-061, FR-067). In the bridge the glasses camera feeds the guidance and the capture button takes the photo; in the simulator a simulated camera walks a sample sheet through too close, too far, tilted and square. The full pipeline is described in the audit scan part of this book.

4.8 UI profiles

A UI profile is a versioned description of what a pair of glasses may show and do: enabled screens and their order, home widgets, allowed actions, a measurement filter and a text scale (0.9, 1, 1.15 or 1.3, NFR-019). Profiles are drafted, published and rolled back in the console. Assignment resolves with precedence device, then user, then role, then organization; the most specific published profile wins entirely and is never merged.

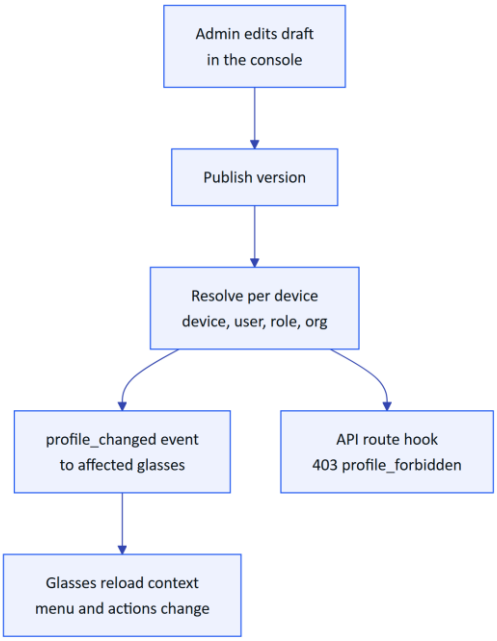


Figure 40. *How a profile reaches the glasses and is enforced*

The server enforces the same profile it sends: a device whose profile hides alarms gets `403 profile_forbidden` from the alarms API even if the operator's role allows it. On the glasses, a blocked action shows a dashed "not permitted by your profile" note rather than disappearing silently. The console can also push a draft into a live preview of the viewport before publishing.

Table 28. **What a profile controls**

Control	Options
Screens	12 screens; Home and Device are always on
Widgets	Next inspection, critical alarm badge, measurements at a glance, checklist progress
Actions	Acknowledge alarms, record and submit inspections, tick checklists, ask AI, open SOPs, view map, mark notifications read
Measurements	Allowed measurands, order by asset type
Text scale	0.9, 1, 1.15, 1.3

4.9 Voice entry point

"Hey Hydris" opens the voice assistant from any screen, and so does the Hey Hydris tab in phone mode or a "Hey Meta, start Hydris" invocation in the bridge (FR-077 Partial). The engine order is the glasses microphone through the bridge, then the browser Web Speech API, then typed questions. Answers are one or two sentences, spoken, and shown with the facts behind them (FR-071, FR-075, NFR-022).

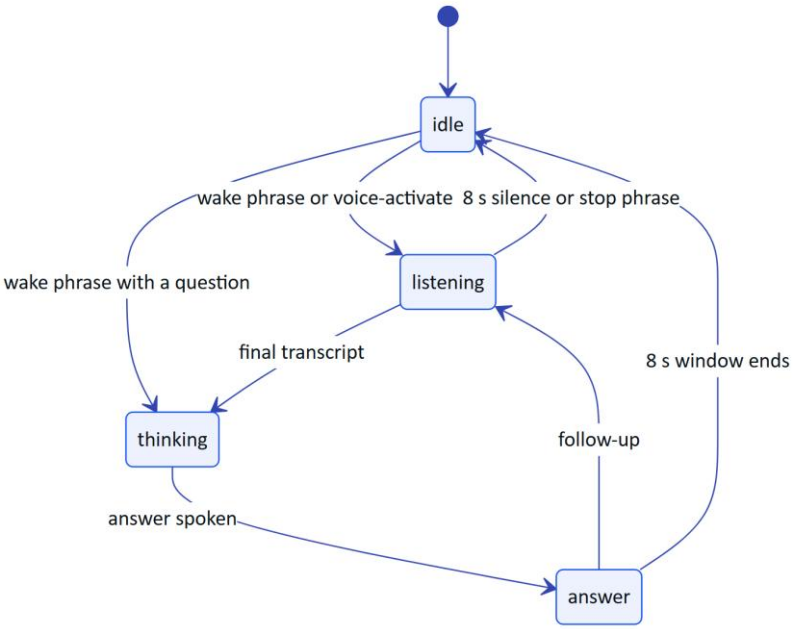


Figure 41. *Voice assistant states (VoiceAssistant.tsx)*

Offline, demo and cached answers come from the same rules engine the API uses, so the assistant never guesses: an unknown question returns "unknown" (FR-072). The assistant has no write tools (FR-078).

4.10 Requirements

Table 29. Glasses app requirements (IDs from the PRD)

ID	Requirement	Pri	Status
FR-059	Heads-up app, 12 screens, six-key input, phone layout	M	Built
FR-050	Pairing by 8-character code and QR, 300 s TTL, 3 s polling	M	Built
FR-051	Device tokens: 15-minute access, 30-day rotating refresh	M	Built
FR-052	Suspend, revoke, reassign; effective within 30 s on the stream	M	Built
FR-058	Log out and unpair from the glasses menu	M	Built
FR-054	UI profiles: draft, publish, rollback, live push	S	Built
FR-041	Inspection execution with offline queue and idempotent replay	M	Built
FR-042	Submit inspections; returned ones can be restarted	M	Built
FR-045	Photo steps captured with the glasses camera	S	Partial
FR-112	Alarm detail shows and opens the linked SOP	M	Built
FR-111	Only approved documents reach the glasses	M	Built
FR-113	Opened SOPs cached for offline reading	S	Built
FR-060	Guided A4 capture with auto-capture	M	Built
FR-071	Hey Hydris wake phrase and short spoken answers	M	Built
FR-077	"Hey Meta, start Hydris" opens the assistant	C	Partial (bridge only)
NFR-019	Text scale per profile	M	Built
NFR-020	Additive-display friendly design	M	Built, to verify on hardware

ID	Requirement	Pri	Status
NFR-025	Inspection responses survive connectivity loss	M	Built
NFR-026	Cached context shown as cached, never as live	S	Built

4.11 Success metrics

Table 30. Glasses app metrics (targets, not measured results)

Metric	Target	Source
Home ready after wake	2 s from cached context, fresh data within 5 s	NFR-005 (target)
Queued steps lost after reconnect	Zero	NFR-025 (Playwright journey passes today)
Voice answer time, end of speech to spoken answer	3 s rules engine, 8 s model engine	NFR-003 (target)
Profile change visible on glasses	Without a reload	FR-054 acceptance
Rounds completed on the glasses in pilot	To be set with the pilot customer	Pilot plan

4.12 Status, gaps and next steps

The web viewport is Built and runs today in the simulator and inside the bridge. It has not yet run on a real glasses display.

> **WARNING**
> Hardware gestures (neural band, temple touchpad) are not delivered to the web runtime yet; `input.ts` keeps a placeholder. The viewport's refresh token sits in local storage until a hardware-backed store exists.

- ☐ Run the HUD on display glasses (Vuzix M400, RealWear Navigator 520, Meta Ray-Ban Display), Planned.
- ☐ Wire glasses camera capture into inspection photo steps (FR-045).
- ☐ Hardware-backed credential storage for the viewport.
- ☐ Inspection execution in the Android app with the same offline guarantees (FR-044, Planned for pilot).
- ☐ Measure NFR-003 and NFR-005 on hardware during the first pilot.

5 Hydris Field Android app

5.1 Purpose and problem

Operators in effluent and sewage treatment plants walk rounds with a phone in their pocket and, increasingly, Ray-Ban Meta glasses on their face. Until now the only app that talked to those glasses was Meta AI, a consumer assistant that knows nothing about the plant. Hydris Field is the native Android companion that takes over the working day: one Hydris account, the plant dashboard, alarms and work, audit scanning, the "Hey Hydris" assistant and the glasses link, all in one app (`apps/android`, Kotlin and Jetpack Compose).

> **IMPORTANT**
> Hydris Field does not replace the Meta AI app, and cannot. Meta's Wearables Device Access Toolkit (DAT) requires Meta AI for Bluetooth pairing, firmware updates, Developer Mode and the approval prompts for camera and microphone. Meta AI stays installed as the connection broker; after a one-time setup the operator opens only Hydris Field.

5
Bottom tabs
Home, Work, Scan, Hydris, Glasses

7
Navigation routes
`Root.kt` NavHost

2
OCR passes per page
on the phone, before upload

12
Minimum Android version
`minSdk` 31

5.2 Users and jobs

Table 31. Who uses the app and for what

User	Job the app does for them
Field operator	See what needs attention on my sites, act on alarms where I stand, scan paper logs, ask questions hands-free
Maintenance engineer	See open work orders and tick checklist items in the field
Shift supervisor	Keep plant status, alarm trend and asset health in my pocket
Prospect or partner	Try the full product in minutes with a named demo account

All of them already hold a Hydris console account. The app adds no new account type and the server needed no mobile-specific authentication.

5.3 Scope

Table 32. In scope and out of scope for this release

In scope	Out of scope
Sign-in, demo mode, session restore	Administration of members, roles, sites and rules (console)
Home dashboard, Work tabs, alarm and sensor detail	Reviewing and approving audit documents (console)
Audit scan from glasses, phone camera or import	Equipment control of any kind
Hey Hydris assistant	A heads-up display on Ray-Ban Meta (they have none)
Glasses connect, test and register	iOS native app
Realtime alarm notifications while the app runs	Replacing the Meta AI app

5.4 Navigation

The app is one activity with a bottom navigation bar. `Root.kt` first decides between restoring a session, Login and the signed-in shell; each tab keeps its scroll position when the user switches away and back.

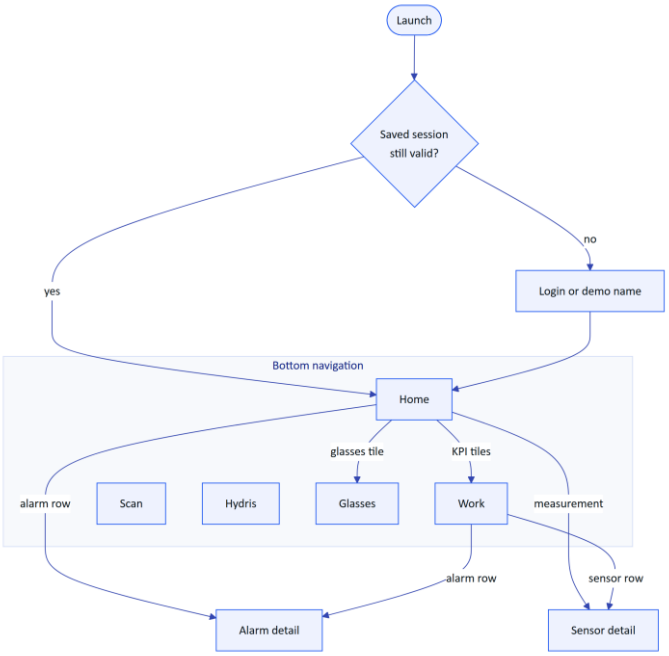


Figure 42. *Navigation map of the five tabs and the two detail screens*

Table 33. What each tab is for

Tab	What the user does there	Part of this book
Home	Reads the plant status at a glance	This part
Work	Works alarms, inspections, maintenance, sensors, audit documents	This part
Scan	Turns a paper record into a document for verification	Audit document scanning
Hydris	Asks questions by voice or text	Hey Hydris voice assistant
Glasses	Connects, tests and registers Ray-Ban Meta glasses	Ray-Ban Meta glasses

5.5 Session and sync

Sign-in and restore

The operator signs in with the console email and password, or taps "Enter demo mode" and types a name of two or more characters to get a personal synthetic account. The short-lived access token lives in memory only; the rotating refresh cookie is stored encrypted with AES-256-GCM under a non-exportable Android Keystore key (`SecureStore.kt`). Killing and reopening the app restores the session without a password.

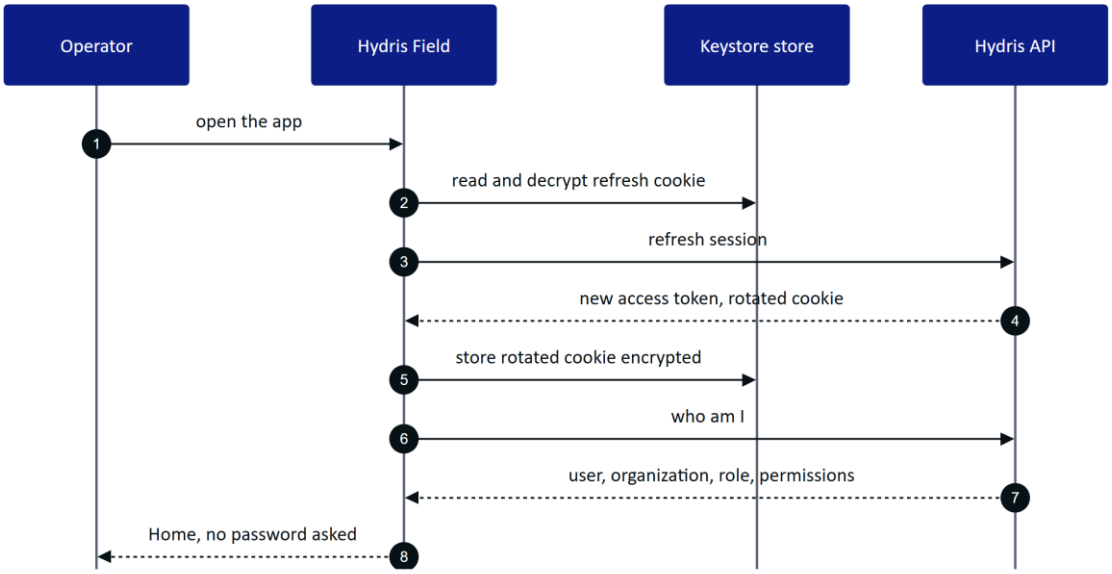


Figure 43. *Session restore when the app starts*

If the cookie is missing or refused, the operator lands on Login. Later, any 401 triggers one refresh; refreshes are serialized by a lock so parallel screens never present an old cookie, which the server would treat as reuse and revoke. If refresh fails the user sees "Session ended. Please sign in again."

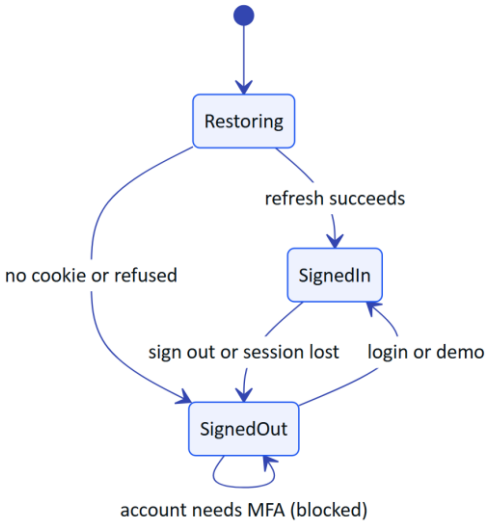


Figure 44. *Session states in Root.kt*

Keeping screens live

Two mechanisms keep data current without a local database. Screens poll while visible (Home and alarms every 15 s, sensors every 10 s, audit documents every 20 s), and a server-sent event stream pushes alarm events. The stream resumes with `Last-Event-ID`, so events missed during a network drop are replayed.

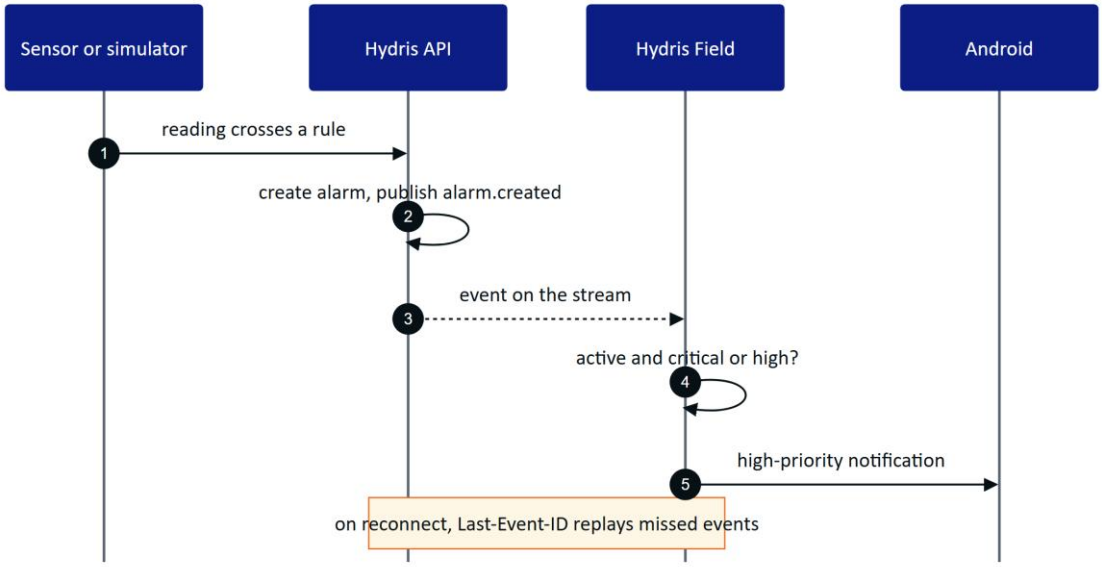


Figure 45. *Realtime alarm notification from reading to phone*

DECISION

Polling plus the event stream is deliberate for the pilot. Every screen stays correct after a network drop without an offline cache. The cost is extra requests while a screen is visible; the Planned offline outbox and cache will change this.

5.6 Home dashboard



Figure 46. *Home: status line, KPI tiles, alarm trend and asset health*

Home answers "what needs me now?" from one call to the same overview endpoint the console uses (FR-089). Top to bottom: a greeting with the operator's first name and a "Demo" chip for demo tenants; a status line ("2 critical alarms need attention." or "All quiet on your sites."); four KPI tiles (open alarms, inspections due and overdue, maintenance, field glasses connected); a 14-day alarm trend; an asset-health donut; live measurements with a freshness dot, age and a

"synthetic" label for simulator data; recent alarms; and the last eight activity events. Tiles and rows are shortcuts into Work, Alarm detail, Sensor detail and Glasses.

A failed refresh keeps the last overview on screen. Missing data shows "No data", never zero, and stale values carry their state.

5.7 Work

Work holds everything open in five tabs, each filtered by the operator's site scope on the server.

Table 34. Work tabs

Tab	What the operator sees	What the operator can do	Status
Alarms	Severity, title, state, age, assignee; critical alarms outlined in orange	Open alarm detail and act	Built
Inspections	Title, status, site, overdue marker, step progress	Read only	Partial: running an inspection on the phone is Planned (FR-044)
Maintenance	Priority, status, checks done; expands to the checklist	Tick checklist items	Built; start, hold and complete Planned
Sensors	Name, tag, freshness, age, open alarms, value	Open sensor detail with 6 h, 24 h, 7 d chart	Built
Audit docs	Status, values, engine agreement, review progress	Read only (review is in the console)	Built

Working an alarm

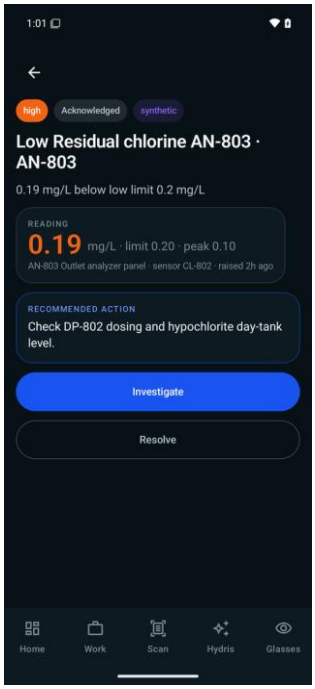


Figure 47. *Alarm detail: reading against limit, recommended action and the next step*

Alarm detail shows the trigger value in the severity colour against its limit and peak, the asset and sensor, a recommended action, and buttons for the next lifecycle step. Resolve asks for a note ("What was done"), because the API refuses a resolution without one (FR-031); the button stays disabled until the note is filled.

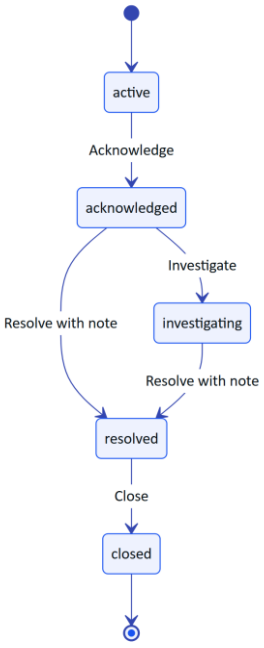


Figure 48. *Alarm lifecycle as the operator moves it on the phone*

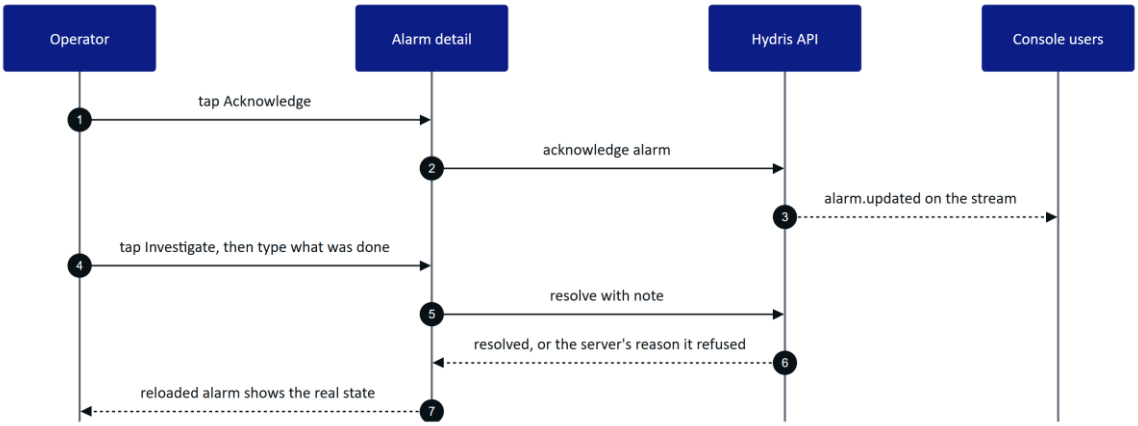


Figure 49. *Acknowledge, investigate and resolve from the field*

Server rules always win: if the operator lacks the permission, or a colleague moved the alarm first, the server's message is shown and the screen reloads the true state.

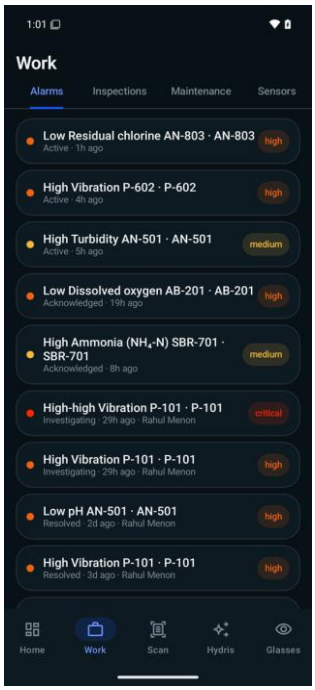


Figure 50. *Work on the Alarms tab*

5.8 Glasses tab

The Glasses tab is the one place to link Ray-Ban Meta glasses through the Meta AI app, test the camera and microphone, and register the glasses as a Hydris device so scans are attributed. If Meta AI is not installed, "Connect glasses" stops early and says so. The full flows are in the Ray-Ban Meta glasses part. Every glasses feature has a phone fallback: phone camera, phone microphone, phone speaker.

5.9 Permissions

The app asks for each permission in context and keeps working when one is denied.

Table 35. Permissions and what happens if denied

Permission	Asked when	If denied
Nearby devices (Bluetooth connect)	App start	Glasses unavailable; phone features work
Notifications	App start	No alarm notifications
Camera	"Scan with phone camera"	Import a photo still works
Microphone	Turning Hey Hydris on	Typing still works
Glasses camera (in Meta AI)	First glasses preview or photo	Clear message; phone camera works
Glasses microphone (in Meta AI)	First glasses listening	Falls back to the phone microphone

> **WARNING**
> Bluetooth and notification permissions are requested on every app start, so a user who denies them is asked again each time. Asking once and explaining in Settings is Planned.

5.10 Performance and native principles

- **Native, not a wrapped website.** Kotlin, Compose and Material 3, dark only, in the shared Hydris palette; charts are drawn directly on a canvas with no chart library.

- **Work on the device first.** Page detection and both OCR passes run on the phone; no image leaves it until "Send for verification".
- **Fail fast, wait when it matters.** 15 s connect timeout; 90 s read timeout for scan uploads; the event stream has no read timeout and relies on server heartbeats.
- **Tolerant of server change.** Unknown JSON fields are ignored, so new API fields do not break older builds.
- **Glanceable for gloves and glare.** Large KPI numbers, 52 to 54 dp primary buttons, explicit empty states ("No open alarms.").
- **One app-wide set of engines.** Session, glasses link and voice live in singletons created once at start; ViewModels, Hilt and Room are Planned.

5.11 Requirements

Table 36. Android app requirements (FR-090 block)

ID	Requirement	Pri	Status
FR-090	Console sign-in; session restored from a Keystore-encrypted refresh cookie	M	Built
FR-091	Demo sign-in with a name	M	Built
FR-092	Home: greeting, 14-day trend, asset health, live measurements with freshness	M	Built
FR-093	Work tabs scoped to the user's sites	M	Built
FR-094	Alarm detail with lifecycle actions	M	Partial: resolve note added; buttons not yet driven by the API's allowed actions
FR-095	Sensor detail with 144-point history and typical range	S	Built
FR-096	Audit scan from glasses, phone or import	M	Built (phone); glasses path Partial
FR-097	Realtime notifications for critical and high alarms while the app runs	M	Built
FR-098	Offline outbox with idempotent replay	S	Planned (GA)
FR-099	Complete MFA sign-in in the app	S	Planned (Pilot)
FR-089	Home and console overview from the same endpoint	M	Built

5.12 Success metrics

Table 37. Android success measures (targets, not measured results)

Measure	Target
Opening Scan to "Sent" for one A4 page	Under 60 s on a mid-range phone
Scans that auto-capture without "Capture now"	Above 70 %
Critical alarm to phone notification	Under 5 s while the app runs
Spoken answer after the question ends	Under 3 s on 4G
Crash-free sessions in pilot	Above 99.5 %

The app was verified manually on an emulated Pixel 7 (API 35): login, demo mode, home, assistant, alarm acknowledgement, and a scan import that extracted 10 values including pH 7.3 correctly.

5.13 Status, gaps and next steps

Table 38. Gaps to close before and during the pilot

Gap	Effect today	Plan
No MFA step (FR-099)	Accounts with TOTP must sign in on the web	Code entry screen, Pilot
No offline outbox (FR-098)	Actions need a network	Room cache and outbox, GA
Actions not driven by allowed actions (FR-094)	A button may be offered that the server refuses	Use the API's list, Hardening
No push service	Notifications only while the app runs	FCM, GA (FR-034)
Notification opens the app, not the alarm	Operator must find it	Deep link to the alarm
No automated tests	Regressions caught by hand	Unit and UI tests before pilot
Release builds signed with the debug key	Cannot be distributed outside the team	Release keystore, Play internal track
Glasses path not run on hardware	DAT flows proven only on the mock device	First hardware run, Q4 2026
Glasses registered to the first site	Multi-site users cannot choose	Site picker before multi-site pilots

RISK

Until release signing is in place, no build may leave the team. MFA, release signing and the first hardware run are the three items that gate a pilot handover.

6 Ray-Ban Meta integration

6.1 Purpose and problem

Ray-Ban Meta glasses are the lightest way to give an operator a camera, a microphone and speakers at eye and ear level, but they have no display. Hydris uses them as hands-free sensors for the phone: the glasses camera frames and shoots paper logs, the glasses microphone hears "Hey Hydris", the open-ear speakers play the answer, and the capture button takes the photo. The phone is the screen. The integration uses Meta's Wearables Device Access Toolkit (DAT) 1.0.1 with four modules: core, camera, speech and inputs.

> **IMPORTANT**
> No Hydris build has yet been run against physical Meta glasses. All DAT calls compile and are type-checked against DAT 1.0.1, and the page side of the bridge was tested with a stand-in bridge. Every hardware behaviour here is **Built, not hardware-verified** unless stated otherwise.

4
DAT modules
core, camera, speech, inputs

0
Displays
the phone is the screen

2
Integration routes
native app, WebView bridge

15
Hardware acceptance tests
none run yet

6.2 Users and jobs

Table 39. Users and jobs

User	Job	Requirements
Field operator	Scan a paper log with the glasses camera without holding the phone	FR-055, FR-056
Field operator	Ask by voice and hear the answer on the open-ear speakers	FR-074, FR-077
Supervisor or admin	See the glasses as a managed, revocable device in the console	FR-057
Solutions engineer	Set up and troubleshoot a pilot kit	Setup and troubleshooting below

6.3 Scope

Table 40. Scope

In scope	Out of scope
Ray-Ban Meta Gen 1 and Gen 2 (primary target: Gen 2)	Video recording or upload
Camera preview for guided capture, full-resolution photo	Replacing the Meta AI app
Glasses microphone, phone TTS on the glasses speakers	Hydris UI on the Ray-Ban Meta (it has no display)
Capture button short press	Long press and other gestures
Registering the glasses as a Hydris device	Several pairs of glasses per phone

6.4 What the glasses can do

Table 41. Capability matrix by glasses family

Capability	Ray-Ban Meta	Oakley Meta (HSTN, Vanguard)	Meta Ray-Ban Display
Supported by DAT	Yes (Meta FAQ)	Yes (Meta FAQ)	Yes (Meta FAQ)
Camera stream and photo	Yes	Expected, unverified	Expected, unverified
Microphone through DAT Speech	Yes	Unverified	Unverified
Speakers	Phone TTS over Bluetooth	Phone TTS over Bluetooth	Phone TTS over Bluetooth
Capture button	DAT Inputs	Unverified	Unverified
Display	**None**	**None**	Yes; Hydris use Planned
Hydris status	Primary target, not yet on hardware	Untested	Phone path expected; display Planned

Every glasses feature has a phone fallback: phone camera, phone microphone and phone speaker. An operator without glasses loses nothing but the hands-free convenience.

6.5 The Meta AI app dependency

Meta's documentation is explicit that the Meta AI app must be used to pair the glasses. It pairs them to the phone, updates firmware, enables Developer Mode for unpublished apps, and approves each third-party app and each permission (camera, microphone). Hydris does not try to replace it.

> **DECISION**
> The Meta AI app must stay installed and signed in. Hydris checks for it (package `com.facebook.stella`) and, if missing, tells the operator to install it and pair the glasses there first. Hydris pairing is separate: it makes the glasses a managed device in the customer's organization.

6.6 DAT lifecycle

Both Hydris phone apps follow the same lifecycle. DAT is initialized only after Android grants the Bluetooth permissions; the app then watches registration state, connected devices and device names.

Registration and permissions

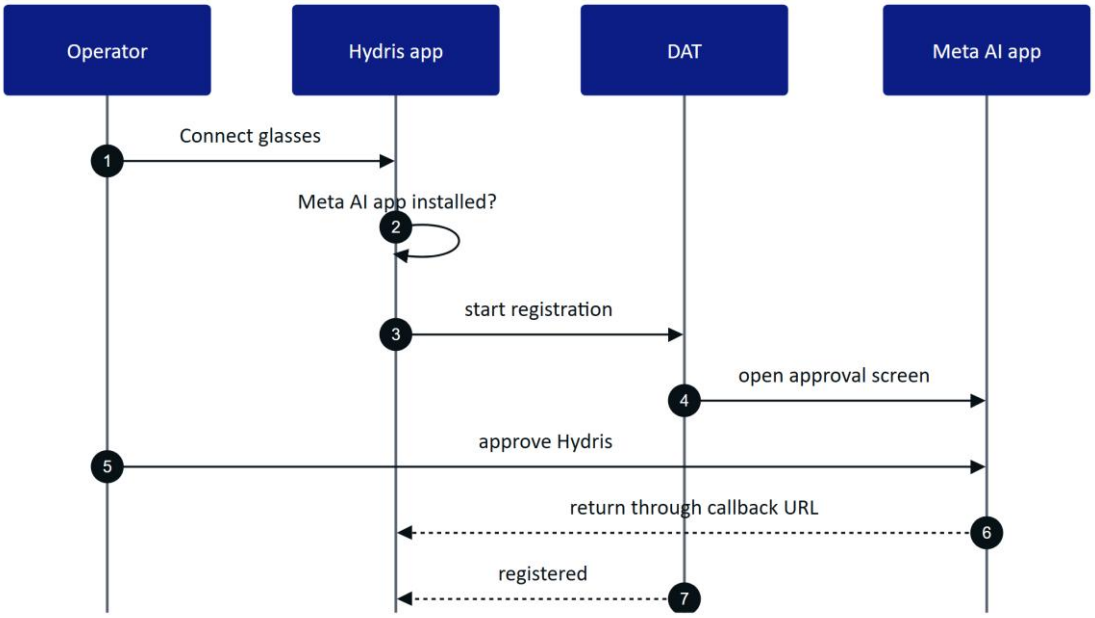


Figure 51. *Registration hands off to the Meta AI app and returns*

Camera and microphone access on the glasses are DAT permissions, separate from Android permissions, and are also approved in the Meta AI app. Hydris asks for one permission at a time, and a cancelled prompt counts as denied.

Session

One device session is shared by camera, speech and inputs. It is created the first time a capability is needed, reused while started, and rebuilt from scratch after it stops (for example when the glasses are switched off).

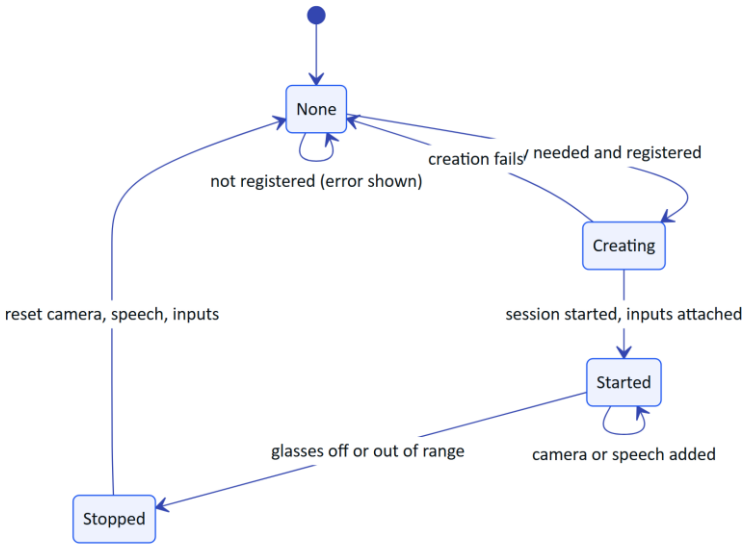


Figure 52. *Device session lifecycle*

Camera stream and photo

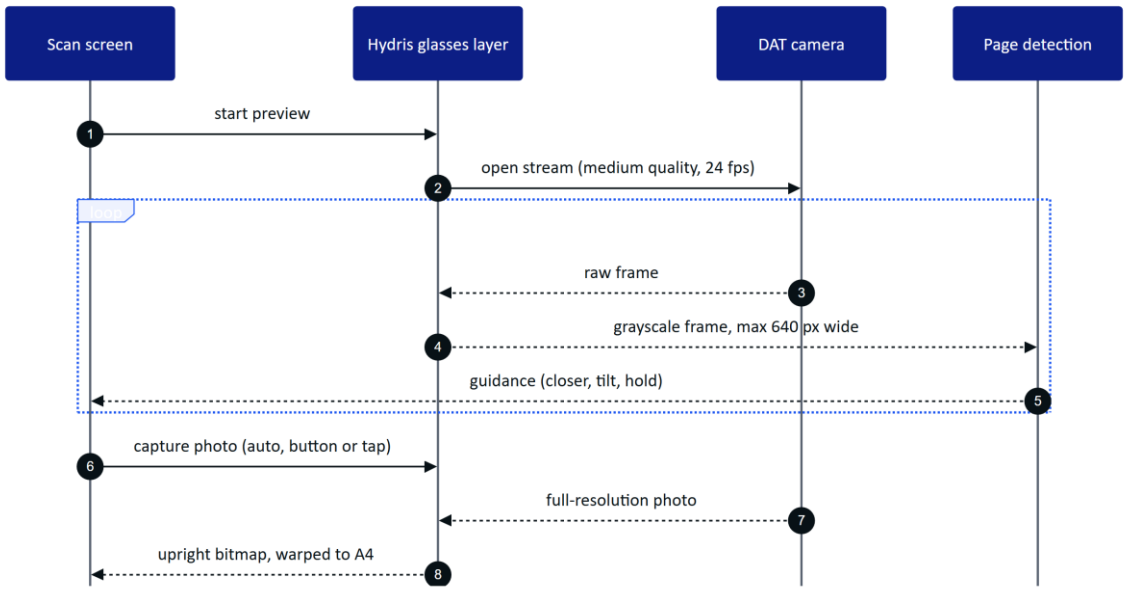


Figure 53. *Guided capture from the glasses camera*

Preview frames are processed in memory for framing only; Hydris reads the luma plane and never stores or uploads frames. The photo is taken only on an active stream, decoded (bitmap or HEIC), rotated upright from EXIF, then flattened to an A4 page of 1654 x 2339 pixels. Images leave the phone only when the operator taps "Send for verification".

Capture button and speech

A short press of the capture button starts a glasses scan from the Scan menu or review, and takes the photo while framing; elsewhere it is ignored. Speech is always on while wanted: transcripts arrive as text, Hydris restarts listening after silence and ignores transcripts while its own answer is playing. Answers are spoken by the phone's text-to-speech and play on the glasses when they are the Bluetooth audio output. Hydris receives transcripts, never audio, and sends only the question text to the assistant.

6.7 Two integration routes

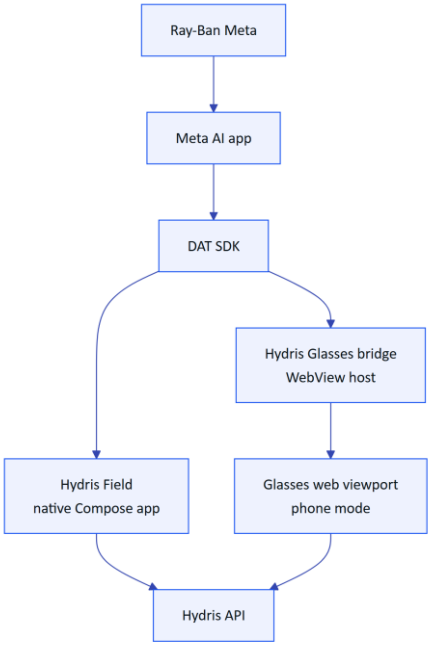


Figure 54. *Native app and WebView bridge share DAT and the backend*

Table 42. The two routes compared

Aspect	Hydris Field (native)	Glasses bridge (WebView)
UI	Native Compose screens	The `/glasses` web viewport in phone mode
How DAT reaches the UI	Kotlin state flows	`HydrisNative` JavaScript interface and page callbacks
Registration as a Hydris device	Self-approved by the signed-in user	Code and QR, approved by a supervisor
"Hey Meta, start Hydris"	Planned	Built
Intended use	**Pilot runtime**	Web viewport and development comparison; not for general distribution

The bridge forwards one JPEG preview frame per 120 ms and full photos at quality 92 to the page; the page gives up on a photo after 20 s. The same page also supports an injected promise API for iOS hosts and a postMessage protocol for any embedding host.

> **WARNING**
> Bridge hardening is Planned: the WebView grants every page permission request and does not restrict navigation away from the Hydris site. Restrict navigation to the Hydris origin before distributing the bridge outside the team.

6.8 Registering glasses as a Hydris device

The Meta AI app pairs the glasses to the phone; Hydris pairing makes them a managed device: assigned to a person and a site, visible to supervisors, revocable, and credited for the scans they capture (FR-057). In the native app the phone plays both roles of the device flow at once.

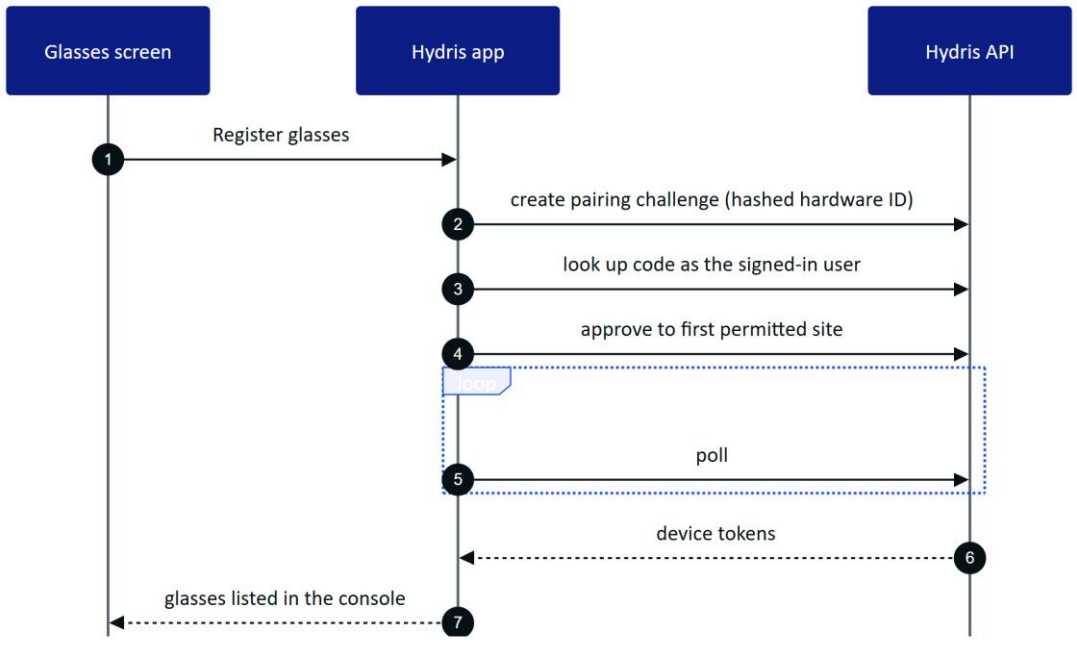


Figure 55. *Self-approved registration in the native app*

The hardware ID is `rbm-` plus a SHA-256 digest of the DAT device identifier, so the raw Meta identifier never leaves the phone and re-registering reuses the same console record. Glasses registered in another organization must be revoked there first (`device_owned_elsewhere`).

> **NOTE**
> The same glasses paired through both routes produce two device records, because the hardware IDs differ. Pilots should use one runtime per pair. The native app assigns the glasses to the first site the user can see; a site picker is Planned before multi-site pilots.

6.9 Setup guide

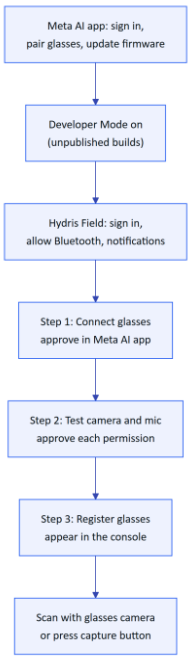


Figure 56. *First-time setup for a pilot operator*

Developer Mode is turned on in the Meta AI app under Settings, App Info, by tapping the app version five times. Speech, Inputs and Voice Invocations are experimental in DAT and must be enabled for the app in Meta's Wearables Developer Center; Hydris treats them as optional and keeps the phone fallbacks. Developer Mode builds use placeholder DAT credentials; production builds use the values issued by the Developer Center.

6.10 Troubleshooting

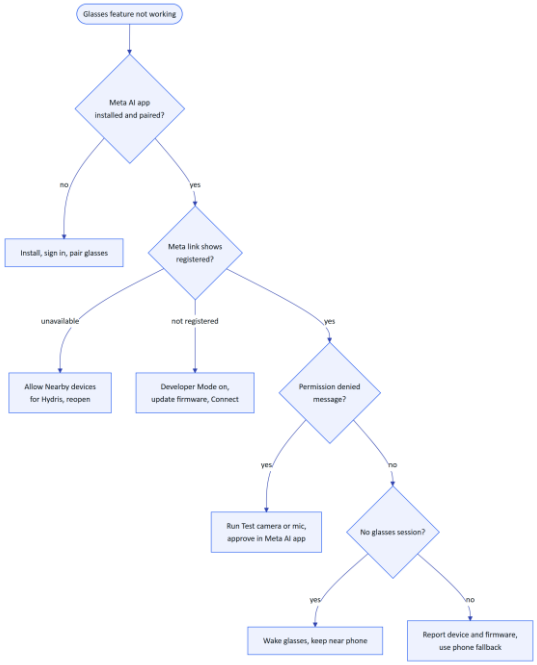


Figure 57. *Troubleshooting decision tree*

Table 43. Other common symptoms

Symptom	Fix
Capture button ignored	Enable Inputs in the Developer Center; open Scan
Answers play on the phone speaker	Reconnect audio to the glasses in Android Bluetooth settings
Preview black or garbled	Report device and firmware; use the phone camera meanwhile
"Your account has no site to assign the glasses to"	Ask an admin for site access
`device_owned_elsewhere`	Revoke the glasses in the other organization

6.11 Requirements

Table 44. Ray-Ban Meta requirements (IDs from the PRD)

ID	Requirement	Pri	Status
FR-055	Glasses camera through DAT: preview stream and full-resolution photo	M	Partial (not on hardware)
FR-056	Capture button takes the photo or starts a scan	S	Partial (not on hardware)
FR-057	Android app registers the glasses as a Hydris device	M	Partial (not on hardware)
FR-074	Glasses microphone with phone fallback; answers on the speakers	M	Partial (phone path Built)
FR-077	"Hey Meta, start Hydris" opens the assistant	C	Partial (bridge only)
NFR-015	Camera only on explicit intent; no continuous recording	M	Built
NFR-028	4-hour shift on Ray-Ban Meta with up to 20 scans	S	Planned (measure)
NFR-029	Android 12 or later	M	Built

6.12 Success metrics

Table 45. Timing targets for the first hardware run (targets, not measured)

Path	Target
First preview frame after opening the glasses scan	Under 3 s with an existing session
Preview rate reaching detection	8 to 10 frames per second
Photo returned after capture	Under 4 s
Speech restart gap after silence	Under 1 s
Battery	4-hour shift with up to 20 scans (NFR-028)

6.13 Limitations and publishing constraints

- **Not yet run on hardware.** Preview stride inference, capture press decoding and speech restart are unverified.
- **The DAT camera stream is not stopped** when the preview ends, only when the session ends; battery and capture LED impact to be checked.
- **Low-resolution preview.** Frames cross the wireless link, so detection runs on small frames; the photo is full resolution.
- **Publishing.** Public publishing of DAT apps is not open yet (Meta FAQ); pilots rely on Developer Mode and Developer Center release channels. Full Developer Center access is limited to countries where Meta AI glasses are supported.
- **Testing.** Hydris has no automated DAT tests yet, and Meta's Mock Device Kit does not support display glasses.
- **Privacy.** Meta glasses show a capture LED that Hydris cannot control; whether it is lit during preview streaming is to be confirmed. Pilot sites should brief staff and mark scan points.

6.14 Hardware acceptance status

Table 46. Hardware acceptance tests (none run yet)

ID	Test	Status
GLS-T01, T02	Connect with and without Developer Mode	Not run
GLS-T03 to T06	Camera permission, guided A4 scan, leaving scan, photo orientation	Not run
GLS-T07, T08	Capture button short and long press	Not run
GLS-T09, T10	Ten minutes of Hey Hydris, answers on the speakers	Not run
GLS-T11 to T13	Register, revoke then upload, glasses off mid-session	Not run
GLS-T14	Repeat camera and voice tests through the bridge	Not run
GLS-T15	Battery over a simulated four-hour shift	Not run

Next steps, in order: the first run on Ray-Ban Meta Gen 2 with this checklist, stopping the DAT stream when the scan screen closes, and bridge hardening (Q4 2026); Oakley Meta validation, Voice Invocations in the native app and DAT mock-device tests (Q1 2027); a Developer Center release channel pilot and a site picker for registration (Q2 2027). Using the Meta Ray-Ban Display screen remains Planned and unverified.

7 Audit document scanning

7.1 Purpose and problem

Treatment plants still run on paper: shift logbooks, round sheets, lab reports, permits, calibration certificates. These records are the evidence a regulator or auditor asks for, yet they are hard to search, easy to lose, and slow to copy into a system by hand, where a moved decimal point turns pH 7.1 into 71. Audit scanning turns a paper record into a verified, searchable digital document. The design rule is dual verification: a machine cross-check of every value between independent readings, then a human review of every value and an independent approval by a second person.

2
OCR passes per page
original and enhanced, on device

6
Pages per document
each under 900 KB

4 eyes
Verify and approve
two different people

10
Values from the demo sheet
pH 7.3 read correctly

7.2 Users and jobs

Table 47. Who takes part in a scan

User	Job
Field operator	Capture the paper record where it is, with glasses or phone, in under a minute
Shift supervisor (reviewer)	Confirm, correct or flag every value against the paper, then verify
Plant manager (approver)	Approve or reject a verified document; never the same person as the verifier
Compliance auditor	Download a searchable PDF with the full verification trail

7.3 Scope

Table 48. In scope and out of scope

In scope	Out of scope
Capture from glasses, phone camera, imported photo, console	Signing on the device: devices capture, people verify
Guided A4 framing, auto-capture, flattening	Electronic signature with re-authentication (Planned, FR-068)
Two OCR passes, structuring of fields, tables, figures	Handwriting-specific recognition models
Plausibility checks and optional vision engine	Bulk upload of existing PDF archives
Field review, verify, approve, PDF with trail	Writing values into plant records automatically

7.4 From paper to verified PDF

The whole journey has three halves that never mix: the device captures and reads, the server structures and cross-checks, and people verify and approve.

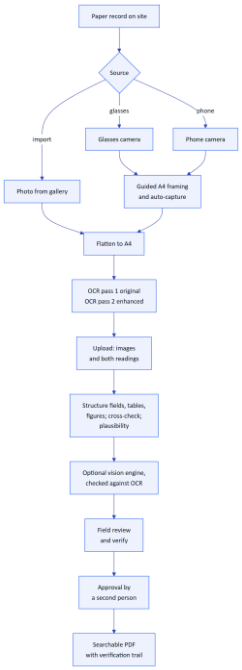


Figure 58. *End-to-end flow from paper record to approved PDF*

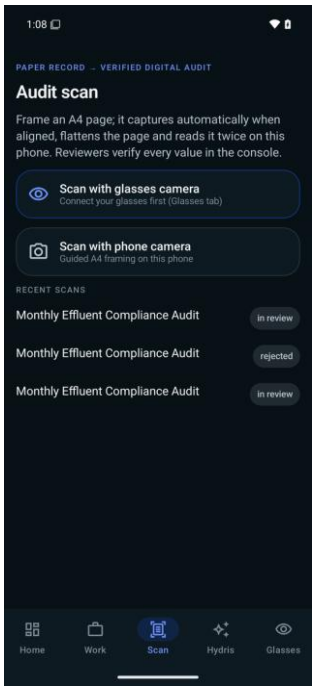


Figure 59. *Scan menu: glasses camera, phone camera or import*

Who does what

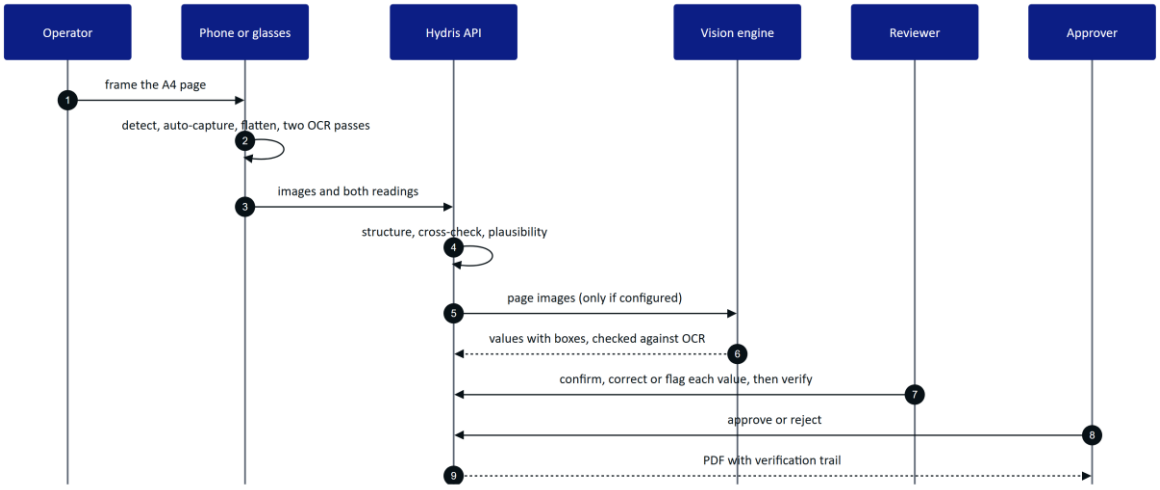


Figure 60. *Capture, machine cross-check, review and approval*

The operator sees "Sent · awaiting dual verification" with the detected title, value and table counts, and the engine agreement. Review and approval happen in the console.

7.5 Guided capture

Framing a sheet on glasses, with no display, is the hard part. The device checks each preview frame (at most every 90 ms) and speaks or shows one instruction at a time, in a fixed order: there is no point asking the wearer to straighten a page that is half out of frame. The phone, the glasses web viewport and the console use the same detection code and thresholds.

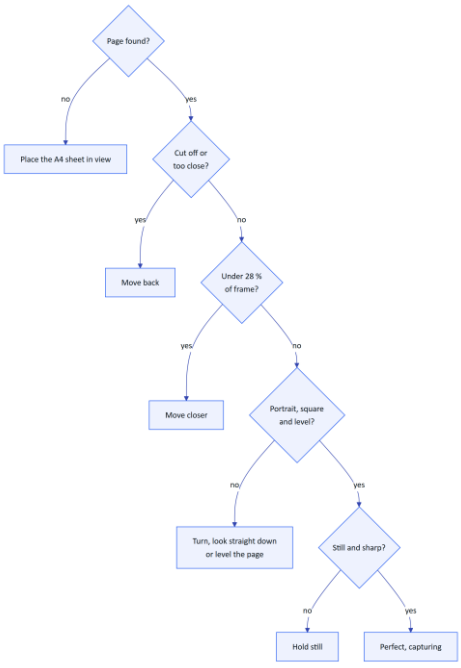


Figure 61. *Guidance order: the first failing check sets the message*

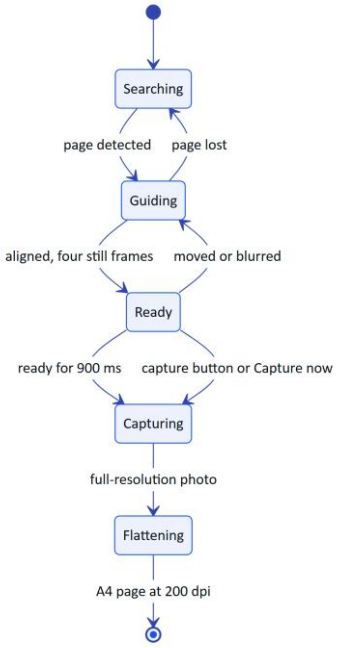


Figure 62. *Guided capture state machine with auto-capture*

On capture the device takes a full-resolution photo, finds the page again, and warps it to a flat A4 image (1654 x 2339 pixels, 200 dpi). A short press of the glasses capture button takes the photo during framing.

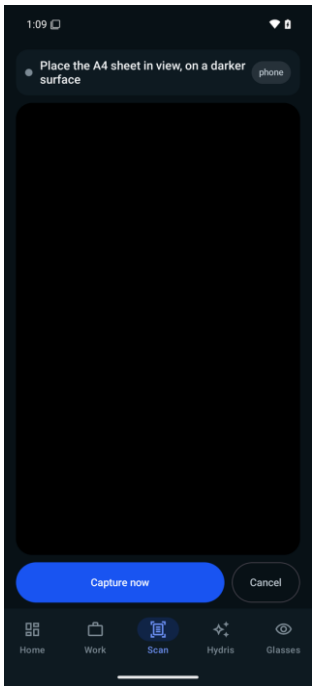


Figure 63. *Guided capture waiting for an A4 sheet*

7.6 Reading the page twice

Each page is read twice on the device before anything is sent. Pass 1 reads the page as captured. Pass 2 reads an enhanced copy: contrast stretched, adaptively thresholded, with table rulings removed so borders do not merge cells. Two independent readings let the server tell a confident value from a lucky one. Up to six pages make one document; the review screen shows each page's confidence and the label and value pairs found.

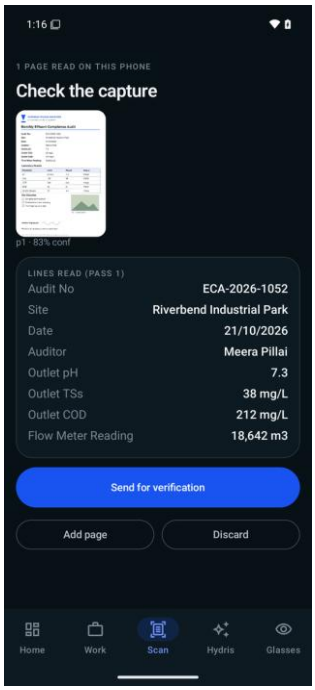


Figure 64. *Review: confidence per page and the label and value pairs found*

7.7 Structuring and cross-checking

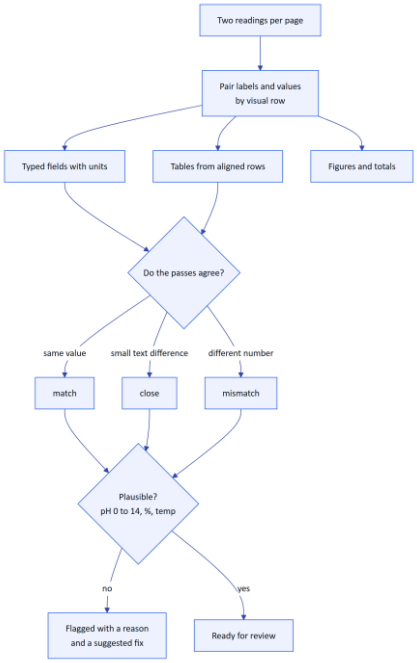


Figure 65. *How the server turns two readings into reviewable items*

Each value carries an agreement level and keeps its position on the page. Plausibility checks run even when both passes agree, because both can be wrong in the same way.

Optional vision enhancement

When an Anthropic credential is configured, the server sends the page images to a vision model that returns fields, tables and figures with boxes. Each vision value is then checked against the OCR text inside its own box; a value the OCR does not support is never marked as agreed. Without a key the OCR result stands and the document says vision is unavailable. The vision model adds reach; it never adds unchecked trust.

7.8 Accuracy design

> **DECISION**
> The primary quality metric is the silent error rate: a value marked as agreed and not flagged, but wrong. A system that is 85 % accurate and flags what it is unsure of is safer than one that is 95 % accurate and confidently wrong on the rest.

Table 49. How the design protects the numbers that matter

Risk	Safeguard
Decimal point dropped or moved	"7.4" against "74" is always a mismatch; thousands separators ("18,857" and "18857") are not
Both passes misread the same way	Range checks still flag pH 71 as implausible, and offer a matching labelled value as the fix
Letters and digits confused	O and 0 are folded inside numbers before comparison
Labels and values read as separate columns	Paired again by visual row (common with ML Kit)
Vision model invents a value	Checked against OCR text in the value's box
Reviewer overwrites the machine value	The first extracted value is always kept beside the correction

7.9 Review, verify and approve

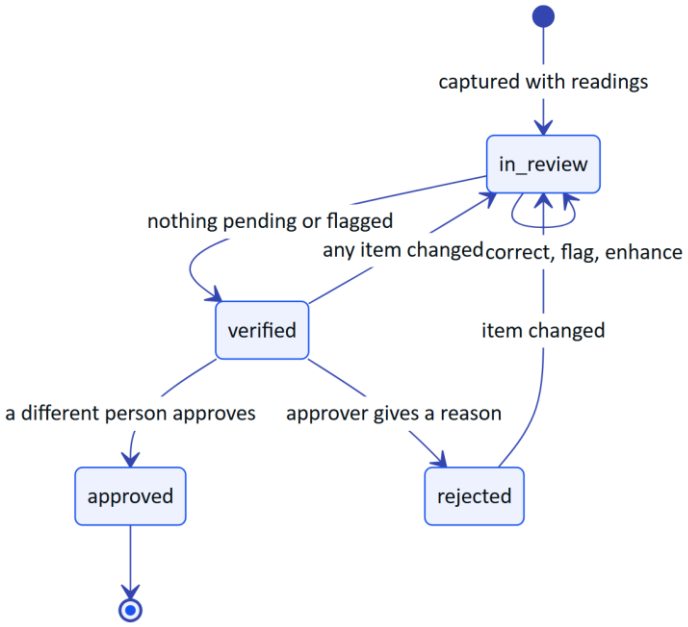


Figure 66. *Audit document lifecycle and the four-eyes rule*

The reviewer works field by field against the paper and can bulk-confirm values where at least two engines match with confidence of 0.8 or more. Verify is blocked while anything is pending or flagged. Approval must come from a different person in a customer organization; demo tenants allow self-approval and record it as such. Only a signed-in person in the console can verify or approve: a device never can.

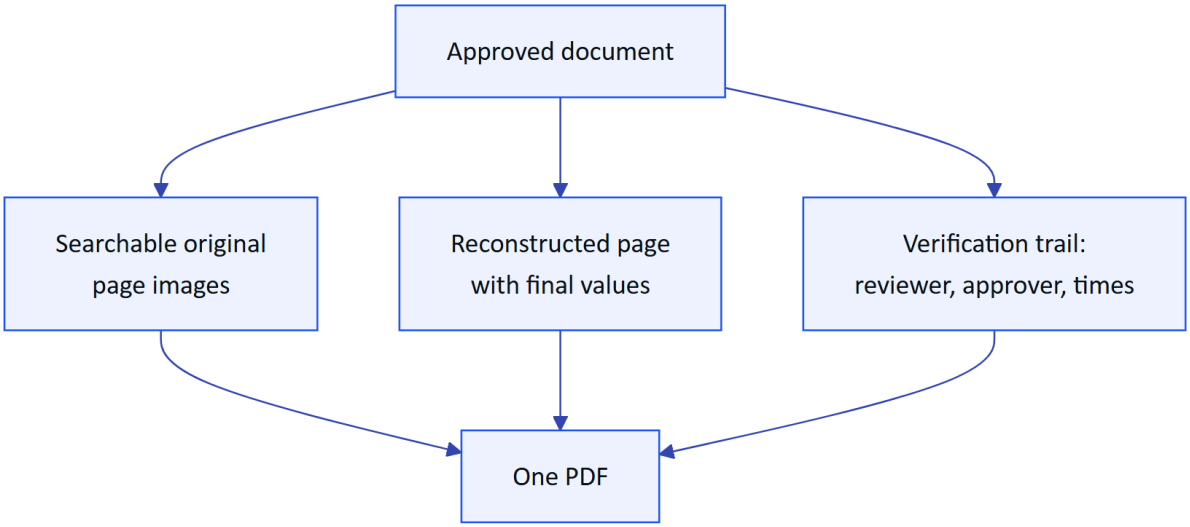


Figure 67. *The approved output an auditor receives*

Every review action is appended to the document's event history, and concurrent edits are rejected with a revision check, so two reviewers cannot silently overwrite each other.

7.10 Requirements

Table 50. Audit scanning requirements

ID	Requirement	Pri	Status
FR-060	Guided A4 capture with live detection, auto-capture, perspective correction	M	Built (web and Android, same thresholds)
FR-061	Dual-pass on-device OCR	M	Built
FR-062	Fields, tables, figures with plausibility checks	M	Built
FR-063	Optional vision engine cross-checked against OCR in each box	S	Built (needs a key)
FR-064	Field review; original kept; verify needs zero pending and flagged	M	Built
FR-065	Independent approval (four eyes)	M	Built
FR-066	PDF with trail, searchable original, reconstructed page	M	Built
FR-067	Up to 6 pages, each at most 900 KB, JPEG or PNG	M	Built
FR-068	Electronic signature with re-authentication	S	Planned (Advanced)
FR-069	Bulk-confirm values where two engines agree confidently	S	Built
FR-096	Scan from glasses, phone or import in the Android app	M	Built (phone); glasses Partial
FR-055	Glasses camera capture through DAT	M	Partial (not run on hardware)

7.11 Success metrics

Table 51. Scanning quality targets (targets, not measured results)

Metric	Pilot target
Fields where the engines agree, typed forms	90 % or more

Metric	Pilot target
Fields a reviewer corrects	Under 5 %
Silent error rate	Under 0.5 %
Decimal correctness	99 %
Planted implausible values flagged	100 %
Capture to review-ready per page	15 s or less
Capture to approval, median	Under 24 hours

EXAMPLE

Demo result (manual verification, emulated Pixel 7 with API 35): a printed lab sheet imported into the Android app was read correctly, including pH 7.3, and 10 values were extracted and uploaded for review. This is one sheet, not a benchmark.

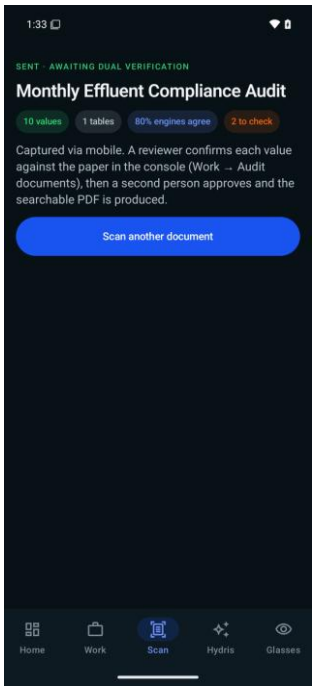


Figure 68. *Sent for verification: values, agreement and next steps*

7.12 Status, gaps and next steps

- **Benchmark not yet run.** A 120-page ground-truth set (printed lab reports, filled templates, handwritten registers, complex tables, degraded captures) is designed; until it runs, every accuracy number above is a target.
- **Glasses capture unproven on hardware.** The DAT path works on Meta's mock device; the first run on Ray-Ban Meta is scheduled for Q4 2026.
- **Electronic signature** with re-authentication and signature meaning (FR-068) is Planned for the Advanced phase.
- **Vision processing consent.** Before a customer pilot the tenant must agree to vision processing and the deployment must record where inference runs; until then pilots run on OCR alone.
- **Page images** move to object storage during Hardening.

8 Hey Hydris voice assistant

8.1 Purpose and problem

An operator at a clarifier with gloves on cannot scroll a dashboard. "Hey Hydris" lets them ask out loud ("what's the outlet pH?", "any open alarms?", "what are my tasks?") and hear a short answer on the glasses speakers or the phone, with the facts behind it on screen. Every answer comes from the caller's own plant data, never from general knowledge, and the assistant cannot change anything.

5
Read-only tools
readings, alarms, my work, assets, SOPs

2
Answer engines
Claude, or the rules engine

0
Write tools
advisory only

0
Invented numbers allowed
target and design rule

8.2 Users and jobs

Table 52. Users and jobs

User	Job
Field operator	Get a reading, its age and alarm state without touching the phone
Maintenance engineer	Hear my open work and the procedure for an asset
Supervisor	Check open alarms on my sites while walking the plant

8.3 Scope

Table 53. In scope and out of scope

In scope	Out of scope
Wake phrase, spoken question, spoken answer	Operating equipment or changing records
Readings, alarms, my work, asset status, approved SOPs	Answers from outside the tenant's data
Follow-up questions with history	Indian languages (Planned, FR-073)
Glasses, phone and browser speech	"Hey Meta, start Hydris" as a supported path (experimental)

8.4 Wake word flow

The wake phrase is detected on the device with a pattern tolerant of how recognizers hear "Hydris" ("hydras", "hi dris", "idris"). After an answer the assistant keeps listening for a follow-up; "stop", "cancel" or "thanks" ends it.

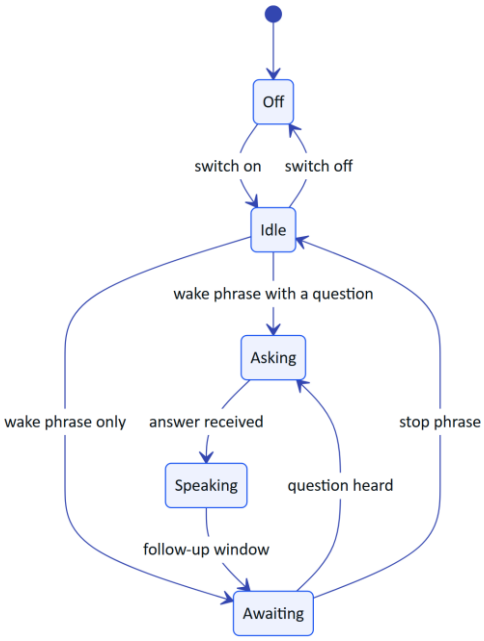


Figure 69. *Wake phrase states on the phone*

> **NOTE**
> The glasses web viewport closes the follow-up window after 8 s of silence; the Android app keeps it open until a stop phrase. Aligning them on 8 s is Planned.

8.5 Speech engines

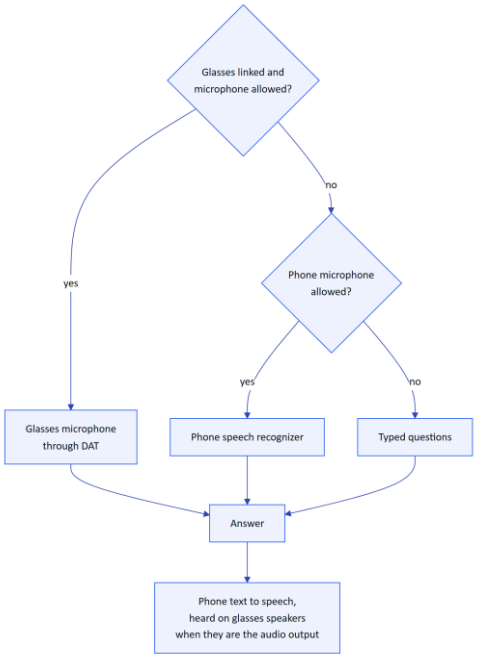


Figure 70. *Which microphone and speaker are used*

Table 54. Speech engines by surface

Surface	Listens with	Speaks with	Status
Android app, glasses	DAT Speech on the glasses microphone	Phone text to speech over Bluetooth	Partial: not yet run on hardware
Android app, phone	Android speech recognizer, restarted after each result	Phone text to speech	Built
Glasses web viewport and simulator	Bridge first, then the browser Web Speech API	Browser speech synthesis	Built

Transcripts are dropped while the assistant is speaking, so it does not answer itself.

8.6 Grounded answering

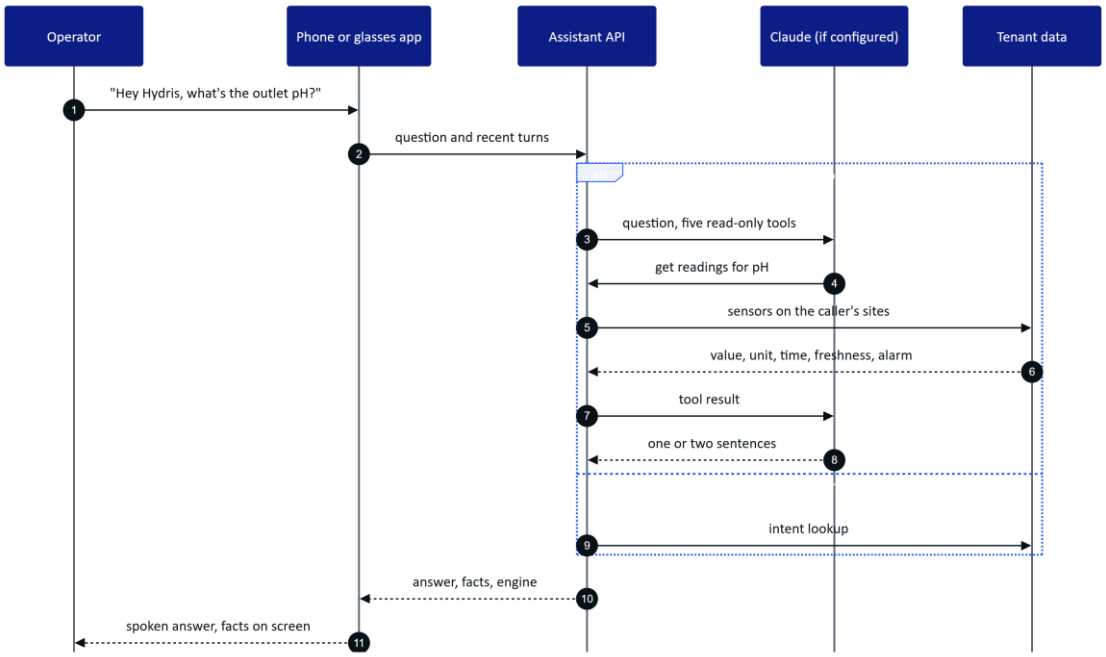


Figure 71. *A spoken question answered from the caller's own data*

When an Anthropic credential is configured, Claude answers with up to five rounds of read-only tool calls. Otherwise, and whenever the model fails, a deterministic rules engine answers readings, alarms, my work, asset status, procedures and help, and says "I can't answer that from your plant data" for anything else. Every tool runs with the caller's row-level and site scope, so the assistant can only say what the caller could see in the console.

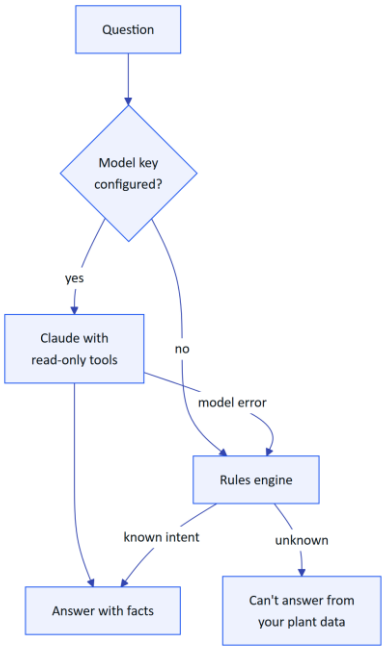


Figure 72. *Choosing the engine for each question*



Figure 73. *The assistant answering with the facts behind it*

8.7 Guardrails

- **Advisory only.** The tool list holds only read tools and the system prompt forbids operating equipment (FR-078).
- **Never invents numbers.** Answers return facts (label, value, unit, time, source); the screen shows up to four under the answer. Rules answers contain only numbers that appear in facts.
- **States the data age.** Answers say "measured N minutes ago"; stale readings add "not current, check the sensor"; missing data is "no data", never zero (NFR-036).
- **Labels synthetic data.** Demo answers mark their facts as synthetic demo data (FR-075).
- **Approved documents only.** Draft SOPs never reach an answer (FR-111).

Table 55. Example dialogue (illustrative of the rules described above, not a recorded transcript)

Operator says	Hydris answers	Facts shown
"Hey Hydris, what's the outlet pH?"	Outlet pH is 7.3, measured a few minutes ago, within range	7.3 · Outlet pH
"And the inlet?"	Answers for the inlet, using the conversation history	Inlet pH value and time
"Any open alarms?"	Counts open alarms on my sites and names the most severe	Alarm titles and states
"What are my tasks?"	Lists my open maintenance tasks and due inspections	Task titles
"What's the weather tomorrow?"	I can't answer that from your plant data	None
"Thanks"	Ends the follow-up window	None

8.8 Requirements

Table 56. Voice assistant requirements

ID	Requirement	Pri	Status
FR-070	Grounded SOP retrieval from approved documents with a safety notice	M	Built
FR-071	Wake phrase and short spoken answers	M	Built
FR-072	Answers only from tenant data through scoped tools; rules fallback never guesses	M	Built
FR-073	Hindi, Tamil, Marathi, Gujarati	S	Planned (Advanced)
FR-074	Glasses microphone with phone fallback; answers on glasses speakers	M	Partial: glasses not run on hardware
FR-075	Facts with source; synthetic data labelled	M	Built
FR-076	Follow-ups with history (up to 8 turns)	S	Built
FR-077	"Hey Meta, start Hydris" opens the assistant	C	Partial (experimental)
FR-078	No write tools	M	Built

8.9 Success metrics

Table 57. Voice targets (targets, not measured results)

Metric	Pilot target
Questions answered with facts, not "can't answer"	85 % or more
Numbers in answers missing from facts	0
Spoken answer after the question ends	Under 3 s on 4G

8.10 Status, gaps and next steps

- **Logging for metrics.** Engine and intent logging on the assistant is Planned for the pilot; until then the targets cannot be measured.
- **Glasses microphone on hardware** is the first hardware run item for Q4 2026.
- **Wake phrase false accepts** are tested against eight hours of plant conversation before tightening the pattern.

- **Model processing consent.** A tenant must agree to Claude processing before a pilot uses it; otherwise the rules engine runs alone.
- **Languages.** Indian-language questions and answers depend on server support and are Planned.

9 Platform: API, data and integrations

Every Hydris surface (website, operator console, glasses HUD, the Hydris Field Android app) talks to one platform: a modular API over one Postgres schema with row-level security. This part explains how that platform is shaped, how a request travels through it, how data is organised and isolated per customer, and how telemetry, AI and background work flow in. It closes with the decisions behind the design, the requirements it meets, and the debt that matters before scale.

15
API route modules
one Fastify instance, `apps/api/src/app.ts`

42
database tables
ten domains, `apps/api/src/db/schema.ts`

3
access control layers
RBAC, site scope, Postgres RLS

0
write paths to plant control
Hydris is read-only to PLC, SCADA and DCS

9.1 Architecture in one picture

Hydris is an npm workspaces monorepo. The API (`apps/api`, Fastify 5) is a modular monolith. It runs either inside the Next.js web deployment through a catch-all route that calls `app.inject` (no network hop), or as its own container. A pure TypeScript domain package (`packages/domain`) holds the state machines, permissions, freshness and voice parsing rules that both server and web use; the Android app mirrors them by hand.

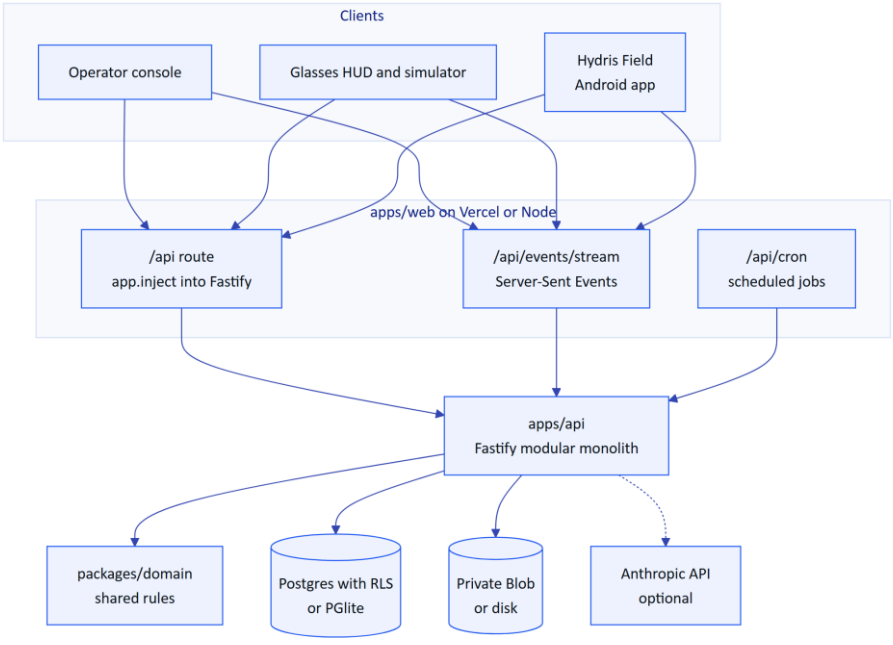


Figure 74. *Containers. The API runs inside the web deployment or as its own container; all clients share one API and one event stream.*

Configuration is one validated environment (`apps/api/src/config.ts`): `DATABASE_URL` selects Postgres or in-process PGlite, `REALTIME_MODE` selects the event bus, a Blob token selects cloud storage over disk, and `API_URL` in the web app switches to an external API. The same code therefore runs on Vercel, in Docker Compose at a plant, in a hybrid set-up and as the self-contained public demo.

9.2 The modular API

`buildApp()` registers shared plugins (helmet, cookies, a global rate limit, JWT, multipart), installs the error handler and the glasses UI profile hook, then mounts fifteen route modules. Modules share a context object and core services; they do not reach into each other's tables except through exported functions.

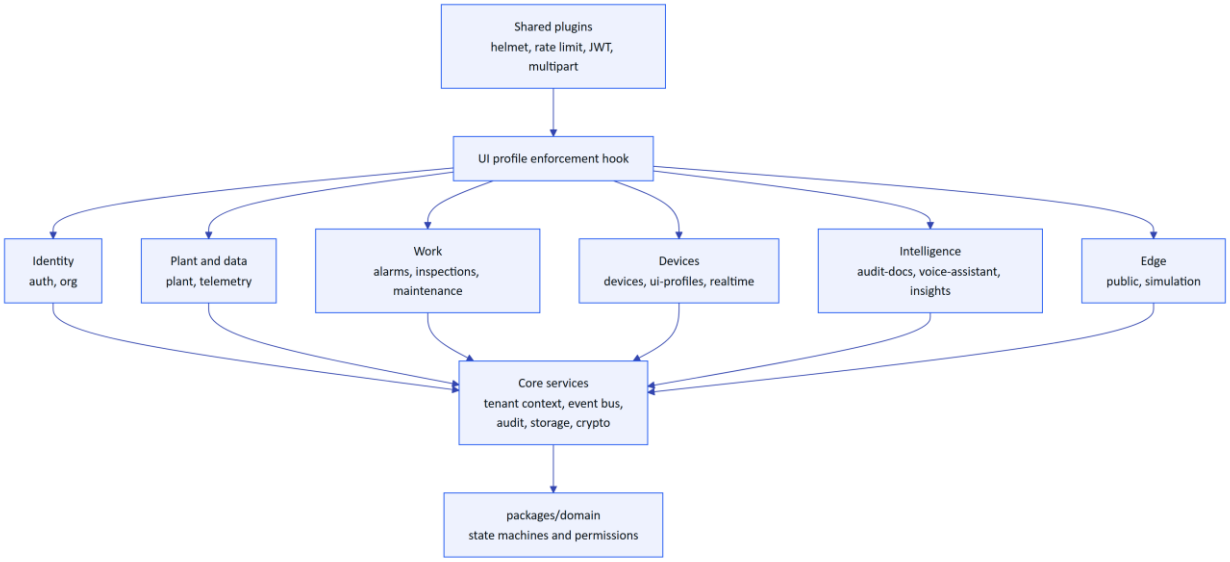


Figure 75. *Module map. Route modules sit on shared plugins and core services; the domain package supplies rules.*

Table 58. API modules and what each owns (route counts from apps/api/src/modules/*.ts)

Module	Routes	Owns	Rule worth knowing
auth	17	Sign-in, MFA, refresh, invitations, resets, sessions, org switch	Lockout after 5 failures for 15 min
org	13	Organization, members, notifications, audit log, platform orgs	No role escalation
plant	28	Sites, facilities, areas, assets, sensors, gateways, documents, attachments	Attachments only through the authorised route
telemetry	6	Ingestion, manual and lab entry, history, CSV, corrections	One `ingest()` path, idempotent per sensor and time
alarms	9	Rules, events, transitions, stale-data evaluation	Deadband, on-delay, note required to resolve
inspections	15	Templates, routes, inspections, responses, review	Offline replay by client operation id
maintenance	7	Work orders, checklists, activity	Transitions from the domain package
devices	16	Pairing, device tokens, device context, fleet admin	Session version and refresh family revocation
ui-profiles	13	Glasses UI profiles, versions, assignments	Enforced on device requests
realtime	1	Event stream (SSE)	Per-connection, per-event authorisation
audit-docs	12	Scanned records: capture, enhance, review, verify, approve, PDF	Four-eyes approval, optimistic concurrency
voice-assistant	1	Grounded spoken answers	Read-only tools, 60 per min
insights	4	Overview KPIs, document assist, search, reports	Read-only
public	1	Website contact and demo requests	5 per 10 min
simulation	3	Demo scenarios	Synthetic data refused for customer tenants

9.3 The request lifecycle

Every authenticated call follows the same path, and the order is deliberate. The principal is resolved from the database on every request, so a revoked session or device stops working at once, not when its token expires. Device principals then pass the UI profile check. Only then does the handler run, inside a tenant transaction where Postgres itself filters rows.

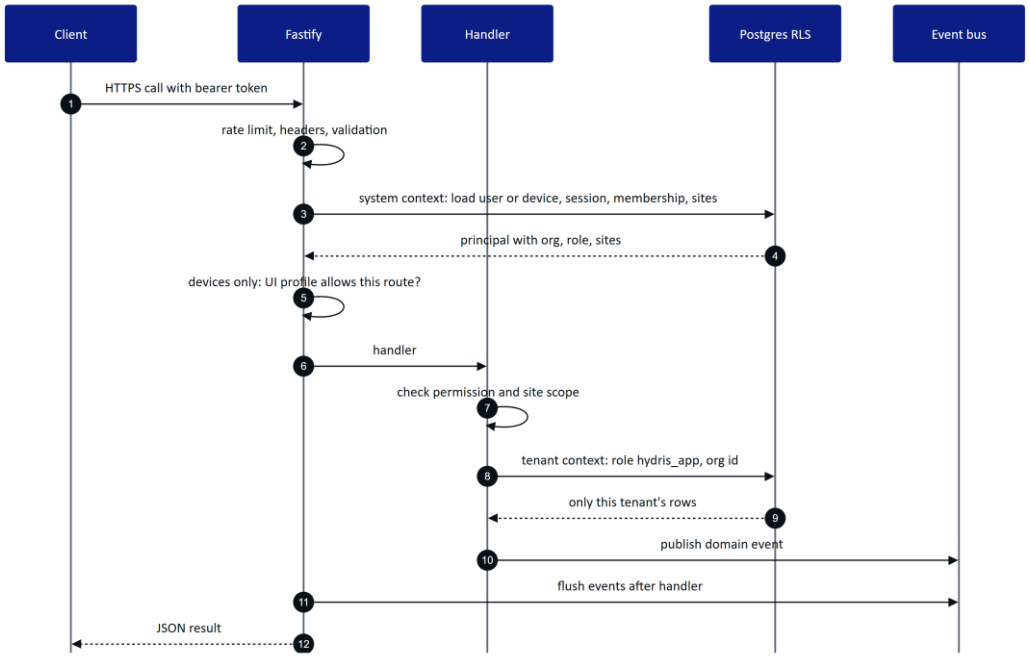


Figure 76. *Request lifecycle. Authentication reloads the principal; the handler runs in a tenant transaction; events flush after the response is built.*

Three outcomes are worth naming. Out-of-scope records return 404, not 403, so a user cannot probe for another site's assets. Every error uses one JSON shape with a stable code (`validation_failed`, `note_required`, `stale`), which the clients turn into plain messages. Unexpected errors return a masked 500 and the stack stays in logs.

9.4 Identity, sessions and device tokens

People sign in with email and password (argon2id hashing, optional TOTP MFA). They receive a 15-minute access token and a 7-day refresh token in an httpOnly cookie. Every refresh rotates the token within a family; a rotated token presented again after a 30-second multi-tab grace is treated as theft and the whole family is revoked. Because each access token carries its family id and the API checks the family on every request, logout and theft detection take effect immediately.

Devices never see a password. Glasses and phones pair through a flow shaped like the OAuth device authorisation grant (RFC 8628): the device shows a short code, a signed-in person approves it with a site and an operator, and the device receives its own tokens. A device then acts on behalf of its assigned operator, capped by a fixed ceiling of 13 device permissions.

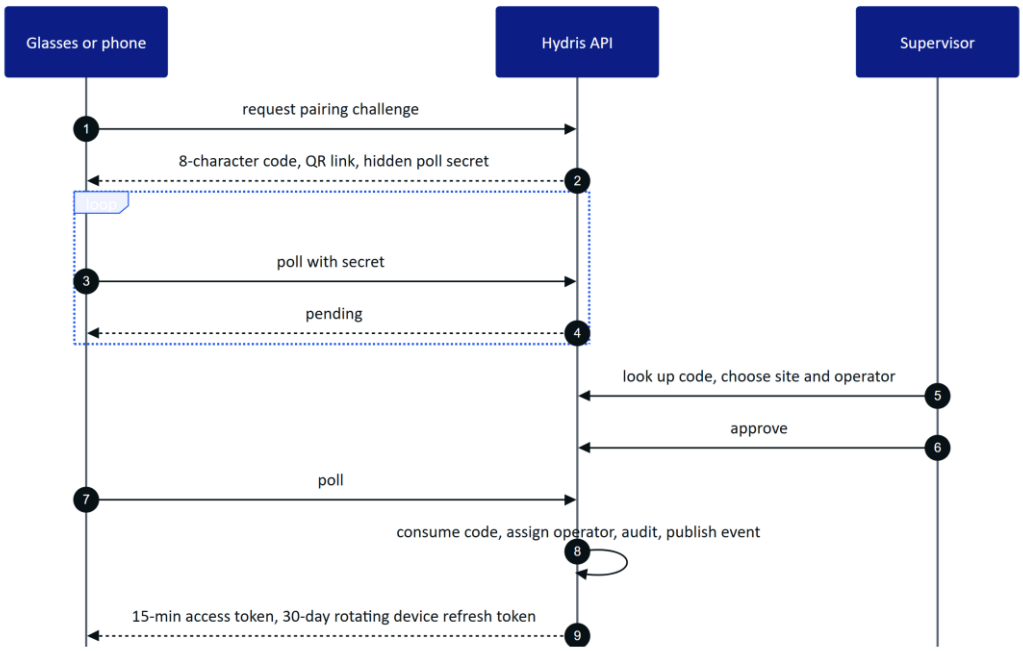


Figure 77. *Device pairing. The code is single use, expires after 300 s, and no credential is ever shown on screen or in a QR code.*

Table 59. Credentials at a glance

Credential	Lifetime	Revoked by
User access token	15 min (12 h in demo mode only)	Family revocation, checked per request
User refresh token	7 days, rotating	Logout, password change, reuse detection, session list
Device access token	15 min	Session version bump on suspend, revoke, reassign
Device refresh token (`hgd_`)	30 days, rotating	Revoke, logout, reuse detection
Gateway key (`hgw_`)	Until revoked	Admin revoke; stored only as a hash

9.5 Realtime events

Alarms, telemetry, work updates and device changes reach clients over Server-Sent Events. The stream is opened with `fetch`, so the bearer token travels in a header and never in a URL. Events carry identifiers and small payloads; clients refetch through the normal API, so authorisation is never bypassed. Every event is checked per connection (organization, target user or device, site, permission) before it is sent.

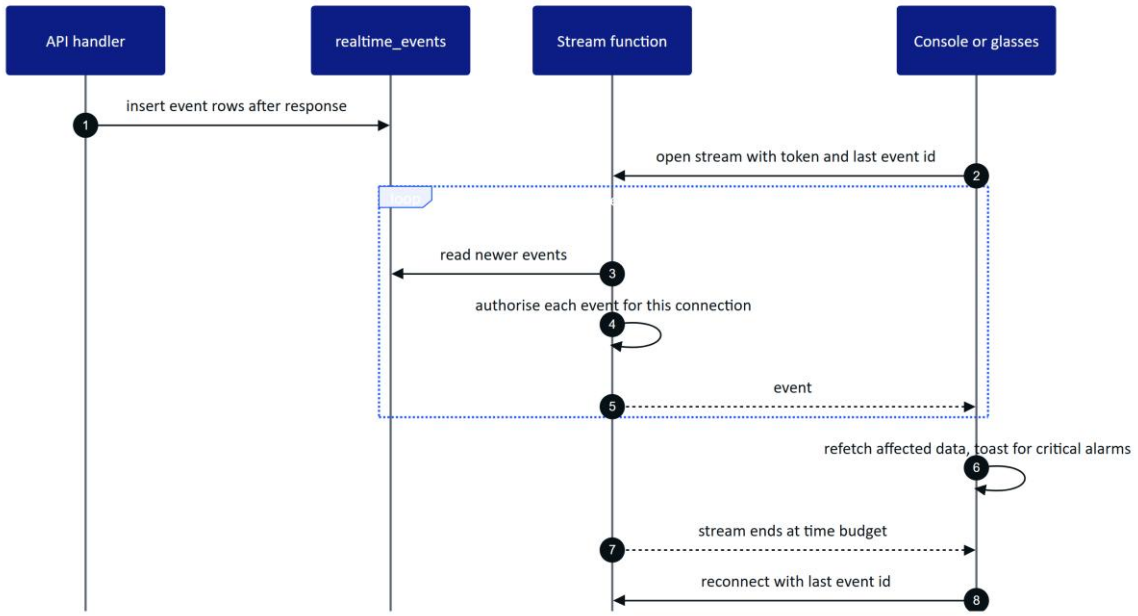


Figure 78. *Realtime in db mode. A handler writes event rows; each open stream polls, filters per connection and resumes after reconnect without gaps.*

Table 60. Realtime modes (REALTIME_MODE in apps/api/src/core/events.ts)

Aspect	memory	db
Fan-out across instances	No, single instance	Yes, all instances read one table
Replay on reconnect	Last 1,000 events	Any event still stored (pruned after 1 day)
Latency	Immediate	Up to 1.5 s poll
Used by	Docker Compose, long-running server	Vercel, demo, multi-instance

The console batches cache invalidations every 1.5 s but raises a toast at once for critical and high alarms; the Android app raises a system notification.

9.6 The data model by domain

The schema has 42 tables in ten domains, each owned by one module. Plant objects are archived, not deleted; telemetry and the realtime log use big integer keys; everything else uses UUIDs. Four views carry the essentials.

8
identity and tenancy tables
users are global, membership is per org

8
plant and telemetry tables
site to sensor hierarchy, immutable readings

11
alarm and work tables
rules, events, inspections, maintenance

10
device, UI and audit document tables
pairing, profiles, scanned records

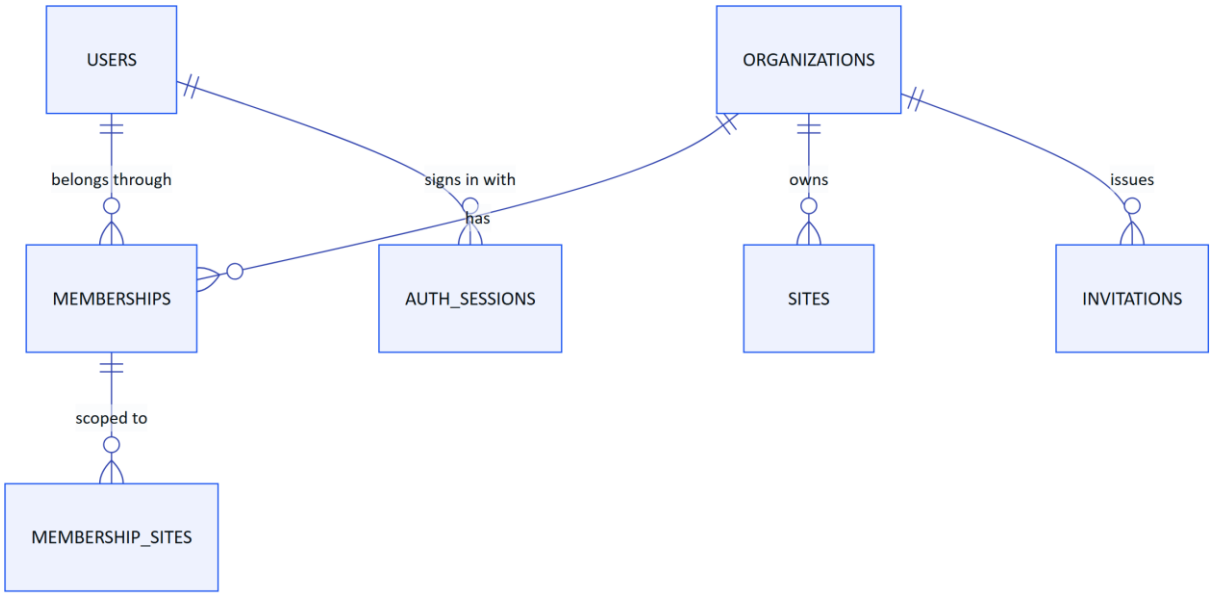


Figure 79. *Identity and tenancy. A global user joins organizations through memberships, optionally scoped to sites.*

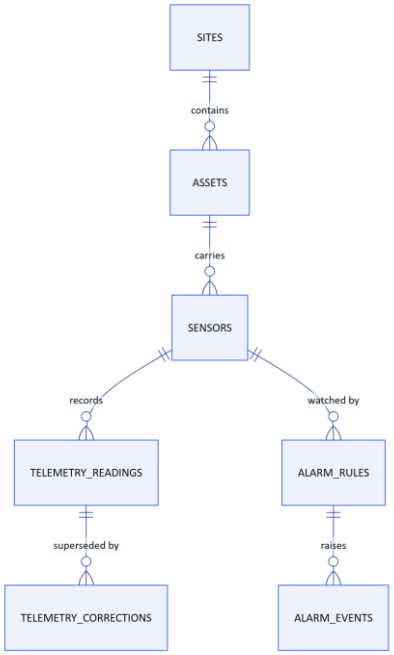


Figure 80. *Plant, telemetry and alarms. Readings are immutable; corrections and alarm events reference them.*

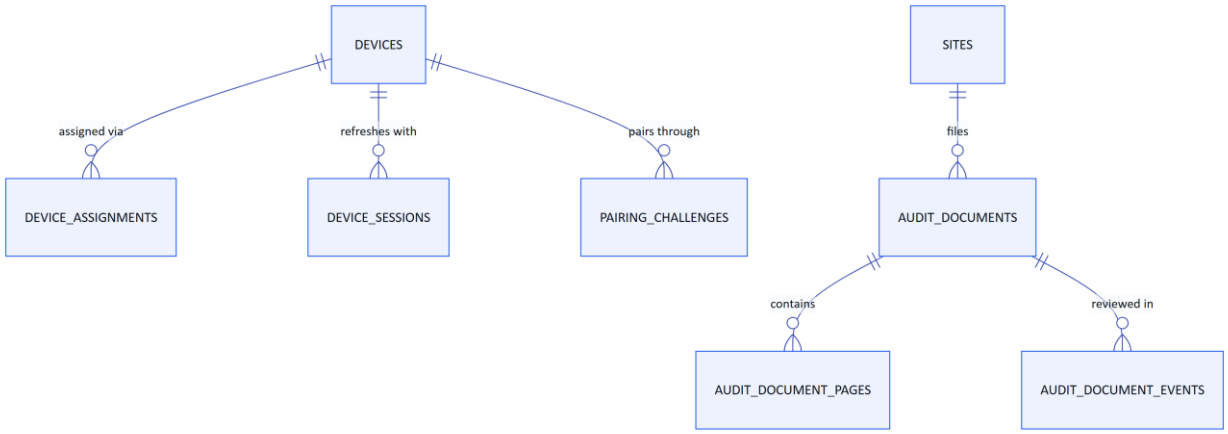


Figure 81. *Devices and scanned audit documents. A device is assigned to one operator; a document holds pages and an append-only event trail.*

Work (inspection templates, routes, inspections, responses, maintenance tasks and activity) hangs off sites and assets in the same way, and inspection records carry a revision counter for safe offline replay. Records are evidence: manual and lab readings keep provenance, corrections never overwrite the original reading, and synthetic data is marked and refused for customer tenants by a database check.

9.7 Tenancy and data isolation

Isolation is enforced by Postgres, not only by application code. Every tenant table has row-level security enabled and forced, with one policy that compares the row's organization with a per-transaction setting. Requests run as a non-privileged role, `hydris_app`, that cannot bypass the policy.

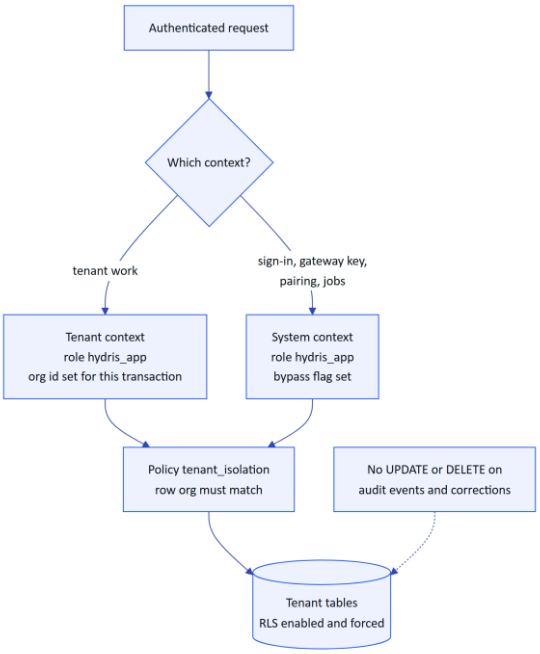


Figure 82. *Tenant isolation. Both contexts run as the non-privileged role; only the session settings differ.*

Settings use `SET LOCAL`, so nothing leaks to the next request on a pooled connection. Append-only grants hold even in system context. The application adds RBAC (six roles, 38 permissions in `packages/domain/src/roles.ts`) and site scope on top, so a forgotten filter in a handler is a non-event rather than a breach.

> **WARNING**
>
> Owner and superuser connections bypass RLS. Production must connect as a runtime user that is a member of `hydris_app` and not the table owner. Today the same owner user runs migrations and the application.

9.8 Telemetry ingestion and integrations

Hydris never connects into a plant. A gateway on the customer side pushes readings outward over HTTPS with a hashed `hgw_` key, up to 500 readings per call. Ingestion is idempotent per sensor and timestamp, so a gateway can retry safely after an outage, and it returns per-reading rejections so the gateway can retry or drop single readings. Manual and lab entries and the demo simulator use the same `ingest()` path.

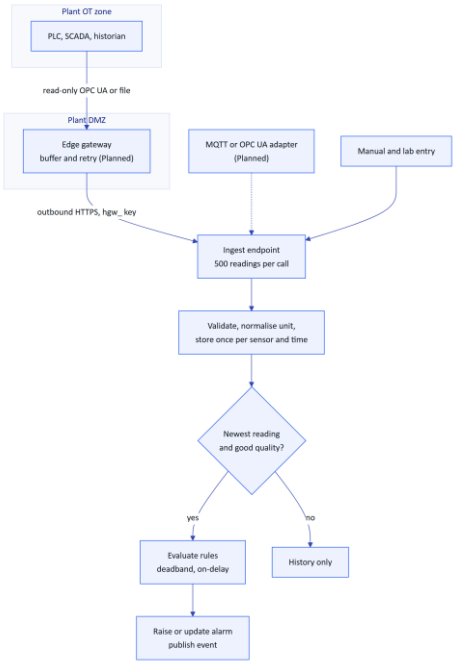


Figure 83. *Telemetry flow from the plant to an alarm. The edge gateway and MQTT or OPC UA adapters are planned; the HTTPS contract is built.*

Design details keep alarms honest: late data is stored for history but does not overwrite a newer value or re-trigger rules; once in alarm, a value inside the deadband reports "holding" so a noisy signal does not flap; a breach must persist for the on-delay before an alarm is raised; and there is one open event per rule. Stale-data rules cannot run on ingestion, because the problem is the absence of readings, so they run as a background job.

Table 61. Integration status

ID	Integration	Status
FR-022	HTTPS gateway ingestion, 500 readings per call, idempotent	Built
FR-023	Manual and lab entry with provenance	Built
FR-024	Append-only corrections	Built
FR-027	MQTT and OPC UA ingestion adapters	Planned (Advanced)
FR-028	Read-only SCADA and historian connectors	Planned (Advanced)
FR-029	Synthetic data labelled and refused for customer tenants	Built
n/a	Edge gateway software (store and forward)	Planned; API contract Built

9.9 AI services and the human in the loop

Two features use a model (Claude, through `@anthropic-ai/sdk`), and both are optional. Without a credential, audit scans keep their on-device OCR results and the assistant uses a deterministic rules engine. The model never sees the database: the assistant can only call five read-only tools (`get_readings`, `get_open_alarms`, `get_my_work`, `get_asset`, `search_procedures`), and they run inside the caller's tenant transaction, so RLS and site scope apply to every lookup.

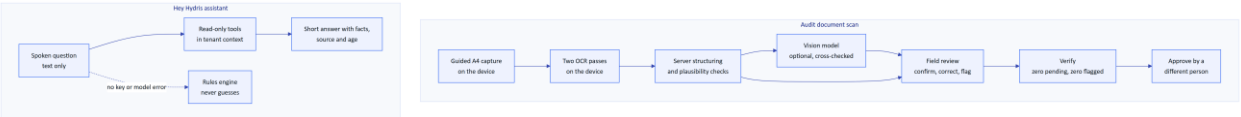


Figure 84. *AI flows. Vision adds a second opinion to on-device OCR; the assistant answers only from tenant-scoped tools; people decide what becomes a record.*

Table 62. AI requirements

ID	Requirement	Status
FR-063	Optional vision engine cross-checked against OCR text	Built (needs API key)
FR-064	Field review; verify requires zero pending and zero flagged items	Built
FR-065	Independent four-eyes approval	Built
FR-072	Assistant answers only from tenant data; rules fallback never guesses	Built
FR-078	Assistant has no write tools	Built
NFR-035	Only values returned by tools in the same conversation	Built (prompt and tool design)
NFR-037	Graceful degradation without the model	Built
NFR-039	Human in the loop for every AI-extracted value	Built
NFR-040	Model provider and region documented per tenant	Planned (GA)

Vision cannot replace an extraction once review has started, and every review write carries a revision so a lost race returns a conflict instead of overwriting a colleague's work.

9.10 Background jobs

Four jobs keep the platform current. On a long-running server they run on timers; on Vercel they run from a cron route protected by a secret; in demo mode a lazy ticker advances them on API activity.

Table 63. Background jobs by topology

Job	Long-running server	Vercel today	Demo
Stale-data alarm rules	Every 30 s	Daily (`0 3 * * *`)	Not ticked
Pairing challenge expiry	Every 60 s	Daily	Expiry checked on use
Realtime event pruning (older than 1 day)	Hourly	Daily	Per instance
Simulation steps	Every 5 s per run	Daily	On activity, at most every 5 s

> **WARNING**
> In the all-Vercel topology stale-data alarms are raised at most daily (FR-035 Partial, TD-02). A production plant needs a per-minute scheduler or the API in a container.

9.11 Key architecture decisions

Table 64. Architecture decision records (all Accepted, ARCHITECTURE.md)

ADR	Decision	Why	Trade-off
002	Fastify modular monolith	One deployable, in-process calls	Modules must stay disciplined

ADR	Decision	Why	Trade-off
003	API inside the Next.js route with `app.inject`	One Vercel project, no network hop	Cold start builds the app and migrates
004	Postgres RLS plus RBAC and site scope	Strong isolation in one schema	Every request is a transaction
005	PGlite for development, tests and demo	Real Postgres semantics, no service	Per-instance memory in demo
007	SSE over `fetch` for realtime	Works on serverless and proxies; token out of URLs	Polling in db mode
008	Two event bus modes, memory and db	No extra infrastructure	1.5 s latency in db mode
009	On-device dual OCR, vision optional	Works offline and without AI	Vision only improves layout
011	RFC 8628-style pairing	No credential on screen or in a QR	Code TTL, no attestation yet
012	Push-only HTTPS ingestion from an edge gateway	Hydris stays outside control networks	Customer or partner runs the gateway
013	Short JWTs plus rotating refresh tokens checked per request	Immediate revocation	One lookup per request
015	LLM grounded through tenant-scoped tools with fallbacks	Answers cite real readings	Narrower coverage without the model
016	Pure domain package shared by server and web	One definition of rules	Android mirrors by hand

9.12 Platform requirements and NFRs

Table 65. Platform requirements (IDs from PRD.md)

ID	Requirement	Status
FR-003	Refresh rotation with family revocation on reuse	Built
FR-007	Site-scoped permissions for every role except org admin	Built
FR-008	Database-enforced tenant isolation	Built
FR-033	Realtime push of alarm events to all clients	Built
FR-050	Pairing with short code, QR, 5-min TTL, 3 s polling	Built
FR-051	Device tokens with rotating refresh and reuse detection	Built
NFR-001	API read latency p95 under 300 ms in-region	Partial (target, not measured)
NFR-002	Realtime alarm delivery 2 s or less	Built (design, target not measured)
NFR-007	Horizontal scale of realtime	Built (db mode)
NFR-008	Scheduled jobs at a cadence suitable for real data	Partial (daily on Vercel)
NFR-009	Backups and tested restore	Planned
NFR-011	Tenant isolation enforced in the database	Built
NFR-012	Short-lived credentials	Built
NFR-013	Rate limiting	Built (per instance)
NFR-030	Provenance on every reading	Built
NFR-031	Optimistic concurrency on review records	Built

ID	Requirement	Status
NFR-032	Shared domain logic between server and clients	Built

9.13 The technical debt that matters

The register in `ARCHITECTURE.md` lists 18 items. These are the ones that decide whether a pilot runs well and whether the platform scales.

Table 66. Technical debt that matters

ID	Debt	Impact	Fix
TD-01	Root `.gitignore` hides the Android `data` package	A fresh clone of the Android app does not compile	Anchor the rule, commit the package
TD-02	Vercel cron runs daily	Stale-data alarms up to a day late	Per-minute scheduler or hybrid topology
TD-03	In-memory rate-limit store	Limits are per instance	Shared store or edge firewall
TD-04	Audit page images in Postgres as base64	Database and backup growth	Object storage behind the storage adapter
TD-05	Empty contracts package; Android models by hand	Contract drift	Generate schemas and Kotlin models
TD-06	No CI pipeline	Regressions found late	GitHub Actions workflow
TD-07	db realtime polls every 1.5 s per stream	Database load grows with connections	`LISTEN/NOTIFY` or managed pub/sub
TD-10	No telemetry partitioning or retention	Index cost at volume	Monthly partitions, retention per contract
TD-11	Ingestion evaluates rules reading by reading	Throughput ceiling	Batch inserts and rule loading
TD-12	No content security policy	Weaker defence against injected scripts	Nonce-based CSP

> **NOTE**
> Planning estimates, not measurements: at 200 sensors per site reporting every minute, one site produces about 288,000 readings a day. The bottlenecks arrive in the order TD-11, TD-07, TD-10, TD-04, which is the order the scaling plan addresses them.

10 Security, delivery and operations

Hydris puts cameras on workers' faces and turns plant readings into regulated records. Trust is therefore part of the product, not a layer on top. This part covers how Hydris protects customers and workers, how it maps to the frameworks buyers ask about, where it runs, how quality is proven, and how a customer is onboarded and supported. Every target is labelled as a target; nothing here is a certification.

3
layers of access control
RBAC, site scope, Postgres RLS

15 min
access token lifetime
rotating refresh with reuse detection

0
write paths to plant control
read-only to PLC, SCADA and DCS

2
people per approved record
verifier and a different approver

10.1 Security model

Eight principles shape every design choice: read-only to the plant; defence in depth for tenancy; records are evidence (append-only trails); least privilege for devices; secrets never stored in the clear; AI assists, people decide; capture on intent only; privacy by default.

Trust boundaries

A trust boundary is anywhere data crosses between a party or runtime Hydris controls and one it does not. Five boundaries matter.

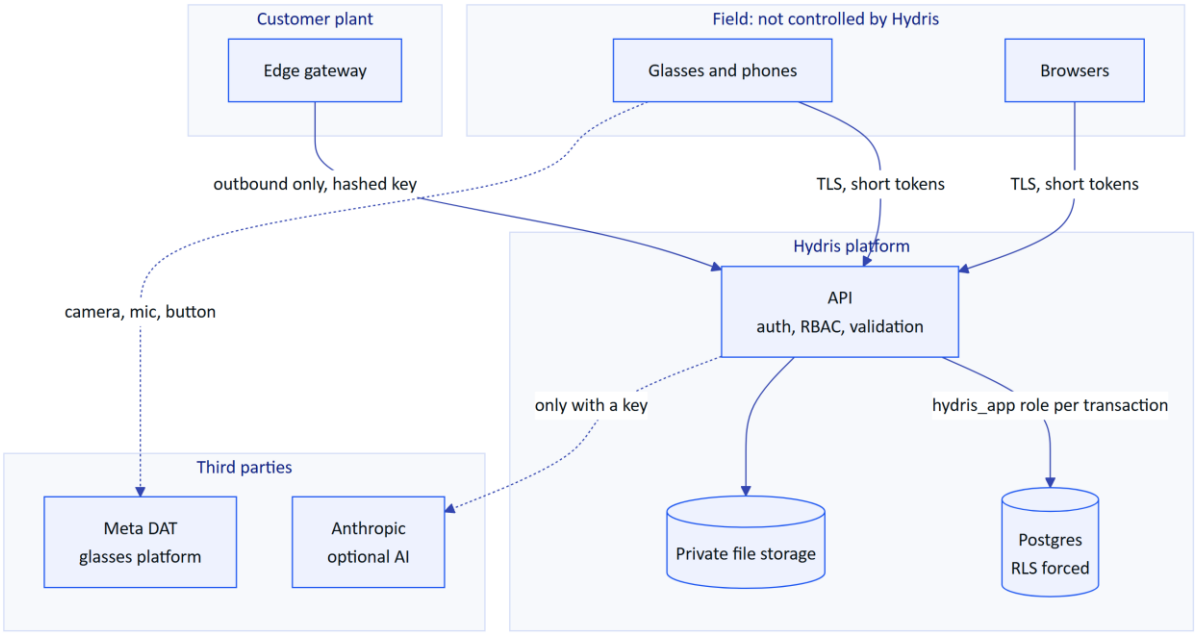


Figure 85. *Trust boundaries. Data crosses five boundaries, each with its own controls.*

Table 67. Top threats and controls (STRIDE extract from SECURITY_PRIVACY.md)

Threat	Scenario	Control in the product	Open gap
Spoofing	Password guessing, credential stuffing	argon2id, lockout after 5 failures, 10 sign-ins per min, optional TOTP	No SSO, MFA not enforceable per org
Spoofing	Rogue device pairs into an org	Signed-in approver with `device.pair`, 300 s single-use code, hashed poll secret	No device attestation

Threat	Scenario	Control in the product	Open gap
Tampering	Silent edit of readings or records	Append-only corrections and audit events at the grant level	No signed export or hash chain
Information disclosure	Cross-tenant read	RLS forced, 404 for out-of-scope records	Owner connection would bypass RLS
Information disclosure	Stolen phone	Keystore AES-256-GCM on Android, device revoke	Web glasses keep the token in `localStorage`
Denial of service	Login or ingest floods	Rate limits, 500 readings per call, body limits	Limits are per instance
Elevation	Device calls admin endpoints	Fixed device permission ceiling; verify and approve need a person	None known
AI	Prompt injection in a scanned page	Strict output schema, OCR cross-check, human review	No injection classifier

Privacy for camera glasses and workers

The camera only runs when a person asks for it, and only the final page leaves the phone.

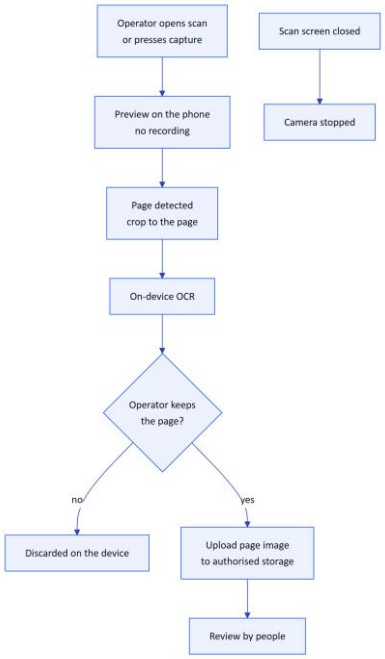


Figure 86. *Camera on intent. Preview frames stay on the phone; only a reviewed page image is uploaded.*

Table 68. Camera and worker commitments

Commitment	Status
No background or continuous recording, no video stored	Built
Only text after "Hey Hydris" is sent; audio never uploaded to Hydris	Built
Meta capture LED cannot be disabled by apps	Vendor control
No location tracking, no keystroke or gaze analytics, no individual performance reports	Built (by design)
Face and badge redaction before upload	Planned
Camera-free zones per site	Planned

10.2 Compliance mapping

The table maps frameworks to product controls. It is a mapping and a set of targets, never a claim of certification.

Table 69. Compliance mapping (targets, not certifications)

Framework	What it asks	Hydris controls	Position
India DPDP Act 2023 and Rules 2025	Notice, safeguards, breach notice, erasure, retention limits	Encryption, RLS, audit trail, consent version on inquiries	Partial; retention and rights tooling planned
CPCB and SPCB consent conditions	Unaltered, retrievable monitoring records	Provenance, append-only corrections, freshness, CSV export	Mapping; Hydris is not the regulatory feed
CERT-In Directions 2022	6-hour incident reporting, 180-day logs in India	Incident plan; India log drain planned	Target
ISO/IEC 27001:2022	ISMS and Annex A controls	Many technical controls in code	Target: ISMS build 2027
IEC 62443	Zones and conduits, secure development	Read-only, outbound-only conduit from the DMZ gateway	Mapping; 4-1 practices as target
SOC 2	Security, availability, confidentiality	Same controls	Target: Type I after first commercial year

Retention today is indefinite for records, with only realtime events pruned after a day. The recommended defaults (for example audit trails at least 7 years, sessions purged 30 days after expiry) are configurable per organization and Planned.

10.3 Deployment topologies

The same code runs in four shapes. A customer chooses by residency, network rules and operations appetite.

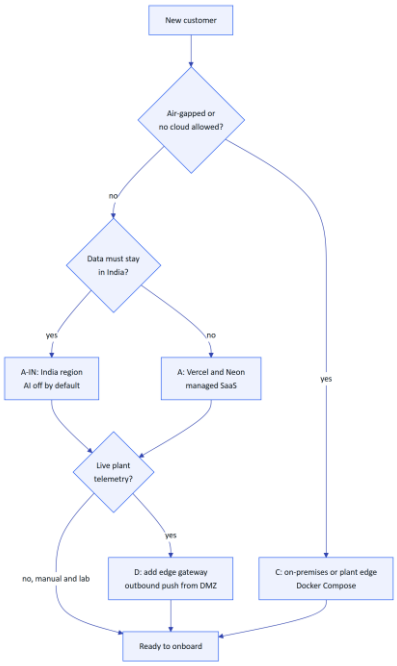


Figure 87. *Topology selection for a new customer*

Table 70. Topologies

Topology	Shape	Status
A, managed SaaS	Vercel functions, Neon Postgres, private Blob	Built (live as demo); production Planned

Topology	Shape	Status
A-IN, India region	Mumbai functions, Indian database and S3, India log drain, AI off unless an addendum is signed	Planned
B, container API	Vercel web, API container next to managed Postgres	Partial (`render.yaml`)
C, plant edge	Postgres 17, API and web on one plant server	Built (`docker-compose.yml`)
D, hybrid gateway	A or A-IN plus an edge gateway pushing over HTTPS	API Built; gateway Planned

Table 71. Environment matrix

Environment	Purpose	Database	Status
dev	Local development	PGlite on disk	Built
preview	One per pull request	Neon branch (target) or demo mode	Partial
staging	Release candidate, hardware tests, pilot rehearsal	Neon staging branch	Planned
prod	Customer tenants	Neon production, region pinned, point-in-time recovery	Planned
demo	Public sales demo	In-memory PGlite per instance, public secrets	Built

Release pipeline

Today there is no CI: a push to `main` deploys through Vercel's Git integration, and tests are run by developers. Migrations run on cold start under an advisory lock. The proposed pipeline gates every pull request and every release tag.

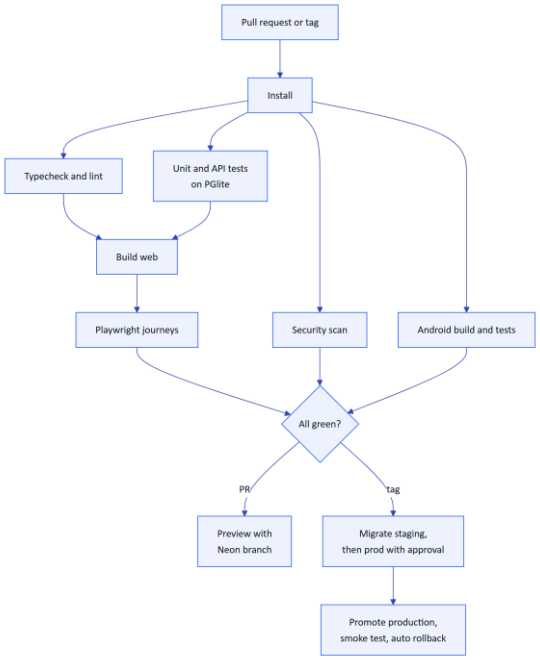


Figure 88. *Proposed CI and release pipeline (Planned)*

DECISION

Demo and production split into two Vercel projects on the same repository: demo tracks `main` with no database, production deploys only on release tags after the staging gate (DEP-11).

10.4 Quality

Hydris tests from fast, deterministic checks at the base up to field trials at a plant.

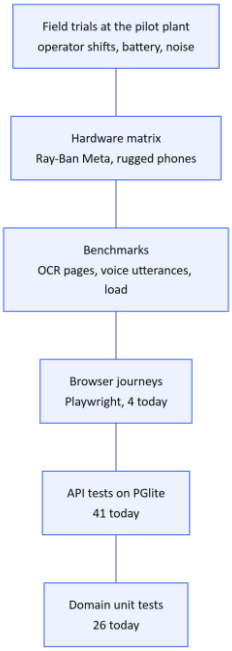


Figure 89. *Test pyramid, from fast deterministic tests to field trials*

The 67 domain and API tests pass (22 October 2026). They cover the security-critical paths: tenant isolation at the database, four-eyes approval, refresh reuse, pairing, alarm hysteresis and the db realtime mode. The four Playwright journeys were not re-run for this edition. The Android app has no automated tests yet and was verified by hand on a Pixel 7 emulator.

OCR and voice accuracy design

Accuracy is measured with benchmarks, all targets for the first pilot.

Table 72. Benchmark design (targets)

Benchmark	Set	Primary metric	Target
OCR	120 pages: printed lab reports, filled templates, handwritten logbooks, tables, degraded captures; ground truth keyed twice	Silent error rate (wrong but marked as matching)	Under 0.5 %
OCR	Same	Field accuracy	90 % printed, 75 % handwritten
Voice	600 utterances by at least 6 speakers in quiet, 65 and 85 dB(A)	Hallucinated numbers	0
Voice	Same, including 40 unsafe action requests	Unsafe compliance	0
Voice	8 hours of plant conversation	Wake false accepts	Under 1 per hour

DECISION

The silent error rate is the primary OCR metric. A system that is 85 % accurate and flags what it is unsure of is safer than one that is 95 % accurate and confidently wrong on the rest.

Release gates and pilot exit

Table 73. Release gates

Gate	Criteria
1, pull request	Typecheck and tests green; new behaviour has tests
2, main	Build, Playwright, accessibility and dependency audit clean
3, release candidate	OCR and voice benchmarks in tolerance; load and security scans clean
4, pilot readiness	Hardware rows H1 and H5 passed; zero open S1 and S2 defects; signed review

Pilot exit criteria, as of today:

- ☑ Domain and API suites pass (67 of 67)
- ☑ Tenant isolation proven at the database boundary
- ☑ Four-eyes approval enforced outside demo tenants
- ☐ CI running gates 1 and 2 on every pull request
- ☐ Email transport wired for invitations and resets
- ☐ Stale-data rules running every minute on Vercel
- ☐ OCR and voice benchmarks run within targets
- ☐ Hardware rows passed on Ray-Ban Meta and a rugged phone
- ☐ External penetration test with no open high findings

10.5 Operations

Customer onboarding

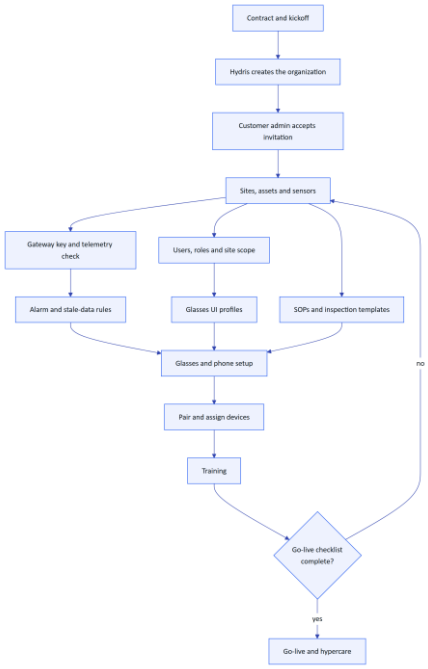


Figure 90. *Customer onboarding from organization creation to go-live*

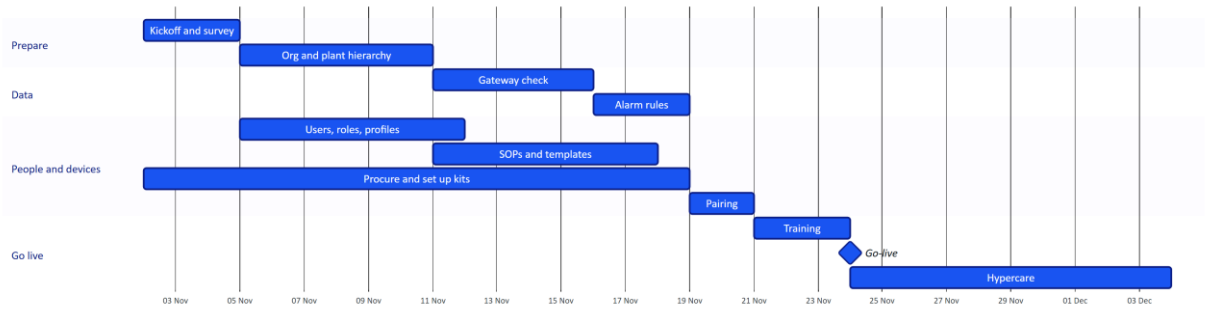


Figure 91. *Typical six-week pilot for one plant with up to ten operators (plan, not a commitment)*

Support and incidents

Operators go to their supervisor, the supervisor to the customer admin, and the customer admin to Hydris.

Table 74. Support tiers

Tier	Handles
Customer admin	First line inside the plant: users, pairing, profiles
Hydris L1	Anything a documented runbook procedure resolves
Hydris L2	Configuration, logs and platform-admin actions
Hydris L3	Code, data and infrastructure; postmortem for Sev 1 and 2

Table 75. Incident severity (targets)

Severity	Definition	Response	Customer update
Sev 1	Platform down or data at risk, including any suspected breach	L3 within 15 min	Within 1 hour, then hourly
Sev 2	Major function down, no workaround	L3 within 1 hour	Within 2 hours, then every 4 hours
Sev 3	Degraded, workaround exists	Next business day	Daily
Sev 4	Minor or cosmetic	Planned release	On release

A suspected breach also triggers external notices: customers within 24 hours, CERT-In within 6 hours for reportable incidents, and the DPDP Board and data principals as the Rules require.

Operations KPIs

5 min
time to pair
target, median

1 day
scan to verified
target, median

5 min
critical alarm acknowledge
target, median

99.5 %
monthly availability
target

Table 76. Operations KPIs (targets) and where they are measured

KPI	Target	Source in the product
Pairing success on first code	Over 90 %	Completed vs expired challenges
Verified to approved	Median 2 business days	Audit document timestamps

KPI	Target	Source in the product
Rescan rate	Under 10 %	Rejected vs total documents
Alarms resolved with a note	100 % (enforced in code)	Resolve transition
Sensors live	Over 98 % of sensor-hours	Freshness history
Devices connected on shift	Over 95 %	Device report

10.6 Known gaps register

The gaps below are drawn from the security, deployment and architecture registers. Each has an owner and a plan; the quadrant shows how they compare.

Table 77. Known gaps that matter before pilot and GA

ID	Gap	Impact	Priority
DEP-01	No CI pipeline	Regressions and unsafe migrations can reach production	High
DEP-02	Cron runs daily on Vercel	Stale-data alarms up to 24 h late	High
DEP-03	Android release signed with the debug key	Cannot distribute through Play	High
DEP-04	Email not wired	Invitations and resets need manual delivery	High
DEP-05	No metrics, tracing or alerting	Incidents found by users	High
DEP-11	Demo and production share one project	Customer data could land in a demo	High
SEC	Owner connection runs the application	RLS bypass if a query escapes the tenant role	High
SEC-03	Web glasses keep the device token in `localStorage`	Token exposed to scripts on the origin	Medium
DEP-10	Per-instance rate limiting	Weaker limits under many instances	Medium
DEP-06	Audit page images in Postgres	Database and backup growth	Medium
DEP-07	No retention or purge jobs	Unbounded growth; DPDP retention limits	Medium
TD-12	No content security policy	Weaker defence against injected scripts	Medium

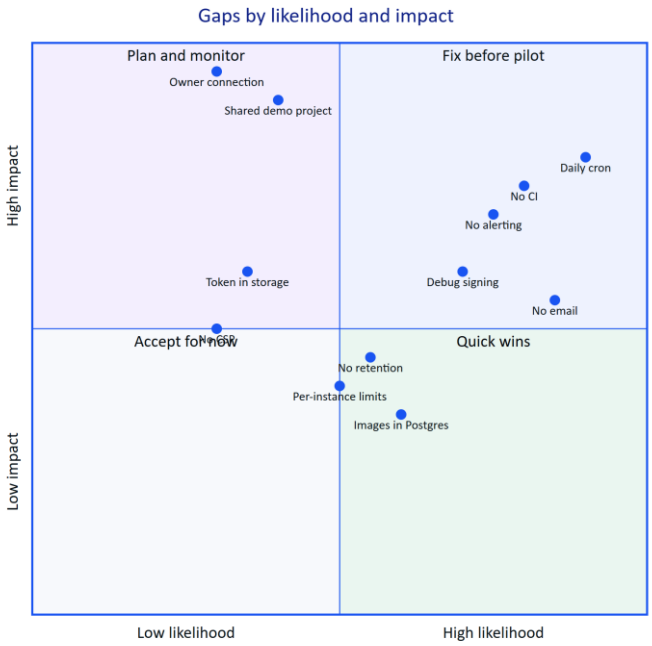


Figure 92. *Gap risk quadrant (team assessment, not measured)*

11 Roadmap and future products

11.1 Purpose of the roadmap

The minimum viable product is built and runs as a zero-configuration demo. The next twelve months turn it into something a plant can rely on every shift, then into something a regulator, a maintenance system and a control room can rely on too. This part is a plan, not a commitment: every date is an estimate that assumes the current team and no major change in Meta's developer programme, and effort figures are team estimates. Status labels follow the book: **Built**, **Partial**, **Planned**.

5
quarters planned
Q4 2026 to Q4 2027

8
themes
from hardening to ecosystem

19
future features analysed
problem, outcome, dependencies

1 to 2
pilot plants
ETP or STP, Q1 2027 (target)

11.2 Horizons: now, next, later

Table 78. Horizons, goals and exit conditions (plan)

Horizon	Window	Goal	Exit condition
Now	Q4 2026 to Q1 2027	A pilot plant uses Hydris every shift without engineering support on site	Pilot exit criteria all checked; 4 weeks of daily use
Next	Q2 to Q3 2027	A customer can buy, deploy and audit Hydris without Hydris engineers	GA release; SOC 2 Type I report; Play or MDM distribution; second paying site
Later	Q4 2027 and beyond	Hydris is the system of record for field work and compliance evidence at multi-site customers	A regulator accepts a Hydris compliance report; one CMMS integration live

Confidence falls with distance: scope is high-confidence for Now, medium for Next and low for Later; dates are medium, low and very low. Pilot customer dates, Meta DAT access, pilot KPIs and team size are what move them.

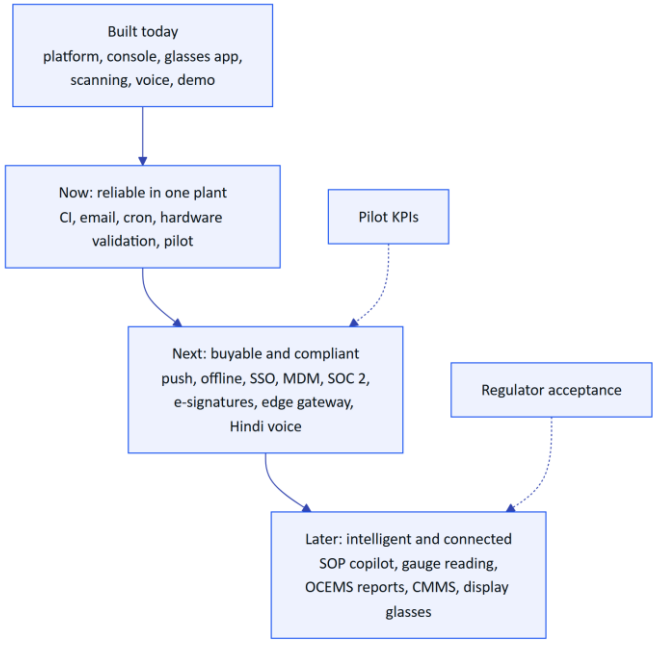


Figure 93. *From MVP to system of record: what each horizon unlocks (plan)*

11.3 Timeline, Q4 2026 to Q4 2027

The timeline and Gantt chart are the plan as of October 2026 and are re-baselined at the end of every quarter.

Table 79. Quarter summary (plan)

Quarter	Headline	Key deliverables	Exit signal
Q4 2026	Make it reliable	CI, email transport, cron schedule, audit images to object storage, purge jobs, Android pilot build	Test gates green on every change
Q1 2027	Prove it in a plant	Ray-Ban Meta hardware matrix, pilot plant 1, field trial, KPI baseline	Pilot exit criteria met
Q2 2027	Make it buyable	GA: push notifications, offline outbox, SSO, MDM, SOC 2 Type I readiness, India-region production	Second site signed
Q3 2027	Make it compliant and connected	E-signatures, edge gateway, Hindi voice	First e-signed record approved at a customer
Q4 2027	Make it intelligent	SOP copilot, gauge reading, first CMMS connector, OCEMS report drafts	Copilot used weekly by 50 % of operators (target)

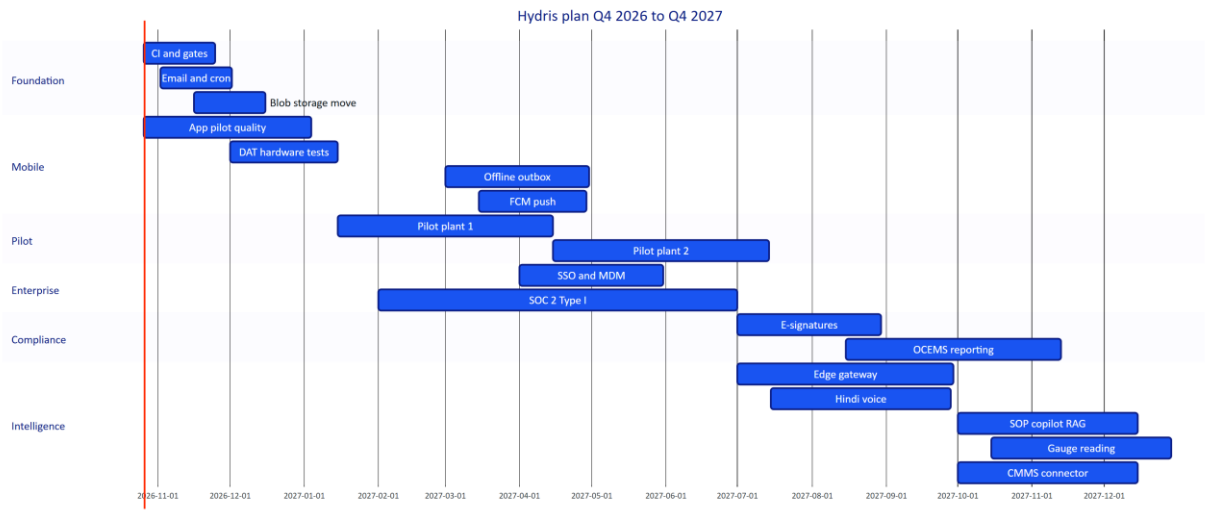


Figure 94. *Quarter plan as a Gantt chart (plan; bars are estimates)*

11.4 Known gaps to close first

These are the debts that would hurt a pilot. They come before any new product.

Table 80. Known gaps and tech debt, in priority order

Gap	Impact	Fix	Target
No CI pipeline	Regressions reach the demo	Typecheck, unit, API and e2e gates on every change (NFR-010)	Q4 2026
Email not sent	Invitations and resets never arrive	Email transport for the outbox (FR-086)	Q4 2026
Cron runs daily on Vercel	Stale-data alarms evaluated once a day (FR-035)	Per-minute schedule or external scheduler	Q4 2026
Audit images stored in Postgres	Database bloat, slow backups	Object storage with lifecycle	Q4 2026

Gap	Impact	Fix	Target
No purge jobs	Expired sessions, challenges and sent mail kept indefinitely	Purge 30 days after expiry	Q4 2026
No Android automated tests	Manual regression only	Android test suite	Q4 2026
DAT paths unverified on hardware	Glasses camera, button and microphone may fail on device (FR-055 to FR-057, FR-074)	Hardware test matrix	Q1 2027
Web glasses refresh token in browser storage	Token theft on a compromised device	Native app keystore (built); shorter or bound tokens for web	Q1 2027
No per-org opt-out of third-party AI	Blocks some customers	Organization setting	Q2 2027
No device attestation	Spoofed clients	Play Integrity	Q2 2027
Audit log not hash-chained	Weaker tamper evidence	Hash chain or WORM export	Q3 2027

11.5 Future products in brief

Each future product reuses a pattern already proven in the MVP: read-only and outbound-only towards the plant, machines propose and people confirm, and every value keeps its provenance.

Table 81. Future products (all Planned; effort is a team estimate)

Product	Problem it solves	Outcome	When (plan)	Effort
Edge gateway (OPC UA, MQTT, Modbus)	Each customer needs custom integration; Hydris accepts HTTPS batches only	A container in the plant network reads tags and forwards readings outbound over HTTPS, with store-and-forward and a tag-mapping UI (FR-027, FR-028)	Q3 2027	RICE effort score 5 (estimate)
Offline-first sync	Plants have dead zones; only glasses inspections queue today	Android outbox with idempotent replay for acknowledgements and checklist ticks (FR-098)	Q2 2027	3 person-months
Predictive maintenance	Pumps and blowers fail with warning signs nobody watches	Per-asset baselines flag drift and create advisory maintenance tasks with evidence	Later	6 person-months
SOP copilot (RAG)	Operators ask "how" and "why" questions that need long procedures	Answers from approved documents only, with citations, never inventing a step	Q4 2027	3 person-months
Multilingual voice	Operators speak Hindi, Tamil, Marathi, Gujarati	Same grounding guarantees in each language (FR-073): Hindi first, then Tamil and Marathi	Q3 2027 onwards	4 person-months
Gauge reading by computer vision	Analogue gauges are read by eye and copied	"Read this": Hydris proposes a value with confidence, the operator confirms, the photo is kept	Q4 2027	5 person-months

Product	Problem it solves	Outcome	When (plan)	Effort
Compliance reporting and OCEMS	Plants assemble CPCB and SPCB returns by hand	Evidence assembled with provenance, exceedances and data availability, draft returns for e-signed approval (FR-120)	Q3 to Q4 2027	4 person-months
E-signature	Approval has no signature manifestation	Re-authentication at signing, stated meaning, name and time bound to the record (FR-068)	Q3 2027	2 person-months
Display glasses	Ray-Ban Meta has no display, so operators look down at the phone	The existing heads-up layout renders on display glasses and industrial headsets behind a display adapter	Later	3 to 5 person-months per target
iOS app	Some supervisors and managers carry iPhones	Supervisor companion first (alarms, reviews, approvals)	Later	6 person-months
CMMS integrations	Hydris tasks would duplicate SAP PM or IBM Maximo work orders	Two-way sync of work orders and assets, plus webhooks for LIMS, ERP and ticketing	Q4 2027 onwards	4 person-months for the first connector

Edge gateway and the compliance pipeline

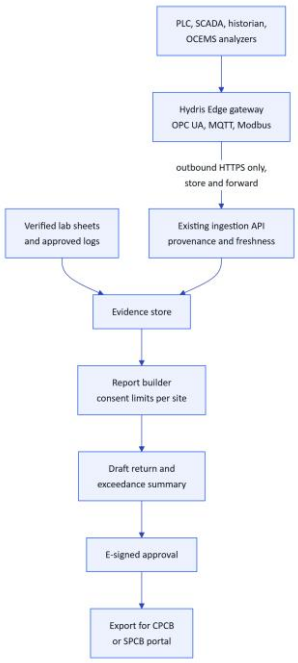


Figure 95. *How the edge gateway and compliance reporting extend today's ingestion (planned)*

The gateway reuses the single ingestion path, so readings from industrial protocols carry the same guarantees as today's HTTPS batches. Compliance reporting then builds on verified scans and e-signatures; direct submission to regulator systems happens only where an official interface exists.

Display glasses and new inputs

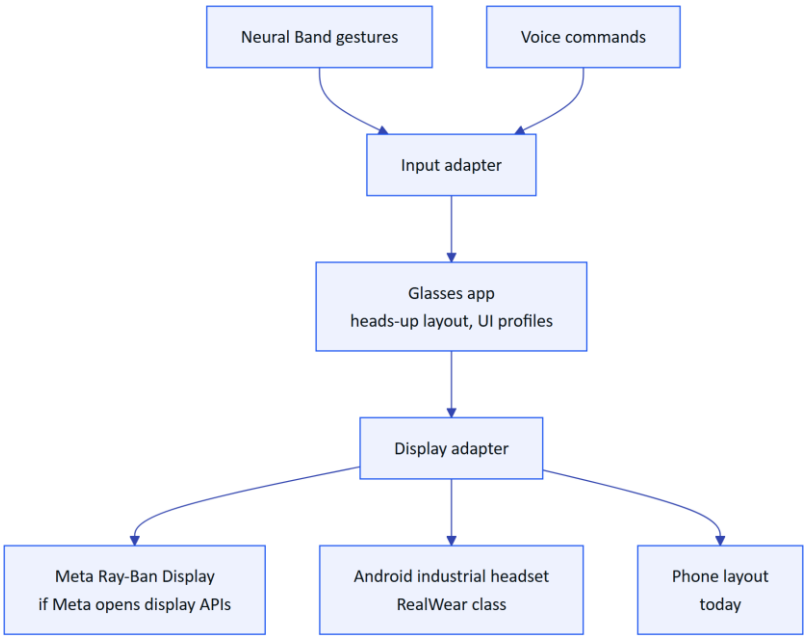


Figure 96. *Display adapter for future display glasses and headsets (planned)*

The heads-up layout and input abstraction already exist; display targets plug in behind adapters. The dependency is Meta exposing display and Neural Band input to third parties, which is not available today.

11.6 Value versus effort

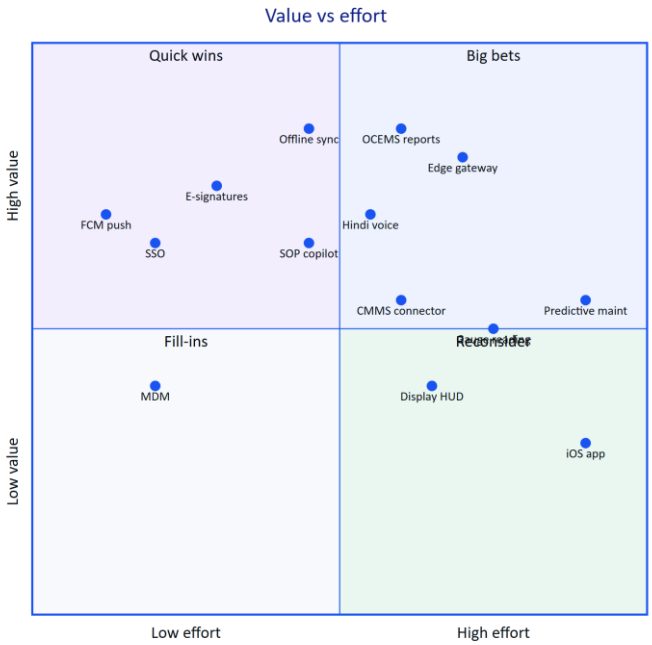


Figure 97. *Value versus effort for candidate epics (team estimates)*

Quick wins (push, SSO, e-signatures) land first because they unblock buying. Offline sync, OCEMS reports and the edge gateway are the big bets that make Hydris a system of record. iOS and predictive maintenance wait for evidence of demand and enough labelled history. A RICE score backs this ordering: push notifications score highest (270), then SSO (180) and offline sync (160); predictive maintenance scores lowest (5).

RISK

The biggest risks are a small team spread across too many surfaces, pilot customer delays (approvals, camera policy) and Meta DAT publishing staying gated for India-based developers. Mitigations: strict horizon discipline (Later items stay unstarted until GA), a site camera-policy pack and a second pilot candidate, and the phone-camera path, which works without DAT.

11.7 Success metrics per horizon

Every figure below is a target.

Table 82. Success metrics per horizon (targets)

Horizon	Metric	Target
Now	Pilot exit criteria complete	100 %
Now	Rounds completed on time at the pilot	95 %
Now	Median alarm acknowledgement, critical and high	Under 5 minutes
Now	Audit fields where independent engines agree	90 %
Next	Weekly active operators over licensed operators	70 %
Next	Paying sites	3
Next	Offline actions synced without duplicates	100 %
Next	SOC 2 Type I	Report issued
Later	Compliance reports generated in Hydris	Accepted by 1 regulator
Later	Copilot answers with citations	90 % of answered questions
Later	Multi-site customers	2

11.8 Release cadence

API and web ship continuously to staging and weekly to production after the gates, with S1 hotfixes within 24 hours. The Android app ships every two weeks through Play internal, closed and managed tracks, with glasses features behind remote flags. The edge gateway will ship monthly as signed container images inside a customer-controlled upgrade window, and this book is updated with each release.